

Оглавление

Введение	4
Глава 1. Теоретическая часть	6
1.1. Временные ряды и постановка задачи	6
1.1.1. Введение	6
1.1.2. Постановка задачи	7
1.1.3. Метрики качества прогноза	8
1.2. Классические модели временных рядов	9
1.2.1. Модели AR, MA, ARIMA, SARIMA	9
1.2.2. ACF, PACF и выбор порядка модели	12
1.2.3. Модели экспоненциального сглаживания и ETS	13
1.2.4. Модель Prophet	15
1.3. Вейвлет-разложения и преобразование MODWT	17
1.3.1. Идея вейвлет-преобразований	17
1.3.2. DWT: дискретное вейвлет преобразование	18
1.3.3. MODWT: максимальное перекрывающееся дискретное вейвлет-преобразование	20
1.3.4. Сравнение методов декомпозиции DWT, MODWT, EWT и VMD	21
1.4. Нейросетевые модели для временных рядов	23
1.4.1. Модель TimesNet	24
1.4.2. Модель N-BEATS	29
1.5. Гибридные модели на основе декомпозиции временных рядов	34
1.5.1. Декомпозиция на компоненты с помощью MODWT	35
1.5.2. Концепция модели HybridPlus	36
Глава 2. Практическая часть	38
2.1. Описание параметров и архитектуры вейвлета + моделей с практической точки зрения	38
2.1.1. ARIMA - один статистический блок ARIMA(p,d,q) на исходном ряде	38
2.1.2. ARIMA (с автоподбором) - ARIMA-блок, но порядок (p,q) выбирается эвристикой по ACF/PACF	38
2.1.3. ETS	39
2.1.4. Prophet с аддитивной моделью и без встроенных календарных сезонностей	39
2.1.5. TimesNet+ (MODWT + TimesNet)	39
2.1.6. N-Beats+ (MODWT + N-Beats)	40
2.1.7. VW + TimesNet + ARIMA (с автоподбором)	41
2.1.8. VW + TimesNet + ETS	41
2.1.9. VW + N-Beats + ARIMA (с автоподбором)	42

2.1.10. VW + N-Beats + ETS	42
2.2. Применение и диагностика моделей на синтетических рядах	42
2.2.1. Используемые профили (ряды для испытаний)	42
2.2.2. Обучение моделей, подбор параметров ARIMA.....	47
2.2.3. Результаты обучения моделей на синтетических рядах.....	54
2.3. Тестирование моделей на наборе данных M3.....	67
2.3.1. Описание набора данных.....	67
2.3.2. Обучение на M3	67
2.3.3. Результаты обучения и оценка предсказаний на наборе данных M3	70
2.4. Тестирование моделей на наборе данных M4.....	77
2.4.1. Описание набора данных.....	77
2.4.2. Симуляция rollout на quarterly рядах	77
2.4.3. Результаты обучения и оценка предсказаний на наборе данных M4	83
2.5. Тестирование моделей на рядах со сложной сезонностью	94
2.5.1. Описание набора данных.....	94
2.5.2. Рассмотрение эвристики краткосрочного прогнозирования на компонентах MODWT в условиях многопериодной сезонности	94
2.5.3. Результаты обучения на рядах со сложной сезонностью	97
Заключение.....	99
Список литературы	101
Приложение.....	105

Введение

Краткосрочное прогнозирование временных рядов используется в задачах планирования и управления (продажи, трафик, потребление ресурсов, цены и др.), где важно получать устойчивые прогнозы на ограниченной истории и при наличии шума, сезонностей и структурных сдвигов. При этом результаты крупных исследований показывают, что **универсальной модели, одинаково хорошо работающей на всех типах рядов, не существует**: одни методы выигрывают на рядах с выраженной трендово-сезонной структурой, другие — на более шумных или нестабильных процессах. Это делает актуальными подходы, которые сочетают сильные стороны статистических и нейросетевых моделей и повышают стабильность качества на разнородных данных.

Классические модели (ARIMA/SARIMA, ETS, Prophet) дают интерпретируемые и часто устойчивые результаты на небольших выборках, но ограничены линейностью/предположениями о структуре данных. Современные нейросетевые архитектуры, такие как TimesNet и N-BEATS, способны извлекать более сложные зависимости и показывают высокие результаты на стандартных наборах, однако могут быть чувствительны к шуму и особенностям окна истории. Поэтому в последние годы активно развиваются **гибридные схемы**, где декомпозиция ряда и комбинирование разных моделей позволяет улучшить устойчивость и качество прогноза.

Цель работы — разработать и исследовать гибридный подход к краткосрочному прогнозированию **унивариативных** временных рядов, основанный на многошкальной декомпозиции и раздельном моделировании компонент с последующей устойчивой реконструкцией прогноза.

Для достижения цели решаются следующие задачи:

1. Формализовать постановку задачи краткосрочного прогноза и метрики оценки качества.
2. Рассмотреть и реализовать базовые статистические модели (ARIMA/ETS/Prophet) и нейросетевые модели (TimesNet, N-BEATS) как сравниваемые подходы.
3. Обосновать применение вейвлет-декомпозиции для раздельного моделирования временных масштабов и выбрать MODWT как инструмент разложения, удобный для гибридизации.
4. Построить гибридную архитектуру на базе MODWT с отдельными моделями для низко- и высокочастотных компонент и схемой агрегации прогнозов.
5. Провести экспериментальную оценку на синтетических профилях и на данных M3/M4, сравнив гибриды с базовыми моделями по выбранным метрикам.

Объект исследования – методы прогнозирования временных рядов.

Предмет исследования – гибридные модели краткосрочного прогнозирования, основанные на декомпозиции ряда и раздельном прогнозировании компонент.

Актуальность работы определяется практической потребностью в методах, которые:

- дают устойчивый прогноз на разнородных рядах без универсального предположения о структуре данных;
- не усиливают высокочастотный шум на горизонте;
- контролируют артефакты на границе последнего наблюдения и первого шага прогноза, которые часто ухудшают метрики в прикладных сценариях.

Новизна работы состоит в практической реализации и исследовании гибридного конвейера *HybridPlus*, где:

- временной ряд раскладывается по масштабам с помощью **MODWT** и далее прогнозируется отдельно по компонентам с последующей реконструкцией;
- низкочастотная компонента прогнозируется специализированными нейросетевыми моделями **TimesNet/N-BEATS**, а высокочастотные детали — более простыми/устойчивыми моделями (ARIMA/ETS), чтобы не разгонять шум;
- в реализации зафиксированы инженерные меры устойчивости, важные именно для краткосрочного прогноза: обработка границ разложения, стабилизация первого шага (ограничение скачка), а также управление вкладом высокочастотных компонент при агрегации прогнозов.

Глава 1. Теоретическая часть

1.1. Временные ряды и постановка задачи

1.1.1. Введение

Временной ряд — это упорядоченная во времени последовательность наблюдений некоторой случайной величины

$$\{y_t\}_{t=1}^T$$

где y_t - значение показателя в момент времени t , а шаг дискретизации (день, месяц, квартал и т.п.) считается фиксированным - такое представление как реализации стохастического процесса является стандартным в классической теории временных рядов [1, 2].

Для описания структуры ряда обычно выделяют несколько компонент [2]:

- **уровень (*baseline*)** - средний уровень показателя;
- **тренд (*trend*)** - медленно меняющееся во времени направление изменения уровня (рост, падение);
- **сезонная составляющая (*seasonality*)** - периодические колебания с фиксированным или почти фиксированным периодом (день недели, месяц, квартал);
- **циклическая составляющая (*cycle*)** - долговременные колебания с непостоянным периодом (экономические циклы и т.п.);
- **случайная составляющая (*noise, irregular*)** - шум, не объясняемый регулярными компонентами.

Во многих задачах удобно рассматривать **аддитивную модель** временного ряда

$$y_t = \ell_t + tr_t + s_t + \varepsilon_t,$$

где ℓ_t - уровень, tr_t - тренд, s_t - сезонность, ε_t - случайная компонента.

В некоторых приложениях используется мультипликативное представление ($y_t = \ell_t \cdot tr_t \cdot s_t \cdot \varepsilon_t$), если амплитуда колебаний растёт с уровнем ряда [1, 2].

Стационарность - одно из ключевых свойств временного ряда, означающее, что его основные статистические характеристики (среднее значение, дисперсия, автокорреляция) не зависят от конкретного момента времени. Именно поэтому предположение о стационарности играет центральную роль в классических моделях, включая семейство ARIMA [1, 2].

- Временной ряд называется **строго стационарным**, если его совокупное распределение не меняется при любом сдвиге по времени. Формально, совместное распределение вектора $(y_{t_1}, \dots, y_{t_k})$ совпадает с распределением $(y_{t_1+h}, \dots, y_{t_k+h})$ для любых моментов времени $t_1, \dots, t_k$ и любого целого сдвига h . Из этого, в частности, следует, что математическое ожидание и дисперсия не зависят от времени:

$$\mathbb{E}[y_t] = \mu < \infty, \quad \text{Var}(y_t) = \gamma(0)$$

а также ковариация между y_t и $y_{t+\tau}$ зависит только от лага τ [2].

- **Слабая стационарность** определяет стационарный ряд только через моменты второго порядка. Ряд называется слабо стационарным, если

$$\mathbb{E}[y_t] = \mu = \text{const}, \quad \text{Var}(y_t) = \sigma^2 = \text{const},$$

а ковариация

$$\text{Cov}(y_t, y_{t+\tau}) = \gamma(\tau)$$

зависит только от лага τ , но не от самого времени t [2].

Вводя **автокорреляционную функцию**:

$$\rho(\tau) = \frac{\gamma(\tau)}{\gamma(0)},$$

можно сказать, что зависимость между наблюдениями ряда характеризуется именно функцией $\rho(\tau)$ по лагу, что используется при анализе и идентификации моделей (в частности, при построении ARIMA) [1].

Для широкого класса стационарных процессов дополнительно выполняется условие

$$\lim_{\tau \rightarrow \infty} \rho(\tau) = 0,$$

то есть автокорреляция стремится к нулю по мере роста лага τ , то есть удалённые наблюдения становятся практически некоррелированными, тогда как на малых лагах могут сохраняться сильные зависимости, полезные для краткосрочного прогноза [2].

Практическая значимость такого понятийного аппарата и необходимость сопоставления методов хорошо видны на крупных бенчмарках по прогнозированию (например, M3/M4), где сравниваются статистические и нейросетевые подходы на большом числе разнородных рядов [3, 4]. В данной работе эти определения служат базисом для выбора моделей и построения гибридов, поэтому вместо «ручной» аддитивной декомпозиции вида тренд+сезонность+остаток далее применяется многошкальная декомпозиция MODWT и раздельное прогнозирование компонент с последующей сборкой прогноза (см. гл. 2) [5 - 8].

1.1.2. Постановка задачи

В настоящей работе рассматривается задача **краткосрочного прогноза** унивариативных временных рядов (последовательность наблюдений *одной* случайной величины, упорядоченных во времени).

Пусть задана последовательность наблюдений

$$\{y_t\}_{t=1}^T$$

Выбирается длина окна истории (L) и горизонт прогноза (H), что соответствует стандартной постановке задачи прогнозирования временных рядов в классической литературе и прикладных протоколах сравнения методов [2 - 4].

На шаге времени t модель получает на вход вектор последних (L) наблюдений:

$$\mathbf{y}_t^{(\text{in})} = (\mathbf{y}_{t-L+1}, \dots, \mathbf{y}_t),$$

и должна предсказать будущие значения на горизонте H

$$\hat{\mathbf{y}}_t^{(\text{in})} = (\hat{\mathbf{y}}_{t+1}, \dots, \hat{\mathbf{y}}_{t+H}).$$

Цель - построить такую модель f , что

$$(\hat{\mathbf{y}}_{t+1}, \dots, \hat{\mathbf{y}}_{t+H}) = f(\mathbf{y}_{t-L+1}, \dots, \mathbf{y}_t)$$

минимизирует выбранную функцию ошибки на тестовых данных [3, 4].

1.1.3. Метрики качества прогноза

Для количественной оценки качества прогнозов широко применяются **ошибки масштаба** и **относительные ошибки**, а также их модификации. Сравнительный разбор свойств таких метрик (чувствительность к выбросам, проблемы процентных ошибок при $y_i \approx 0$, сопоставимость на разных рядах) приведён в классическом обзоре мер точности прогнозов [9].

В работе используются следующие метрики:

- **MSE (mean squared error)** - среднеквадратичная ошибка

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Она **сильно штрафует крупные промахи** за счёт квадрата ошибки, поэтому чувствительна к выбросам, что типично отмечается при обсуждении квадратичных потерь.

- **RMSE (root mean squared error)** - корень из среднеквадратичной ошибки

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

RMSE имеет **тот же масштаб**, что и исходный ряд, что облегчает интерпретацию по сравнению с MSE.

- **MAPE (mean absolute percentage error)** - средняя абсолютная процентная ошибка

$$MAPE = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Эта метрика показывает среднюю относительную ошибку в процентах, но плохо определена при значениях ($y_i \approx 0$) и может давать нестабильные значения на рядах с малыми уровнями [9].

- **sMAPE (symmetric mean absolute percentage error)** - симметричная средняя абсолютная процентная ошибка

$$sMAPE = \frac{100\%}{N} \sum_{i=1}^N \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

В отличие от MAPE, sMAPE симметрична по отношению к фактическим и прогнозным значениям и менее чувствительна к нулевым значениям ряда. На практике часто используется в бенчмарках по прогнозированию (в частности, в протоколах МЗ, М4) как одна из основных метрик сравнения методов [3, 4, 9].

1.2. Классические модели временных рядов

1.2.1. Модели AR, MA, ARIMA, SARIMA

Модель скользящего среднего порядка q ($MA(q)$) задаётся формулой

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q},$$

где

- μ - постоянный средний уровень ряда,
- ε_t - белый шум, т.е. последовательность независимых случайных величин с нулевым средним и дисперсией σ^2 ,
- $\theta_1, \dots, \theta_q$ - параметры модели.

Такое задание МА-моделей и трактовка ε_t как инноваций является стандартной в классической литературе по анализу временных рядов [1, 2].

Интерпретация МА(q) состоит в том, что текущее значение ряда представляется как сумма фиксированного среднего и линейной комбинации текущей и нескольких прошлых случайных ошибок. Несмотря на простоту, этот класс моделей достаточно богат, согласно теореме Вольда, любой (слабо) стационарный процесс допускает представление в виде МА-модели с бесконечным числом лагов [1, 2].

Авторегрессионная модель порядка p ($AR(p)$) описывается выражением

$$y_t = c + \varphi_1 y_{t-1} + \dots + \varphi_p y_{t-p} + \varepsilon_t,$$

где

- c - константа,
- $\varphi_1, \dots, \varphi_p$ - авторегрессионные коэффициенты,
- ε_t - белый шум.

Это базовое определение AR-моделей принято в классических работах по временным рядам [1, 2]. Таким образом, текущее значение ряда выражается через несколько предыдущих значений и случайный шум, а AR(p) можно также рассматривать как **линейную регрессию**, где объясняемая переменная - y_t , а регрессоры - лаги $y_{t-1}, \dots, y_{t-p}$.

Удобно записывать модель с помощью лагового оператора L , действующего как

$$Ly_t = y_{t-1},$$

тогда

$$\varphi(L)y_t = c + \varepsilon_t,$$

где

$$\varphi(L) = 1 - \varphi_1 L - \dots - \varphi_p L^p$$

— авторегрессионный полином; лаговая запись и полиномиальная форма широко используются в стандартных изложениях AR/MA-моделей [1, 2].

Стационарность AR(p)-процесса связана с корнями этого полинома: модель задаёт стационарный ряд, если все корни уравнения $\varphi(z) = 0$ лежат **вне единичного круга** (их модули больше 1) [1, 2].

В этом случае можно показать, что AR(p) представим как бесконечная MA-модель:

$$y_t = \mu + \sum_{j=0}^{\infty} \psi_j \varepsilon_{t-j},$$

что подчёркивает связь между классами AR и MA [2].

Модель ARMA(p, q) сочетает авторегрессионную часть и часть скользящего среднего:

$$y_t = c + \varphi_1 y_{t-1} + \dots + \varphi_p y_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}.$$

а в лаговой записи:

$$\varphi(L)y_t = c + \theta(L)\varepsilon_t,$$

где

$$\varphi(L) = 1 - \varphi_1 L - \dots - \varphi_p L^p, \quad \theta(L) = 1 + \theta_1 L + \dots + \theta_q L^q.$$

Такая запись является канонической для ARMA-моделей [1, 2], при этом выполняются 2 условия:

- **Условие стационарности ARMA-части:** все корни $\varphi(z) = 0$ находятся вне единичного круга [1, 2].
- **Условие обратимости MA-части:** корни $\theta(z) = 0$ также лежат вне единичного круга, что позволяет представить модель в виде авторегрессии бесконечного порядка [1, 2].

Модели ARMA используются для описания уже **стационарных** временных рядов. Если исходный ряд содержит тренд или сезонность, сначала требуется привести его к стационарному виду [1, 2].

Для приведения нестационарных рядов к стационарным используется модель **ARIMA(p, d, q)** - «интегрированная ARMA», где сначала ряд **дифференцируют** d раз, а уже затем к полученному стационарному ряду применяют $ARMA(p, q)$ [1, 2].

В лаговой форме:

$$\varphi(L)(1-L)^d y_t = c + \theta(L)\varepsilon_t,$$

где оператор $(1-L)^d$ реализует дифференцирование порядка d :

- при $d = 1$: $\Delta y_t = y_t - y_{t-1}$,
- при $d = 2$: $\Delta^2 y_t = \Delta(\Delta y_t)$, и т.д.

Таким образом, **ARIMA(p, d, q)** — это $ARMA(p, q)$ - модель, построенная не для исходного ряда, а для его d -й разности. При корректном выборе d полученный ряд становится стационарным, что оправдывает использование линейной модели [1, 2].

На практике выбор порядков (p, d, q) часто автоматизируют (в том числе через информационные критерии и поисковые процедуры), что отражено в популярной реализации *auto.arima* [1, 10].

Во многих приложениях (продажи, потребление энергии, трафик) ярко выражена **сезонность**: поведение ряда повторяется с периодом s (например, 12 месяцев или 7 дней), поэтому используют расширение ARIMA - **сезонную модель SARIMA** (стандартная конструкция в подходе Box–Jenkins) [1]:

$$SARIMA(p, d, q) \times (P, D, Q)_s.$$

Здесь:

- (p, d, q) - параметры несезонной части (обычный ARIMA),
- (P, D, Q) - параметры сезонной авторегрессии, дифференцирования и скользящего среднего,
- s - длина сезонного периода.

В лаговой записи **SARIMA**-модель можно представить схематично:

$$\varphi(L) \Phi(L^s) (1-L)^d (1-L^s)^D y_t = c + \theta(L) \Theta(L^s) \varepsilon_t,$$

где $\Phi(L^s)$ и $\Theta(L^s)$ - полиномы, содержащие лаги кратные s (сезонные лага) [1, 2].

Такая модель одновременно [1]:

- устраняет долгосрочный тренд за счёт несезонного дифференцирования порядка d ,
- сглаживает сезонные эффекты за счёт сезонного дифференцирования порядка D ,
- учитывает как краткосрочные, так и сезонные зависимости через авторегрессионные и МА-компоненты.

При отсутствии сезонности ($P = Q = D = 0$) модель *SARIMA* сводится к обычной *ARIMA*(p, d, q) [1, 2].

1.2.2. ACF, PACF и выбор порядка модели

При анализе стационарных временных рядов важную роль играют автоковариационная функция и производная от неё автокорреляционная функция (ACF). Именно ACF является базовым инструментом диагностики линейной зависимости и используется в процедуре идентификации моделей семейства Box–Jenkins [1, 2]. На основе ACF строится и частичная автокорреляционная функция (PACF), позволяющая выявлять порядок авторегрессионной части AR(p) и, в целом, служащая стандартным элементом первичного анализа при построении ARMA/ARIMA-моделей [1, 2].

Рассмотрим слабостационарный временной ряд $\{y_t\}$ с мат. ожиданием μ и дисперсией σ^2 , не зависящими от времени.

Автоковариация на лаге k определяется как

$$\gamma(k) = \text{Cov}(y_t, y_{t+k}) = \mathbb{E}[(y_t - \mu)(y_{t+k} - \mu)].$$

Из стационарности следует, что $\gamma(k)$ зависит только от лага k , а не от конкретного момента времени t . При $k = 0$ получаем дисперсию: $\gamma(0) = \text{Var}(y_t) = \sigma^2$ [2].

Автокорреляционная функция (ACF) нормирует автоковариацию на дисперсию:

$$\rho(k) = \frac{\gamma(k)}{\gamma(0)} = \frac{\text{Cov}(y_t, y_{t+k})}{\text{Var}(y_t)}.$$

По определению $|\rho(k)| \leq 1$, $\rho(0) = 1$. Значения $\rho(k)$ показывают степень линейной связи между наблюдениями, разделёнными лагом k , что непосредственно используется при анализе структуры зависимостей в AR/MA-компонентах [1, 2].

На практике работают с конечной выборкой $y_1, \dots, y_T$. Тогда выборочное мат. ожидание и выборочная автоковариация задаются как

$$\hat{\mu} = \frac{1}{T} \sum_{t=1}^T y_t,$$

$$\hat{\gamma}(k) = \frac{1}{T} \sum_{t=1}^{T-k} (y_t - \hat{\mu})(y_{t+k} - \hat{\mu}), k = 0, 1, \dots,$$

а выборочная автокорреляция - как

$$\hat{\rho}(k) = \frac{\hat{\gamma}(k)}{\hat{\gamma}(0)}.$$

График значений $\hat{\rho}(k)$ по лагам k называется **коррелограммой** и является основным визуальным инструментом идентификации структуры зависимости, применяемым в Box–Jenkins-подходе и во многих практических процедурах подбора порядка ARMA/ARIMA [1, 2], в частности, эвристики на основе ACF/PACF часто служат стартовой точкой даже

для автоматизированного подбора порядка (например, в *auto.arima*), где далее используются критерии качества и поиск по кандидатам [10].

Для большого числа наблюдений T при истинной автокорреляции $\rho(k) = 0$ можно использовать приблизительные границы значимости

$$\hat{\rho}(k) \approx \mathcal{N}\left(0, \frac{1}{T}\right),$$

поэтому интервал $\pm \frac{1,96}{\sqrt{T}}$ часто применяют как ориентировочную область случайных колебаний вокруг нуля на коррелограмме при визуальной проверке значимости лагов [2].

Частичная автокорреляционная функция (PACF) показывает связь между y_t и y_{t-k} после исключения влияния промежуточных лагов $y_{t-1}, \dots, y_{t-k+1}$ и поэтому удобна для определения порядка авторегрессии [1, 2]. Формально частичная автокорреляция α_k определяется как коэффициент при y_{t-k} в линейной регрессии:

$$y_t = \beta_1^{(k)} y_{t-1} + \dots + \beta_k^{(k)} y_{t-k} + u_t,$$

где $\beta_j^{(k)}$ - коэффициенты линейной регрессии, а u_t - ошибка. Тогда $\alpha_k = \beta_k^{(k)}$, нормированный так, чтобы он интерпретировался, как корреляция между остатками:

$$\alpha_k = \text{Corr}(r_t^{(1)}, r_t^{(2)}),$$

где $r_t^{(1)}$ - остатки регрессии y_t на $(y_{t-1}, \dots, y_{t-k+1})$, а $r_t^{(2)}$ - остатки регрессии y_{t-k} на те же самые промежуточные лаги. Таким образом, α_k измеряет чистый эффект лагов порядка k , не объяснённый более короткими лагами [2].

Для стационарного $AR(p)$ процесса PACF имеет характерное свойство: **значимые значения α_k наблюдаются до лага p , а дальше PACF близка к нулю**, что используется как эвристика идентификации порядка p в подходе Box–Jenkins [1, 2]. На практике PACF вычисляют рекурсивно по уравнениям Юла–Уокера с помощью алгоритма Дербина–Левинсона [2], а полученные оценки применяют совместно с ACF и критериями качества при настройке ARIMA (в том числе в автоматизированных процедурах) [10].

1.2.3. Модели экспоненциального сглаживания и ETS

Класс экспоненциального сглаживания относится к методам прогнозирования, в которых состояние ряда обновляется рекурсивно, а вклад более старых наблюдений затухает экспоненциально. Такие модели широко применяются для краткосрочного прогноза благодаря устойчивости к шуму и возможности явно учитывать уровень, тренд и сезонность [2, 11]. Исторически базовые схемы экспоненциального сглаживания развивались от моделей **Хольта** (уровень + тренд) и **Винтерса** (уровень + тренд + сезонность), которые до сих пор лежат в основе практических методов прогнозирования [12, 13].

Самый простой вариант экспоненциального сглаживания - **простое экспоненциальное сглаживание (Simple Exponential Smoothing, SES)**, применимое к рядам без выраженного тренда и сезонности.

Пусть ℓ_t - оценка уровня (*level*) в момент t , тогда обновление уровня задаётся рекурсией [11]:

$$\ell_t = \alpha y_t + (1 - \alpha)\ell_{t-1}, 0 < \alpha < 1,$$

а прогноз на один шаг вперёд равен

$$\hat{y}_{t+1|t} = \ell_t.$$

Из этой формулы следует, что ℓ_t является взвешенной суммой всех прошлых наблюдений с весами вида $\alpha(1 - \alpha)^j$, поэтому вклад наблюдений убывает геометрически по мере удаления в прошлое [11].

Если ряд содержит тренд, SES обычно недостаточно. В этом случае используется **двойное экспоненциальное сглаживание (Holt's linear method)**, в котором поддерживаются две скрытые компоненты:

- ℓ_t - уровень,
- b_t - тренд (наклон).

Типичная аддитивная форма обновления [11, 12]:

$$\begin{aligned}\ell_t &= \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}), \\ b_t &= \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}, 0 < \beta < 1,\end{aligned}$$

а прогноз на h шагов вперёд:

$$\hat{y}_{t+h|t} = \ell_t + h b_t.$$

Для случаев, когда тренд со временем ослабевает, часто используют **затухающий тренд**, вводя коэффициент $\phi \in (0,1)$.

$$\hat{y}_{t+h|t} = \ell_t + (\phi + \phi^2 + \dots + \phi^h) b_t.$$

Такие модификации рассматриваются в рамках унифицированного представления экспоненциального сглаживания [2, 11], которые уменьшают риск разгона прогноза на более дальнем горизонте.

При наличии сезонности вводится сезонная компонента s_t с периодом m (например, $m = 12$ для месячной сезонности). В наиболее распространённой **аддитивной сезонной модели (Holt–Winters additive)** обновления имеют вид [11, 13]:

$$\begin{aligned}\ell_t &= \alpha(y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}), \\ b_t &= \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}, \\ s_t &= \gamma(y_t - \ell_t) + (1 - \gamma)s_{t-m}, 0 < \gamma < 1,\end{aligned}$$

а прогноз:

$$\hat{y}_{t+h|t} = \ell_t + h b_t + s_{t+h-m \cdot \lceil \frac{h}{m} \rceil}$$

Экспоненциальное сглаживание удобно рассматривать в формализованном виде **ETS (Error – Trend – Seasonal)**, где каждая модель описывается тройкой:

- **E (Error)** - тип ошибки: аддитивная (A) или мультипликативная (M);
- **T (Trend)** - отсутствует (N), аддитивный (A), затухающий (Ad);
- **S (Seasonal)** - отсутствует (N), аддитивная (A), мультипликативная (M).

Такая классификация и её автоматизированное использование основаны на том, что у ряда есть скрытое внутреннее состояние, предложенное Hyndman и соавторами, где модели задаются уравнениями наблюдений и состояния [11]. Например:

- **ETS(A, N, N) = SES;**
- **ETS(A, A, N)** - соответствует методу Холта (линейный тренд);
- **ETS(A, A, A)** - соответствует аддитивному Холту-Уинтерсу.

Главная идея ETS - описывать ряд через скрытые состояния (уровень, тренд, сезонность) и задавать для них **состояния-уравнения** и **уравнение наблюдений**.

В **аддитивной постановке** типично:

$$y_t = \hat{y}_{t|t-1} + \varepsilon_t,$$

где $\hat{y}_{t|t-1}$ выражается через ℓ_{t-1} , b_{t-1} , s_{t-m} , а затем компоненты обновляются через сглаживающие коэффициенты.

В **мультипликативной постановке** ошибка входит как множитель:

$$y_t = \hat{y}_{t|t-1} (1 + \varepsilon_t),$$

что естественно для рядов, где вариативность растёт вместе с уровнем [2, 11].

1.2.4. Модель Prophet

Prophet - прикладной метод прогнозирования временных рядов, ориентированный на бизнес-метрики (продажи, трафик, заявки), где часто присутствуют тренд, несколько сезонностей и календарные эффекты. Модель была предложена Taylor и Letham как подход «прогнозирование в масштабе», когда нужно строить и поддерживать прогнозы сразу для очень большого числа рядов (сотни/тысячи/миллионы: товары, регионы, каналы, метрики) быстро, автоматически и стабильно, а строится как **аддитивная регрессия по времени** [14]:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t,$$

где

- $g(t)$ - тренд,
- $s(t)$ - сезонные компоненты,

- $h(t)$ - эффекты праздников/событий,
- ε_t - ошибка.

Тренд $g(t)$: кусочно-линейная и логистическая формы

- **Кусочно-линейный тренд.**

Выбирается набор моментов времени $\{s_j\}_{j=1}^J$ - *changepoints* (точки потенциального изменения тренда).

Вводится индикаторный вектор

$$a(t) = (a_1(t), \dots, a_J(t)), \quad a_j(t) = \mathbb{I}(t \geq s_j),$$

и задаётся тренд вида [14]:

$$g(t) = (k + a(t)^\top \delta) t + (m + a(t)^\top \gamma),$$

где

- k - базовый наклон,
- m - базовый уровень,
- δ - изменения наклона после *changepoints*.

Параметры γ подбираются так, чтобы функция была непрерывной в точках s_j [14].

- **Логистический тренд.**

Если ряд ограничен сверху (например, ёмкость рынка), используют насыщаемый рост с $C(t)$ [14]:

$$g(t) = \frac{C(t)}{1 + \exp(-r(t)(t - t_0))},$$

где $r(t)$ может изменяться по тем же *changepoints* (аналогично кусочно-линейному случаю). Это позволяет моделировать S-образную динамику [14].

Сезонность $s(t)$: разложение в ряд Фурье

Prophet описывает сезонные колебания с периодом P (например, неделя или год) через тригонометрический базис (ряд Фурье) [14]:

$$s_P(t) = \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right) \right).$$

Здесь N - число гармоник, определяющее «гибкость» сезонности. Увеличение N позволяет описывать более сложную форму сезонной кривой [14].

Обычно, суммарная сезонность — это сумма нескольких компонент [14]:

$$s(t) = s_{\text{week}}(t) + s_{\text{year}}(t) + s_{\text{day}}(t) + \dots$$

Праздники и события $h(t)$: регрессия по календарю

Эффекты праздников/особых дат задаются как линейная регрессия по бинарным (или категориальным) признакам [14]:

$$h(t) = z(t)^T \kappa,$$

где

- $z(t)$ - вектор индикаторов («сегодня праздник X», «день до праздника», «день после праздника» и т.п.),
- κ - оцениваемые коэффициенты эффектов.

Это делает компоненту $h(t)$ интерпретируемой: знак и величина κ показывают влияние события на целевую метрику [14].

1.3. Вейвлет-разложения и преобразование MODWT

1.3.1. Идея вейвлет-преобразований

Традиционный инструмент частотного анализа сигналов - **преобразование Фурье**, в котором сигнал представляется, как линейная комбинация синусоид:

$$x(t) = \int_{-\infty}^{\infty} X(\omega) e^{i\omega t} d\omega.$$

Такое представление хорошо выявляет, *какие частоты присутствуют в сигнале*, но почти *не отвечает* на вопрос, *когда именно они проявляются*: синусоида бесконечна по времени, поэтому локальные особенности описываются менее эффективно, что и является одной из мотиваций вейвлет-анализа в классических работах по мультиразрешающему разложению [15, 16].

Вейвлет-подход решает эту проблему, используя базисные функции, которые одновременно локализованы **во времени и по частоте**. Фундаментальные идеи мультиразрешающего анализа (MRA) и вейвлет-представления сигналов были систематизированы Mallat и далее развиты в классической монографии Daubechies, где вместо глобальных синусоид берутся короткие волны — вейвлеты [15, 16].

Для фиксированного материнского вейвлета $\psi(t)$ строится семейство функций, получаемых масштабированием и сдвигом [15, 16]:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right),$$

где

- a - **масштаб** (при большом a функция растягивается и описывает низкие частоты; при малом a - сжимается и описывает высокие частоты),
- b - **временной сдвиг**, отвечающий за положение окна во времени [15].

Разложение сигнала по $\psi_{a,b}(t)$ позволяет одновременно видеть структуру на разных масштабах (от медленных компонент до быстрых колебаний) и локализацию этих компонент во времени, что делает вейвлет-методы особенно удобными для анализа и обработки временных рядов, включая декомпозиции и фильтрацию на разных частотных уровнях [5].

1.3.2. DWT: дискретное вейвлет преобразование

Рассмотрим дискретный временной ряд $\{y_t\}_{t=0}^{T-1}$.

В вейвлет-преобразованиях используются **КИХ-фильтры** - цифровые фильтры с *конечной импульсной характеристикой*, то есть если на вход подать «единичный импульс» (единицу в одном отсчёте и нули в остальных), то реакция фильтра будет длиться **только конечное число шагов**, а затем станет нулевой, поэтому такой фильтр можно полностью задать конечным набором коэффициентов [15, 16].

Пусть на вход подаётся сигнал (ряд) $\{x_t\}$, а фильтр задан коэффициентами $b_0, b_1, \dots, b_{M-1}$, тогда выход фильтра $\{z_t\}$ вычисляется как свертка [15, 16]:

$$z_t = \sum_{\ell=0}^{M-1} b_{\ell} x_{t-\ell}.$$

То есть в каждом моменте времени t новое значение z_t — это **взвешенная сумма** текущего и нескольких предыдущих значений входного ряда, а число M — это длина памяти фильтра [15, 16].

Вейвлет-разложение строится на паре КИХ-фильтров:

- h_{ℓ} - **низкочастотный** фильтр, выделяющий гладкую часть ряда (уровень/тренд и медленные изменения);
- g_{ℓ} - **высокочастотный** фильтр, выделяющий детали (быстрые колебания, локальные изменения).

где обычно $\ell = 0, \dots, M-1$ [15, 16].

Фильтры h_{ℓ} и g_{ℓ} подбираются так, чтобы из полученных коэффициентов можно было восстановить исходный ряд, то есть чтобы они образовывали **банк фильтров идеальной реконструкции** [15, 16].

DWT (дискретное вейвлет-преобразование)

На первом уровне DWT мы делаем два действия:

1. Сначала фильтруем ряд фильтрами h_{ℓ} и g_{ℓ} ;
2. После этого оставляем только каждое второе значение, из-за чего число коэффициентов уменьшается вдвое — это и составляет классический алгоритм Mallat для DWT [15, 16].

Коэффициенты аппроксимации и деталей:

$$a_{1,k} = \sum_{\ell=0}^{M-1} h_{\ell} y_{2k-\ell}, \quad d_{1,k} = \sum_{\ell=0}^{M-1} g_{\ell} y_{2k-\ell}.$$

Здесь:

- $a_{1,k}$ - гладкая часть (аппроксимация) на масштабе 1,
- $d_{1,k}$ - детали на масштабе 1 [15, 16].

Дальше преобразование повторяют **не для исходного ряда**, а для аппроксимации (то есть снова разлагают только гладкую часть).

Для $j \geq 2$ [15]:

$$a_{j,k} = \sum_{\ell=0}^{M-1} h_{\ell} a_{j-1, 2k-\ell}, \quad d_{j,k} = \sum_{\ell=0}^{M-1} g_{\ell} a_{j-1, 2k-\ell}.$$

То есть на каждом уровне мы снова получаем:

- $a_{j,k}$ - аппроксимацию,
- $d_{j,k}$ - детали.

Из-за урезания значений на каждом уровне длины коэффициентов уменьшаются примерно в 2^j раз:

$$|a_j| \approx |d_j| \approx \frac{T}{2^j}.$$

Именно поэтому DWT называют **критически дискретизированным** – коэффициентов получается примерно столько же, сколько исходных данных (без существенной избыточности) [15, 16].

Если восстановить компоненты из коэффициентов (выполнить обратное преобразование), то можно получить стандартную сумму:

$$y_t = A_J(t) + \sum_{j=1}^J D_j(t),$$

где:

- $A_J(t)$ - сглаженная часть ряда на грубом масштабе J ,
- $D_j(t)$ - детальная компонента масштаба j , восстановленная из $d_{j,k}$ [15, 16].

Главный практический минус DWT — **чувствительность к сдвигу**. Именно из-за того, что выполнялась фильтрация каждого второго значения даже небольшое смещение исходного ряда может заметно изменить коэффициенты на нескольких уровнях.

Именно эту проблему и решает MODWT [5].

1.3.3. MODWT: максимальное перекрывающееся дискретное вейвлет-преобразование

MODWT построено так, чтобы устранить основной практический недостаток DWT — **чувствительность к сдвигу**, поэтому ключевая идея MODWT состоит в том, что оставляются все значения после фильтрации, поэтому вейвлет- и скейлинг-коэффициенты остаются привязаны ко всем моментам времени t и при сдвиге исходного ряда в основном просто сдвигаются вместе с ним [5].

По сравнению с DWT:

1. Урезание не выполняется, поэтому на каждом уровне получаются коэффициенты длины T , а не $\frac{T}{2^j}$, как в DWT (см. главу 1.3.2) [5].
2. В классическом изложении MODWT базовые фильтры h_l, g_l (для DWT) переводятся в MODWT-вариант через нормировку (в простейшей записи деление на $\sqrt{2}$, а в общем виде — масштабная нормировка по уровню) [5]:

$$\tilde{h}_\ell = \frac{h_\ell}{\sqrt{2}}, \quad \tilde{g}_\ell = \frac{g_\ell}{\sqrt{2}}$$

(Эквивалентная форма в литературе — уровень-зависимая нормировка, обеспечивающая корректную энергию коэффициентов на каждом масштабе) [5].

3. Фильтры растягиваются по масштабам — на уровне j между соседними коэффициентами вставляют $2^{j-1} - 1$ нулей, чтобы фильтр соответствовал более крупному масштабу и выделял более низкие частоты [5].

Такая многошкальная фильтрационная интерпретация согласуется с общей идеологией мультиразрешающего анализа и банков фильтров (см. главу 1.3.2) [15, 16].

Коэффициенты MODWT считаются как свертка, часто с периодическим продолжением индексов по модулю T для аккуратной обработки границ [5]:

$$\tilde{a}_{j,t} = \sum_{\ell} \tilde{h}_{j,\ell} \tilde{a}_{j-1,t-\ell}, \quad \tilde{d}_{j,t} = \sum_{\ell} \tilde{g}_{j,\ell} \tilde{a}_{j-1,t-\ell}, \quad \tilde{a}_{0,t} = y_t.$$

где $\tilde{h}_{j,l}$ и $\tilde{g}_{j,l}$ — растянутые фильтры для уровня j [5].

Свойства MODWT:

- $\tilde{a}_{j,t}$ и $\tilde{d}_{j,t}$ имеют длину T для всех j , поскольку не избавляемся от каждого второго значения [5].
- Преобразование становится **избыточным**: всего $(J + 1) \cdot T$ коэффициентов, что является платой за дополнительные свойства (в сравнении с критически дискретизированным DWT (см. главу 1.3.2)) [5].

- При сдвиге исходного ряда коэффициенты в основном сдвигаются аналогично, без резких перестроений по масштабам, характерных для DWT (см. главу 1.3.2) [5].

В дальнейшем под $A_j(t)$ и $D_j(t)$ будем понимать реконструированные сглаживание и детали **MODWT-разложения**, которые используются как компоненты в гибридной модели [5, 26, 27, 30].

Особенности реализации MODWT в данной работе:

- В реализации предусмотрено **автоматическое ограничение глубины разложения**.

Фактический уровень J выбирается как $J = \min(level, J_{max})$, где J_{max} зависит от длины ряда T и длины вейвлет-фильтра, что предотвращает некорректные разложения на коротких рядах и делает процедуру устойчивой при пакетной обработке данных.

- Помимо стандартного периодического продолжения (*wrap*), реализована обработка границ для непериодических рядов (*reflect/mirror/nearest*): ряд симметрично дополняется (*padding*), затем свертка выполняется в режиме *wrap*, а результат обрезается до исходной длины T (это снижает артефакты на границах по сравнению с чисто циклической свёрткой).
- Реализована не только декомпозиция на коэффициенты, но и **многоуровневая реконструкция (MRA)**: вычисляются восстановленные компоненты $A_j(t)$ и $D_j(t)$ длины T , которые напрямую используются как компоненты будущей гибридной модели.
- Для удобства гибридизации детали возвращаются в порядке **от более грубых масштабов к более мелким**, чтобы отдельно назначать модели на низких и высоких частотах.

1.3.4. Сравнение методов декомпозиции DWT, MODWT, EWT и VMD

Как уже было сказано, DWT и MODWT относятся к классической вейвлет-парадигме: задана пара КИХ-фильтров низких и высоких частот, образующая банк идеальной реконструкции, и разложение строится как многоуровневая фильтрация (см. выше) [15, 16].

Ключевое различие между ними связано с оставлением каждого второго отсчета. DWT урезает коэффициенты, получая компактное представление, но становится **чувствительным к сдвигу** (см. главу 1.3.2), а MODWT, напротив, сохраняет все отсчёты после фильтрации, поэтому коэффициенты остаются привязаны ко всем моментам времени и гораздо стабильнее при сдвиге ряда. Плата за это – **избыточность** (число коэффициентов растёт до $(J + 1) \cdot T$) [5].

Практически для гибридного прогнозирования это означает:

- DWT выгоден, когда важна компактность и скорость, а устойчивость к фазовым сдвигам не критична.

- MODWT предпочтительнее, когда компоненты должны быть стабильны во времени и напрямую использоваться как входы моделей (как $A_j(t)$ и $D_j(t)$). Практические схемы декомпозиции, отдельного моделирования компонент, а также последующей сборки прогноза широко применяются в задачах прогнозирования, включая варианты с линейными и нейросетевыми моделями поверх MODWT-компонент [5 - 8].

EWT сохраняет идеологию банков фильтров и мультиразрешения [13, 14], но вместо фиксированной разметки частотных масштабов строит **адаптивные частотные полосы по спектру конкретного сигнала** — это полезно, когда частотные компоненты выражены не на стандартной вейвлетной сетке, а концентрируются в специфических спектральных областях. На практике EWT используют как этап предобработки перед прогнозированием, то есть сначала формируются адаптивные компоненты, которые затем подаются в прогностическую модель, что может улучшать качество на нестационарных рядах [19]. Сравнительные прикладные исследования также показывают, что выбор между EWT и методами семейства EMD влияет на стабильность выделяемых компонент и качество последующей обработки [17, 18].

VMD по своей идее ближе не к классическим вейвлетам, а к модовым методам разложения вроде **EMD**. Изначально его предложили, как способ разбирать нелинейные и нестационарные временные ряды на набор эмпирических мод, а затем анализировать эти компоненты с помощью преобразования Гильберта [15, 34], однако в прикладных задачах прогнозирования для **EMD** обнаружились проблемы стабильности и корректности постановки (информационные утечки при построении признаков на временных рядах), поэтому выбор модовой декомпозиции должен учитывать воспроизводимость временной схемы обработки [33].

VMD формулирует разложение как **вариационную задачу** поиска K полосограниченных мод с настраиваемыми центральными частотами [36]. VMD часто используется как этап выделения компонент перед моделью: компоненты моделируются отдельно или подаются в ансамбль, и в прикладных работах показывается выигрыш по точности на задачах краткосрочного прогнозирования [37, 40]. Кроме того, VMD активно применяется для выделения информативных компонент при извлечении признаков в задачах анализа сигналов, что демонстрирует универсальность механизма разложения на моды, а также существуют примеры VMD + DL в краткосрочном многогоризонтном прогнозировании нагрузки [38, 39]

Сравнительная таблица всех методов разложения исходного ряда представлена в таблице 1.3.4.1.

Таблица 1.3.4.1. Сравнительная таблица методов разложения исходного ряда.

Критерий	DWT	MODWT	EWT	VMD
Базовая идея	Фиксированный вейвлетный банк фильтров + урезание каждого второго отсчета [15, 16]	Вейвлетный банк без урезания, коэффициенты длины (T) [5]	Адаптивный вейвлетный банк, построенный по спектру сигнала (в рамках банков	Вариационная оптимизация: (K) полосограниченных мод [36]

			фильтров) [17, 18, 19]	
Адаптивность к данным	Низкая (задаётся выбором вейвлета/уровней) [15, 16]	Низкая (аналогично DWT) [5]	Высокая (полосы строятся под сигнал) [19]	Высокая (моды и центральные частоты подстраиваются в оптимизации) [36]
Избыточность	Низкая (критическая дискретизация) [15, 16]	Высокая $((J + 1) \cdot T)$ [5]	Обычно умеренная (зависит от числа полос) [19]	Зависит от K , но обычно не критическая (разложение на K компонент) [36]
Устойчивость к сдвигу	Низкая (чувствителен к фазовым сдвигам из-за урезания каждого второго отсчета) [5, 15, 16]	Высокая (коэффициенты в основном сдвигаются вместе с рядом) [5]	Средняя/высокая (зависит от стабильности спектральной сегментации) [19]	Средняя/высокая (зависит от параметров и устойчивости оптимизации) [36]
Основные параметры	Вейвлет, уровень J , обработка границ [15, 16]	Вейвлет, уровень J , обработка границ, периодизация [5]	Число полос и правило сегментации спектра [19]	Число мод K , регуляризация/штраф, критерии остановки [36]
Риск плохого разложения	При неверном выборе вейвлета/уровня — смешивание частот [15, 16]	Аналогично DWT, но меньше артефактов от сдвига [5]	При нестабильной сегментации — дрожание границ полос и компонент [17, 18]	При неверных K и параметрах — дробление/смешивание мод [36]
Удобство для гибридного прогноза	Хорошо , как компактный признак/предобработка [15, 16]	Очень хорошо: компоненты длины T удобны как входы моделей [5, 6, 7, 8]	Хорошо при устойчивом выделении спектральных зон [19]	Хорошо: часто используется как декомпозиция перед прогнозом [40]; важно контролировать корректность временной схемы и утечки [33, 34]

1.4. Нейросетевые модели для временных рядов

Нейросетевые методы стали конкурентоспособными для прогнозирования временных рядов на больших и разнородных коллекциях данных. Они в отличие от статистических моделей учатся по данным извлекать нелинейные зависимости и взаимодействия лагов без ручного задания структуры, тогда как ARIMA строится вокруг линейной динамики после приведения ряда к стационарности (через дифференцирование и т.п.) [1, 2]. Для

современных глубоких подходов отдельно обсуждается вопрос устойчивости прогнозов и влияние архитектурных решений на качество вне обучающего распределения [25].

В работе рассматриваются специализированные архитектуры **TimesNet** и **N-BEATS**.

TimesNet предлагает представление временного ряда через преобразование в 2D-структуру и моделирование временных паттернов как 2D-вариаций, что позиционируется как универсальная схема для ряда задач анализа временных рядов, включая прогнозирование [27]. Практическая модификация TimesNet применяется и в прикладной задаче прогнозирования PM2.5, где акцент сделан на интерпретируемое разделение внутривариационных и межвариационных вариаций [28].

N-BEATS строит прогноз через итеративные блоки, представляющие собой последовательное объяснение истории и уточнение прогноза, демонстрируя сильные результаты на стандартных наборах и предлагая интерпретируемые базисы (в зависимости от конфигурации блоков) [23]. Расширение **NBEATSx** добавляет экзогенные переменные и показывает эффективность на прикладном прогнозировании цен, сохраняя идею базисного разложения [24].

Для полноты контекста современные архитектуры для прогнозирования временных рядов также развиваются в семействе Transformer-подходов. Так, например:

- Autoformer использует декомпозиционную идею и механизм автокорреляции для долгосрочного прогноза; [29]
- Informer оптимизирует внимание для длинных последовательностей; [30]
- iTransformer показывает эффективность инвертированного представления для прогнозирования; [31]
- TSLANet переосмысливает трансформер-блоки для представлений временных рядов; [32]

1.4.1. Модель TimesNet

Пусть входной фрагмент временного ряда (или набора рядов) задан тензором

$$X_{1D} \in \mathbb{R}^{T \times C},$$

где T - длина окна, C - число переменных (для унивариативного ряда $C = 1$). Основная идея TimesNet, введенной в работе Wu и соавт. [27], - превратить 1D-ряд в несколько картинок (2D-тензоров), каждая из которых соответствует одной из доминирующих периодичностей, а затем обработать их 2D-свертками.

Такое преобразование формирует два типа локальности: **внутривариационную** (по колонкам) и **межвариационную** (по строкам), что и составляет основу «temporal 2D-variation modeling» в TimesNet [27].

Оценка периодов через FFT (блок Period)

TimesNet использует спектральный анализ: вычисляется дискретное преобразование Фурье и амплитудный спектр [27].

Пусть

- $FFT(\cdot)$ - преобразование Фурье;
- $Amp(\cdot)$ - вычисление амплитуд;
- $Avg(\cdot)$ – функция, усредняющая амплитуды по каналам;

Удобно записывать это в виде [27]:

$$A = Avg(Amp(FFT(X_{1D}))), \quad A \in \mathbb{R}^T$$

Каждое значение A_j интерпретируется как «интенсивность» частоты j (периодической базисной функции) и связано с длиной периода порядка $\lceil T/j \rceil$.

Далее выбирают $top - k$ наиболее значимых частот $\{f_1, \dots, f_k\}$ по амплитудам, чтобы отсеять шумовые/малополезные частоты - из-за сопряжённости спектра рассматривают частоты только в диапазоне $f \in \{1, \dots, \lceil T/2 \rceil\}$ [27].

Итог работы блока *Period* можно записать так [27]:

$$A, \{f_1, \dots, f_k\}, \{p_1, \dots, p_k\} = Period(X_{1D}),$$

$$p_i = \left\lceil \frac{T}{f_i} \right\rceil - \text{длины периодов по доминирующим частотам, } i = 1, \dots, k.$$

В прикладных модификациях TimesNet идея отдельного учёта внутрипериодных и межпериодных вариаций сохраняется и используется для повышения интерпретируемости (например, в TimesNet-PM2.5) [28].

Преобразование $1D \rightarrow 2D$ (reshape)

После выбора частот $\{f_i\}$ и соответствующих периодов $\{p_i\}$ строится набор 2D-тензоров: (при необходимости ряд дополняют нулями) [27]:

$$X_{2D}^i = \text{Reshape}_{p_i f_i}(\text{Padding}(X_{1D})), \quad i = 1, \dots, k.$$

Здесь:

- $\text{Padding}(\cdot)$ - **дополнение нулями по времени**, чтобы длина стала совместима с матрицей размера $p_i \times f_i$.
- $\text{Reshape}_{p_i f_i}(\cdot)$ укладывает последовательность в матрицу с p_i строками и f_i столбцами.

Полученный тензор $X_{2D}^i \in \mathbb{R}^{p_i \times f_i \times C}$ так устроен, что **столбцы** отвечают за **intraperiod-variation** (изменения *внутри одного периода*), а **строки** - за **interperiod-variation** (изменения *между соседними периодами*) при длине периода p_i . Именно эта идея $1D \rightarrow 2D$ по найденным периодам является ключевой отличительной чертой TimesNet по

сравнению с прямой 1D-сверткой/трансформерами без явного «периодного» разворота [27].

Главная идея преобразования временного ряда из 1D в 2D по найденному периоду, позволяющее отдельно учитывать внутрипериодные и межпериодные изменения показана на рисунке 1.4.1.1.

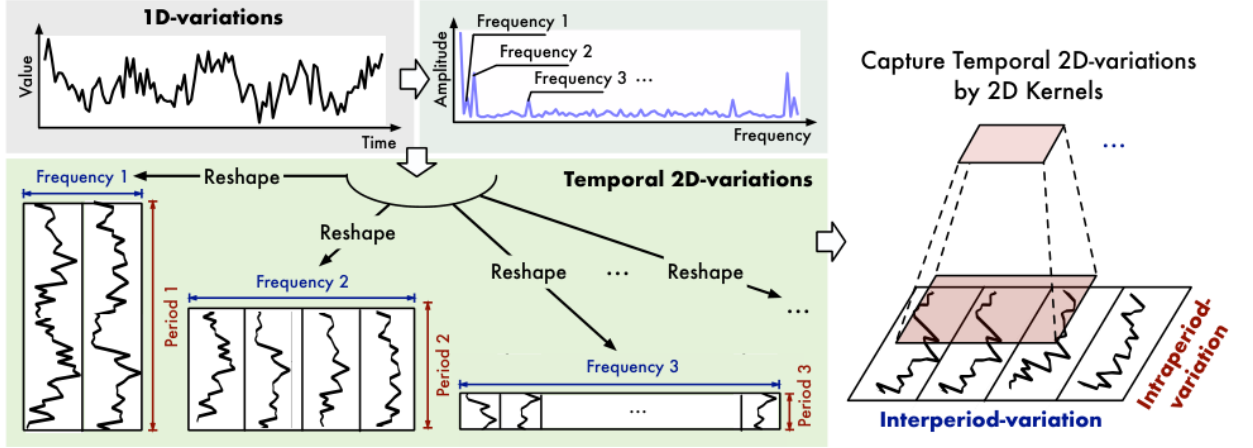


Рисунок 1.4.1.1. Идея TimesNet: преобразование временного ряда из 1D в 2D по найденному периоду, позволяющее отдельно учитывать внутрипериодные и межпериодные изменения.

TimesBlock и общая архитектура

Сеть начинается с преобразования входа в скрытое пространство:

$$X_{1D}^0 = \text{Embed}(X_{1D}) \in \mathbb{R}^{T \times d_{\text{model}}}$$

Затем TimesNet строится как стек блоков TimesBlock с резидуальным соединением [27]:

$$X_{1D}^l = \text{TimesBlock}(X_{1D}^{l-1}) + X_{1D}^{l-1}.$$

Внутри l -го TimesBlock выполняются два шага:

(1) Захват temporal 2D-variations

В оригинальной постановке TimesNet периодичности оцениваются не по исходному ряду, а по текущим признакам X_{1D}^{l-1} (то есть *Period* применяется на каждом слое к *embedding*-признакам) [27].

Дальше для каждого периода i выполняется один и тот же конвейер [27]:

$$A^{l-1}, \{f_i\}, \{p_i\} = \text{Period}(X_{1D}^{l-1}),$$

$$X_{2D}^{l,i} = \text{Reshape}_{p_i, f_i}(\text{Padding}(X_{1D}^{l-1})),$$

$$\hat{X}_{2D}^{l,i} = \text{Inception}(X_{2D}^{l,i}),$$

$$\hat{X}_{1D}^{l,i} = \text{Trunc}(\text{Reshape}_{1,(p_{if_i})}(\hat{X}_{2D}^{l,i})).$$

Здесь:

- $\text{Inception}(\cdot)$ - параметрически лёгкий inception-блок (или MSUnit), содержащий **несколько 2D-ядер разных размеров** (мультимасштабная обработка) [27].
- $\text{Trunc}(\cdot)$ отрезает добавленные padding-значения, возвращая исходную длину T [27].
- Важный смысл: 2D-ядра одновременно агрегируют колонки и строки, покрывая зависимости по соседним точкам и соседним периодам, а один и тот же inception/MSUnit **разделяет параметры** между всеми k представлениями (для разных периодов), поэтому размер модели не растёт с k - он инвариантен к выбору числа периодов [27].

(2) Адаптивная агрегация признаков по периодам.

После получения для всех выполняется взвешенное объединение: веса зависят от амплитуд спектра и нормируются softmax. После этого признаки суммируются [27].

$$X_{1D}^l = \sum_{i=1}^k \tilde{A}_{f_i}^{l-1} \hat{X}_{1D}^{l,i}$$

Интуитивно: если некоторый период выражен сильнее (больше амплитуда в спектре), то признаки, извлечённые из reshape по этому периоду, получают больший вес [27].

На рисунке 1.4.1.2. показана архитектура TimesNet, блока TimesBlock с выбором периодов (FFT/Top-k), формированием 2D-представлений, обработкой 2D-свёртками (MSUnit/Inception) и адаптивной агрегацией по периодам.

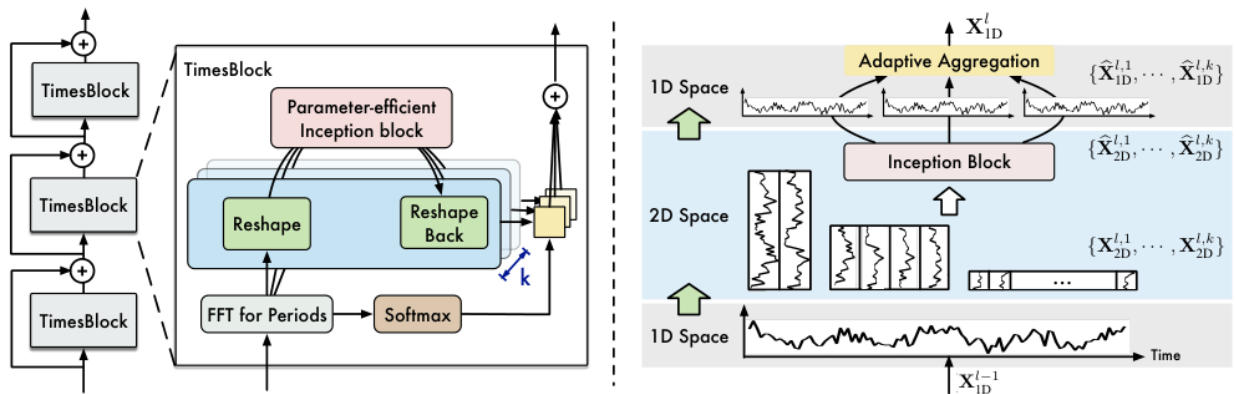


Рисунок 1.4.1.2. Архитектура TimesNet: блок TimesBlock с выбором периодов (FFT/Top-k), формированием 2D-представлений, обработкой 2D-свёртками (MSUnit/Inception) и адаптивной агрегацией по периодам, а также остаточной связью.

Особенности реализации TimesNet в данной работе

В данной работе реализован вариант **TimesNetV2**, адаптированный под унивариативный краткосрочный прогноз и последующую гибридизацию:

- **Периоды выбираются один раз по исходному входному окну** (через $rFFT$ и $top - k$ по средним амплитудам), и используются далее во всех слоях, снижая вычислительную стоимость по сравнению с каноническим вариантом [27].
- В операции $Padding(\cdot)$ используется **дополнение повторением последнего значения**, а не нулевое дополнение, чтобы уменьшить искусственные скачки на границе окна при укладке в матрицу $p_i \times f_i$ и сделать 2D-свертку более устойчивой на концах последовательности (особенно для коротких окон).
- **Inception-обработка** реализована как облегчённый $MSUnit$ с нормировкой и проекцией обратно в число каналов, что сохраняет идею мультимасштабной 2D-обработки из TimesNet [27], но делает блок компактным для практических экспериментов.
- После агрегации по периодам добавлены **LayerNorm** и **FFN-блок** ($Conv1d - MLP$), а прогноз формируется через $MLP - head$; также предусмотрена опция сдвига прогноза на последний наблюдаемый уровень, что повышает устойчивость уровня прогноза на горизонте при краткосрочном прогнозировании.

TimesNet после выхода быстро стал базовой моделью, с которой сравнивают новые методы представлений временных рядов (например, в TSLANet) [32], и которую часто адаптируют под прикладные задачи, где важно понятно разделять изменения *внутри периода* и *между периодами* (например, TimesNet-PM2.5) [28].

В данной работе TimesNet используется как часть общего экспериментального контура и гибридных схем (см. главу 2), так как важна стабильная работа на коротких окнах и предсказуемая вычислительная стоимость.

Параллельно активно развиваются трансформер-подходы для прогнозирования временных рядов. В них зависимости извлекаются механизмами внимания, а ряд подаётся как последовательность токенов. Самая распространённая идея — **patching/tokenization**: ряд разбивают на короткие отрезки (patches) и считают каждый отрезок токеном. Такой подход наглядно показан в PatchTST [26]: токенов становится меньше, обучение на длинных окнах проще, при этом остаётся возможность улавливать дальние зависимости.

Дальше появились модификации под мультивариативные ряды и очень длинные окна.

- **iTransformer** предлагает делать токены не по времени, а по переменным, чтобы лучше ловить межпеременные связи и устойчивее работать при большом lookback-окне [31].
- **Autoformer** встраивает идею декомпозиции и использует механизм auto-correlation для выделения и агрегации периодических зависимостей [29].
- **Informer** нацелен на ускорение работы с длинными последовательностями: применяет разреженное (ProbSparse) внимание и «distilling» в энкодере, чтобы уменьшить как вычислительные затраты, так и затраты на память [30].

Краткая сравнительная таблица подходов различной обработки временных рядов указана в таблице 1.4.1.1.

Таблица 1.4.1.1. Сравнительная таблица подходов различной обработки временных рядов.

Критерий	TimesNet	Transformer-подход (PatchTST / iTransformer / Autoformer / Informer)
Основная идея	FFT → top-k периодов → 1D → 2D по периодам → 2D-свертки + агрегация [27]	Токены (отрезки/переменные/временные шаги) → механизмы внимания; оптимизации под длинные окна [26, 29–31]
Индуктивный приоритет	Явная периодичность + <i>intraperiod/interperiod</i> локальности [27]	Универсальные зависимости; периодичность может извлекаться неявно (или через <i>auto-correlation</i> в [29])
Длинные окна	Стоимость контролируема при фикс. k ; периодный блок зависит от T [27]	Требует борьбы с длиной токенов: patching [26], разреженное внимание [30], инверсия осей [31]
Мультивариативность	Поддерживается через каналы C , периоды оцениваются по признакам/каналам [27]	Сильные варианты именно для мультивариативности: iTransformer [31]; <i>patch-based</i> дизайн [26]
Интерпретируемость	Периоды и веса агрегации интерпретируемы [27, 28]	Обычно менее прямолинейна; зависит от токенизации/анализа внимания [26, 31]

1.4.2. Модель N-BEATS

Пусть дано окно истории (*lookback*) длины L и требуется предсказать H будущих значений:

$$x \in \mathbb{R}^L, \hat{y} \in \mathbb{R}^H.$$

Архитектура **N-BEATS** (Neural Basis Expansion Analysis for Interpretable Time Series Forecasting) введена Oreshkin и соавт. как специализированная нейросетевая модель для унивариативного прогнозирования, где прогноз строится как сумма вкладов последовательности блоков [23]. В оригинальной постановке часто берут L кратным H (типично $2H \dots 7H$), что упрощает обучение на фиксированных параметрах окна и горизонта [23].

Модель используется как сильный нейросетевой базовый уровень в исследованиях и экспериментах на стандартных наборах, включая M4 [3, 23]. Развитие идеи N-BEATS —

модель NBEATSx, где сохраняется базисное разложение, но дополнительно учитываются экзогенные переменные, что важно для прикладных рядов с внешними факторами [24].

Ключевая идея N-BEATS: строить прогноз как **сумму частичных прогнозов блоков** (рисунк 1.4.2.1), где каждый блок одновременно:

1. объясняет часть входа x (делает *backcast*),
2. добавляет вклад в прогноз (делает *forecast*) [23].

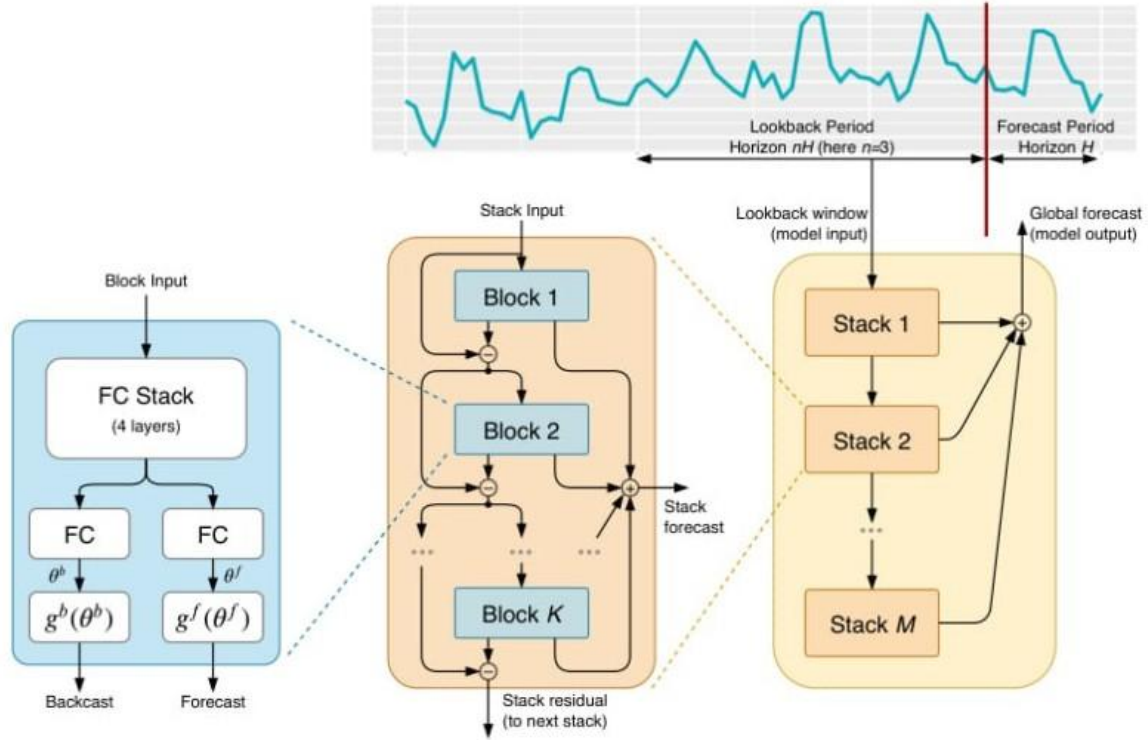


Рисунок 1.4.2.1. Общая архитектура N-BEATS: стек блоков с выходами *backcast* и *forecast*, остаточным обновлением входа и суммированием частичных прогнозов.

Базовый блок: *fork*-архитектура и коэффициенты разложения

Рассмотрим ℓ -й блок. Он получает вход x_ℓ и выдаёт:

- $\hat{x}_\ell \in \mathbb{R}^L$ - backcast (приближение текущего входа),
- $\hat{y}_\ell \in \mathbb{R}^H$ - частичный прогноз.

Внутри блока сначала стоит стек полносвязных слоёв с ReLU, после которого двумя линейными головами θ_ℓ^b и θ_ℓ^f [23] предсказываются коэффициенты разложения:

$$h_{\ell,1} = \text{FC}_{\ell,1}(x_\ell), \quad \dots, \quad h_{\ell,4} = \text{FC}_{\ell,4}(h_{\ell,3}),$$

$$\theta_\ell^b = \text{LINEAR}_\ell^b(h_{\ell,4}), \quad \theta_\ell^f = \text{LINEAR}_\ell^f(h_{\ell,4}).$$

Далее коэффициенты $\theta_\ell^b, \theta_\ell^f$ преобразуются в backcast/forecast через **базисные слои** $g_\ell^b(\cdot)$ и $g_\ell^f(\cdot)$:

$$\hat{y}_\ell = g_\ell^f(\theta_\ell^f), \quad \hat{x}_\ell = g_\ell^b(\theta_\ell^b).$$

Они реализуют разложение по базисным векторам:

$$\hat{y}_\ell = \sum_{i=1}^{\dim(\theta_\ell^f)} \theta_{\ell,i}^f v_i^f, \quad \hat{x}_\ell = \sum_{i=1}^{\dim(\theta_\ell^b)} \theta_{\ell,i}^b v_i^b.$$

где $\{v_i^f\}, \{v_i^b\}$ задают семейство базисов для прогноза и для аппроксимации входа [23].

Doubly residual stacking (DRESS)

Блоки соединяются по схеме «doubly residual stacking». Это значит, что следующий блок получает **остаток** после вычитания backcast предыдущего блока, а forecast накапливается суммированием [23]:

$$x_\ell = x_{\ell-1} - \hat{x}_{\ell-1}, \quad \hat{y} = \sum_{\ell} \hat{y}_\ell.$$

Интуитивно ранние блоки объясняют простую часть динамики и вычитают её из входа, упрощая задачу последующим блокам. Таким образом, каждый блок параллельно добавляет свой вклад в итоговый прогноз [23].

Сравнение вариантов соединения блоков N-BEATS и влияние топологии (в т.ч. «doubly residual stacking») на формирование прогноза показано на рисунке 1.4.2.2 [23].

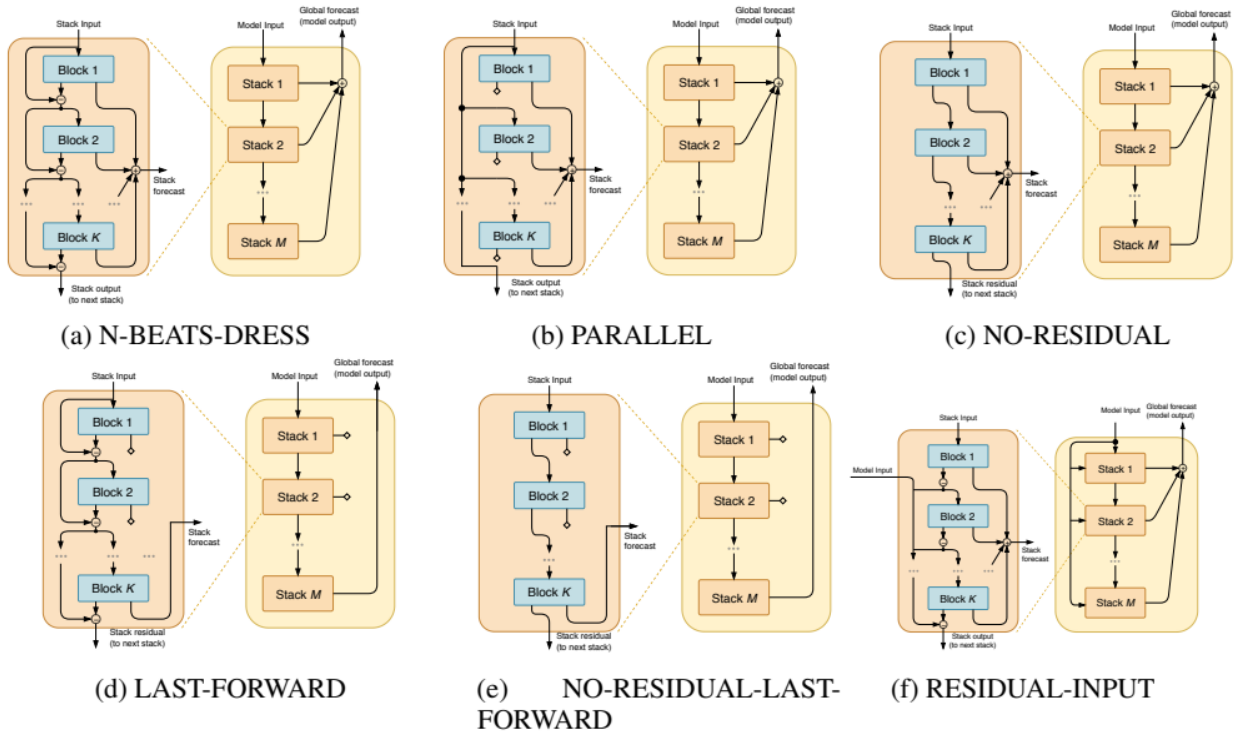


Рисунок 1.4.2.2. Варианты топологии N-BEATS: сравнение схем соединения блоков и роль «doubly residual stacking».

Generic vs Interpretable: от «обучаемого» базиса к тренду и сезонности

- **Generic (N-BEATS-G)**

В generic конфигурации базисные преобразования можно сделать полностью обучаемыми, например, как линейную проекцию [23]:

$$\hat{y}_\ell = V_\ell^f \theta_\ell^f + b_\ell^f, \hat{x}_\ell = V_\ell^b \theta_\ell^b + b_\ell^b,$$

где столбцы V_ℓ^f можно понимать как обучаемые шаблоны во временной области прогноза.

Однако без дополнительных ограничений такие шаблоны не обязаны иметь трендовую или сезонную структуру, поэтому разложение оказывается неинтерпретируемым [23].

- **Interpretable (N-BEATS-I): тренд + сезонность**

Чтобы получить интерпретируемые компоненты, фиксируют функциональный вид базиса на уровне стека: один стек отвечает за тренд, второй - за сезонность.

Тренд-стек. Прогноз тренда ограничивают полиномом небольшой степени p на сетке времени [23]:

$$t = \frac{1}{H} [0, 1, \dots, H - 1]^T,$$

$$\hat{y}_{s,\ell}^{\text{tr}} = \sum_{i=0}^p \theta_{s,\ell,i}^f t^i \Leftrightarrow \hat{y}_{s,\ell}^{\text{tr}} = T\theta_{s,\ell}^f, \quad T = [1, t, \dots, t^p].$$

Сезонный стек. Сезонность задают через ряд Фурье (сумма синусов и косинусов) [23]:

$$\hat{y}_{s,\ell}^{\text{seas}} = \sum_{j=0}^{\lfloor H/2-1 \rfloor} (\theta_{s,\ell,j}^f \cos(2\pi jt) + \theta_{s,\ell,j+H/2}^f \sin(2\pi jt)) \Leftrightarrow \hat{y}_{s,\ell}^{\text{seas}} = S\theta_{s,\ell}^f.$$

В интерпретируемой версии тренд-стек идёт первым: благодаря backcast-остатку тренд вычитается из входа перед сезонным стеком, а частичные прогнозы двух стеков можно трактовать как отдельные компоненты (тренд и сезонность).

Пример декомпозиции прогноза в N-BEATS показан на рисунке 1.4.2.3. [23]:

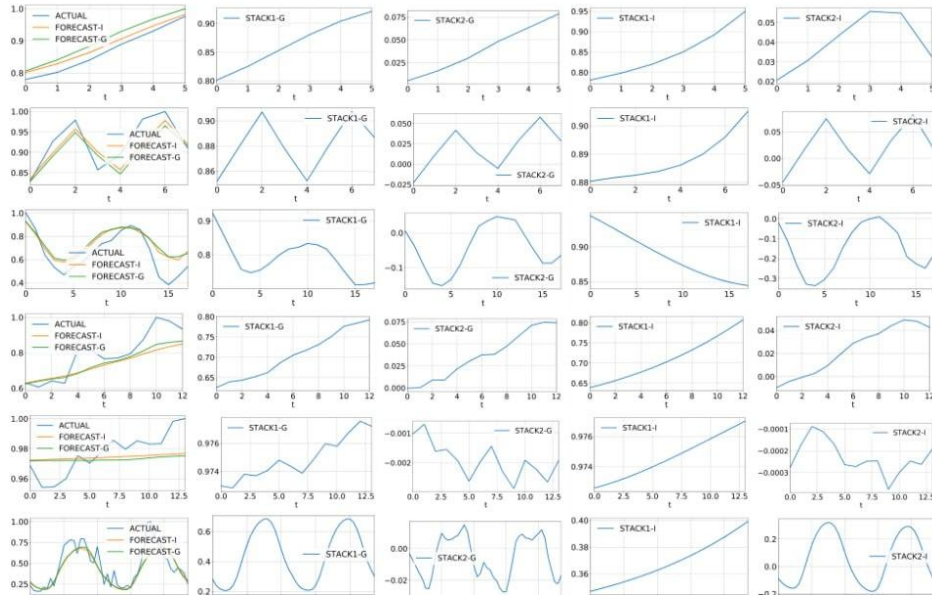


Рисунок 1.4.2.3. Пример декомпозиции прогноза в N-BEATS: сумма вкладов блоков (трендовая и сезонная компоненты)

Особенности реализации N-BEATS в данной работе

В данной работе реализован вариант **NBEATSV2**, ориентированный на устойчивый краткосрочный прогноз и удобную настройку под разные типы рядов:

- Модель может включать **одновременно** трендовый стек, сезонный стек и generic-стек (включаются/выключаются флагами), что позволяет гибко выбирать баланс интерпретируемости и выразительности в зависимости от данных.

- Предусмотрен режим, где **один и тот же блок переиспользуется n раз внутри стека**, что уменьшает число параметров по сравнению с классическим вариантом отдельного блока на каждую итерацию», сохраняя DRESS-схему
- Для тренда используется **полиномиальный** базис, для сезонности — базис **Фурье**, причем число гармоник выбирается автоматически (по умолчанию ограничивается сверху), а временная шкала нормируется на длину *lookback*.
- Степень полинома выбирается **автоматически** в зависимости от горизонта, чтобы не переусложнять тренд на коротких горизонтах и не недообъяснять его на более длинных.
- Реализована эвристика подавления одиночного всплеска на первом шаге (если $\tilde{y}_1$ резко выбивается относительно последующих разностей), что повышает устойчивость прогноза на коротком горизонте при шумных рядах.

Таким образом, в отличие от «канонического» описания N-BEATS [23], в данной работе сделан акцент на практической устойчивости и управляемой сложности: поддерживается смещение стеков, параметро-экономичная схема, а также дополнительные стабилизаторы прогноза (подавление единичного выброса первого шага), что важно для краткосрочного прогнозирования в общем экспериментальном контуре (см. главу 2).

1.5. Гибридные модели на основе декомпозиции временных рядов

Несмотря на успех как классических статистических моделей (ARIMA, ETS, Prophet), так и современных нейросетевых архитектур (TimesNet, N-BEATS), ни один подход не является универсально лучшим для всех типов временных рядов [1 - 3, 11, 14, 23, 27].

Эмпирически это подтверждается результатами крупных сравнений и соревнований по прогнозированию: лучшие решения часто оказываются гибридными или ансамблевыми, сочетая сильные стороны статистических и ML/NN-подходов. Пример — ES-RNN (гибрид экспоненциального сглаживания и рекуррентной сети) как один из наиболее сильных подходов в контексте M4, а также FFORMA как пример ансамбля, подбирающего веса моделей по признакам ряда [3, 21, 22]. Это стимулировало развитие гибридных схем, которые комбинируют разные индуктивные предположения (линейность/стационарность, тренд/сезонность, нелинейность) и повышают устойчивость качества прогноза на разнородных рядах [20 – 22, 25, 35].

Условно гибридные схемы можно разделить на несколько типов:

- **Декомпозиционные гибриды:** ряд предварительно разлагается на компоненты (тренд, сезонность, шум, частотные диапазоны), для каждой компоненты строится отдельная модель, а итоговый прогноз получается агрегированием прогнозов компонент. Такой подход характерен для вейвлет-прогнозирования: ряд декомпозируют по масштабам и прогнозируют компоненты отдельно, после чего собирают общий прогноз [5 – 8, 42]. Хорошим примером декомпозиционного гибрида является вейвлет-гибрид с последующим ARIMA и LSTM для финансовых рядов [41].
- **Остаточные гибриды:** базовая модель (например, ARIMA) строит первичный прогноз, затем отдельная модель обучается предсказывать остатки $e_t = y_t - \hat{y}_t^{base}$; итоговый прогноз — сумма базового прогноза и прогноза по остаткам.

Классический пример — гибриды ARIMA + ANN, где ARIMA захватывает линейную структуру, а нейросеть — нелинейную [20].

- **Ансамблевые гибриды:** несколько моделей обучаются независимо, а прогнозы объединяются (усреднение, взвешивание, стеккинг). Современный пример — FFORMA, где веса ансамбля выбираются метамоделью по признакам ряда [21].

В данной работе основной интерес представляет **декомпозиционный подход на основе MODWT**. В отличие от типичных декомпозиционных гибридов, где одинаковые модели ставятся на все компоненты, здесь делается акцент на разделение ролей: низкочастотная компонента (где сосредоточены тренд и крупномасштабные изменения) прогнозируется специализированными нейросетями (TimesNet / N-BEATS), а высокочастотные детали — более простыми и устойчивыми моделями (ARIMA/ETS), чтобы шумоподобные компоненты не разгоняли ошибку на горизонте [1, 2, 5, 11, 23, 27].

1.5.1. Декомпозиция на компоненты с помощью MODWT

Пусть временной ряд y_t разложен посредством MODWT на аппроксимацию и детали:

$$y_t = A_J(t) + \sum_{j=1}^J D_j(t),$$

где:

- $A_J(t)$ - низкочастотная компонента (аппроксимация), содержащая тренд и медленно меняющиеся составляющие;
- $D_j(t)$ - высокочастотные компоненты (детали) на разных масштабах, соответствующие быстрым колебаниям, сезонностям и шуму [5].

MODWT удобно использовать в прикладных гибридах, поскольку оно (в отличие от DWT) снижает чувствительность к сдвигу и сохраняет длину коэффициентов равной T на каждом масштабе, раздельное моделирование компонент и последующую реконструкцию прогноза [5].

Практические примеры прогнозирования на базе MODWT (включая сочетания с классическими и нейросетевыми моделями) представлены в работах, использующих MODWT как стадию выделения компонент перед прогнозом [6 – 8]. Также в прикладных задачах встречаются связки MODWT с автоматически настраиваемыми DNN и сравнением DL-моделей [43].

Естественная идея декомпозиционного гибрида состоит в том, чтобы строить **отдельные модели** для компонент:

- для $A_J(t)$ использовать модель, хорошо описывающую тренд и глобальные зависимости (например, *TimesNet* или *N – BEATS*) [23, 27];
- для $D_j(t)$ использовать более простые модели, чтобы не усиливать высокочастотную ошибку при сборке прогноза [5 – 8].

Стоит отметить, что порядок/глубина разложения влияет на качество DL-прогноза, поэтому выбор J — очень важен [48].

1.5.2. Концепция модели HybridPlus

Под *HybridPlus* в контексте данной работы понимается гибридная архитектура, реализующая следующую схему:

1. Исходный временной ряд y_t с помощью MODWT разбивается на набор компонент:

$$y_t = A_J(t) + D_1(t) + D_2(t) + \dots + D_J(t)$$

2. Для компоненты $A_J(t)$, содержащей основную часть тренда и крупномасштабных изменений, используется одна из современных нейросетевых моделей:
 - **TimesNet** — для захвата периодических структур через поиск доминирующих периодов и обработку 2D-представлений [27];
 - **N-BEATS** — для построения прогноза как суммы вкладов блоков с последовательным объяснением истории и уточнением прогноза, а также разложения на тренд и сезонность через фиксированные базисы [23].
3. Для компонент $D_j(t)$ используются модели, ориентированные на устойчивое описание остаточных зависимостей и шумоподобных колебаний — **ARIMA/ETS** [1, 2, 11], чтобы детали не разгоняли ошибку.
4. После получения прогноза на горизонт H для каждой компоненты агрегируем результат:

$$\hat{y}_{t+k} = \hat{A}_J(t+k) + \sum_{j=1}^J \hat{D}_j(t+k), \quad k = 1, \dots, H.$$

В литературе вейвлет-гибриды обычно строятся через разложение на компоненты, прогноза для каждой компоненты и суммирования, при этом ключевая инженерная сложность часто скрыта в обработке границ и в том, чтобы высокочастотные компоненты не ухудшали прогноз на горизонте [5–8]. В настоящей работе это фиксируется в алгоритме следующими механизмами:

- Прогноз деталей D_j на малых горизонтах смешивается с последним наблюдаемым уровнем детали (плавный переход), чтобы избежать разрыва между y_t и $\hat{y}_{t+1}$ при обратной сборке.
- Вклад высокочастотных компонент D_j монотонно уменьшается с ростом шага прогноза k (не доверяем далеким прогнозам шумоподобных компонент), что стабилизирует суммарный прогноз.
- Разность $\hat{y}_{t+1} - y_t$ ограничивается робастной оценкой типичного шага ряда (по историческим разностям), что снижает риск единичных выбросов первой точки прогноза при суммировании компонент.
- Помимо локального обучения на одном ряде, предусмотрен режим, когда модели обучаются на окнах компонент, собранных по множеству рядов (отдельно для A_J и для каждого D_j), что по смыслу сближает HybridPlus с подходами, показавшими эффективность на больших коллекциях временных рядов и в ансамблевых/гибридных схемах [3, 21, 22].

Глава 2. Практическая часть

2.1. Описание параметров и архитектуры вейвлета + моделей с практической точки зрения

Для выполнения практической части были реализованы следующие классические статистические методы и гибридные архитектуры:

2.1.1. ARIMA - один статистический блок ARIMA(p,d,q) на исходном ряде.

Параметры архитектуры:

$order = (p, d, q)$, по умолчанию (1,1,1).

- В *statsmodels.ARIMA*:
 - $trend = t$ при $d > 0$, иначе "c" (дрейф через линейный тренд при дифференцировании).
 - $enforce_stationarity = False, enforce_invertibility = False$.
 - $fit(..., maxiter = 200)$.

Примечание:

В **ARIMA**, а также в **ARIMA** с автоподбором была специально добавлена защита от вырожденного прогноза:

- Если ряд слишком короткий (< 6) $\rightarrow$ линейная экстраполяция.
- Если прогноз "плоский" ($std \approx 0$), но у последних разностей ряда std не нулевая $\rightarrow$ линейная экстраполяция (борьба с деградацией/сдвигом к константе).
- При ошибках: перебор альтернативных порядков ((0,1,1), (1,1,0), (0,1,0), (1,0,1)), затем линейная экстраполяция.

2.1.2. ARIMA (с автоподбором) - ARIMA-блок, но порядок (p,q) выбирается эвристикой по ACF/PACF.

Параметры автоподбора:

- $d = 1$ (по умолчанию), ограничения: $max_p = 3, max_q = 3$.
- $max_lag = \min(24, N//2)$.
- Порог значимости: $crit = 1.96 / \sqrt{N}$; выбирается **максимальный** значимый лаг $PACF \rightarrow p, ACF \rightarrow q$.
- Если данных мало ($N < 10$) или ACF/PACF недоступны $\rightarrow fallback (1, d, 0)$.

Fallback-логика:

- Если автоподбор падает → мини grid-search по AIC в окрестности: $d \in \{0,1\}$, $p, q \in \{0,1,2\}$, затем ARIMA с лучшим AIC.
- По аналогии с ARIMA: защита от «плоского» прогноза → линейная экстраполяция.

2.1.3. ETS

Параметры архитектуры:

- $trend = add$ (по умолчанию).
- Сезонность включается **только если** $seasonal_periods > 1$, иначе $seasonal = None$.
- Обучение: $fit(optimized = True, use_brute = True)$.

Fallback: если ряд короткий (< 4) или ошибка обучения → повтор последнего значения.

2.1.4. Prophet с аддитивной моделью и без встроенных календарных сезонностей

Параметры архитектуры:

- $seasonality_mode = additive$.
- $yearly_seasonality = False, weekly_seasonality = False, daily_seasonality = False$.
- Индекс времени создаётся искусственно: старт 2000 – 01 – 01, частота $freq = "D"$ по умолчанию.

Fallback: при ошибке или слишком коротком ряде → повтор последнего.

2.1.5. TimesNet+ (MODWT + TimesNet)

Конвейер (HybridPlus):

1. Разложение MODWT на A_j (тренд + сезонность) и детали D_j (остатки):
 - $wavelet = db4, level = 1$ (по умолчанию, будем задавать кол-во компонент D_j при запуске обучения).
 - Реализация разложения делается через собственные $modwt/modwtmra$, а не через $pywavelets$: $pywt$ используется только чтобы взять коэффициенты фильтров dec_lo/dec_hi .
2. Для каждой компоненты из $[A] + D$ обучается отдельный *TimesNetV2* (или переиспользуются готовые компоненты при совпадении числа мод).
3. После этого происходит стандартизация ($mean/std$) внутри $WindowDatasetStd(scale = True)$.
4. В конечном итоге суммируем прогноз компонент, затем стабилизируем границы.

Архитектура TimesNetV2 (используется как базовая нейросеть):

- Параметры по умолчанию:
 - $d_model=64$
 - $layers=4$
 - $topk=3$

- dropout=0.1
 - use_anchor=True.
- Вход/выход: $Conv1d(1 \rightarrow d_model) \rightarrow layers \text{ блоков} \rightarrow Conv1d(d_model \rightarrow 1) \rightarrow MLP - head$.
- Выбор периодов: FFT по входу, берём $topk$ пиков по средним амплитудам, веса — softmax по амплитудам выбранных частот.
- Блок 2D-вариаций: reshape в $(period \times cols)$, обработка inception-блоком, агрегация по периодам с весами.
- Inception-ядро MSUnit:
 - ветки $1 \times 1, 3 \times 3, 5 \times 5, 7 \times 7 + avgpool$ — ветка
 - $GroupNorm(1, 5 \cdot hidden_{ch})$
 - Dropout2d
 - 1×1 проекция.
- $hidden_ch = \max(16, \min(64, d_model//2))$.

Борьба со сдвигом на первом шаге:

После слияния прогноза в работе возникала проблема со сдвигом предсказания на первом шаге. Эта проблема была решена следующим образом:

- В TimesNet включён anchor: к прогнозу прибавляется последнее наблюдение $x[-1]$, чтобы первый шаг не проваливался к среднему.
- После суммирования компонент применяется ограничитель плохого скачка первого шага:

$$k = \min(24, \text{len}(y) - 1)$$

$$scale = \text{quantile}(|\Delta y|, 0.75) + 1e - 8$$

$$max_jump = 4 \cdot scale$$

первый шаг клипруется и исправляется “плохое направление” скачка.

2.1.6. N-Beats+ (MODWT + N-Beats)

Конвейер тот же HybridPlus (см. 2.1.5.), меняется только базовая нейросеть: вместо TimesNetV2 используется NBEATSV2.

Архитектура NBEATSV2:

- Параметры по умолчанию:
 - width = 256
 - depth = 4
 - nblocks = 4
 - dropout = 0.1
 - use_anchor = False
 - share_weights = True

- Стэки $\begin{cases} \text{use_trend} = \text{True} \\ \text{use_seasonality} = \text{True} \text{ (trend + seasonality + generic (если все выключены} \rightarrow \text{generic))} \\ \text{use_generic} = \text{True} \end{cases}$
- Распределение блоков по стэкам: $\text{counts} = \text{base} + (i < \text{extra})$ при $\text{base} = \text{nblocks}$
- Полином тренда: $\text{poly_deg} = 3$ если $\text{horizon} < 18$, иначе 4.
- Схема вычислений: двойная *residual*-стековка — каждый блок выдаёт *backcast/forecast*, *backcast* вычитается, *forecast* накапливается.

Используется борьба со сдвигом на первом шаге такая же, как и в TimesNet+ (см. 2.1.5):

2.1.7. VW + TimesNet + ARIMA (с автоподбором)

Если в TimesNet + и N – Beats + на деталях D_j для предсказаний использовались сами нейросетевые архитектуры, то для следующих моделей низкочастотная часть A_j предсказывается нейросетью, высокочастотные детали D_j — классической моделью, затем результаты суммируются.

В легенде графика название MODWT было заменено на VW для лучшего восприятия – это одно и то же.

Параметры архитектуры:

- MODWT: $\text{wavelet} = \text{db4}$, $\text{level} = 1$ (также регулируем), $\text{boundary} = \text{reflect}$.
- Детали: $\text{detail_method} = \text{"arima_auto"}$ (ветка *auto_arima_forecast*).
- Стабилизация деталей:
 - $\text{detail_anchor} = \text{True}$, $\text{detail_transition_steps} = 3$ (первые шаги смешиваются с $\text{last} = \text{comp}[-1]$).
 - $\text{detail_dampen} = \text{True}$, $\text{detail_dampen_tau} = 3.0$: экспоненциальное затухание $e^{-\frac{t}{\tau}}$ для подавления смещения на длинном горизонте.
- Стабилизация суммы деталей (*residual*):
 - $\text{residual_anchor} = \text{True}$
 - $\text{residual_transition_steps} = 6$, где $\text{resid_last} = y[-1] - A[-1]$, и первые k шагов плавно переходят от *resid_last* к смоделированному остатку.
- Финал: $\hat{y} = \text{aj_pred} + \text{resid_pred}$, затем *clamp* первого шага.

2.1.8. VW + TimesNet + ETS

То же, что и в VW + TimesNet + ARIMA (см. 2.1.7), но для каждой детали D_j используется ETS:

- Переключение по *detail_method* in {"ets", "holtwinters"}.
- Сезонность включается только если задан *seasonal_period* и > 1 : $\text{seasonal} = \text{"add"}$, иначе *None*.
- Остальные параметры (*boundary/anchor/dampen/residual_anchor/clamp*) — те же

2.1.9. VW + N-Beats + ARIMA (с автоподбором)

То же, что и в VW + TimesNet + ARIMA (см. 2.1.7), но *aj_model_fn* — *NBEATSV2* (архитектура из 2.1.6), а детали — *auto_arima_forecast*.

Стабилизация (*detail_anchor/detail_dampen/residual_anchor* + clamp первого шага) — идентична предыдущему пункту

2.1.10. VW + N-Beats + ETS

То же, что 2.1.9, но *aj_model_fn* — *NBEATSV2*, а детали — *ETS*-ветка

2.2. Применение и диагностика моделей на синтетических рядах

Вся диагностика и разработка моделей велась на синтетических рядах. Синтетический ряд строится, как аддитивная модель:

$$y(t) = 10.0 + (trend_{slope} \cdot t) + season(t) + \varepsilon_t, \quad \varepsilon_t \sim N(0, noise_std)$$

Сезонность включается только, если *season_period* > 1 и *season_amp* > 0:

$$season(t) = season_amp \cdot \sin\left(\frac{2\pi t}{season_period}\right)$$

2.2.1. Используемые профили (ряды для испытаний)

- *flat_low_noise* (рисунок 2.2.1 и 2.2.2)
 - *trend_slope* = 0.0
 - *season_period* = None
 - *season_amp* = 0.0
 - *noise_std* = 0.1

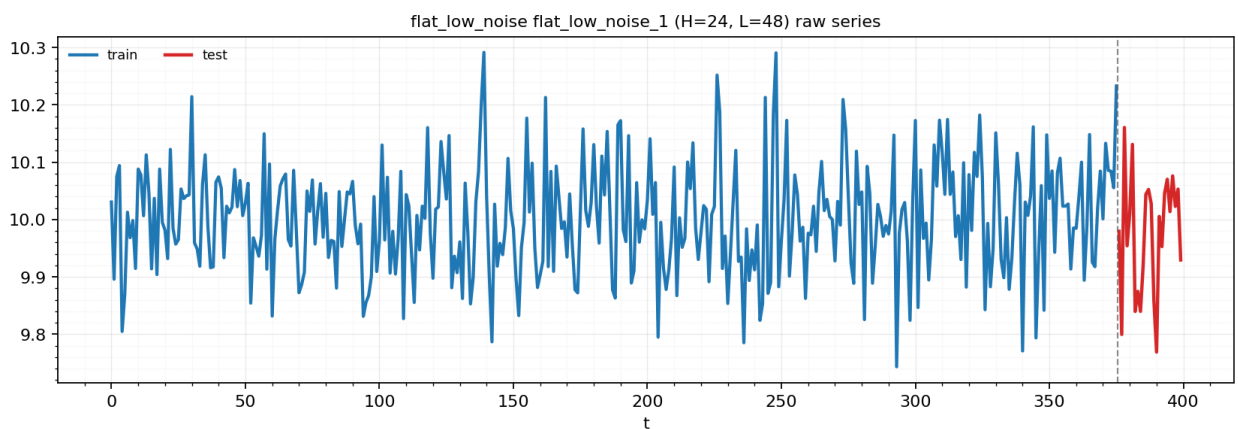


Рисунок 2.2.1. Синтетический ряд *flat_low_noise_1*, разделенный на *test* и *train*.

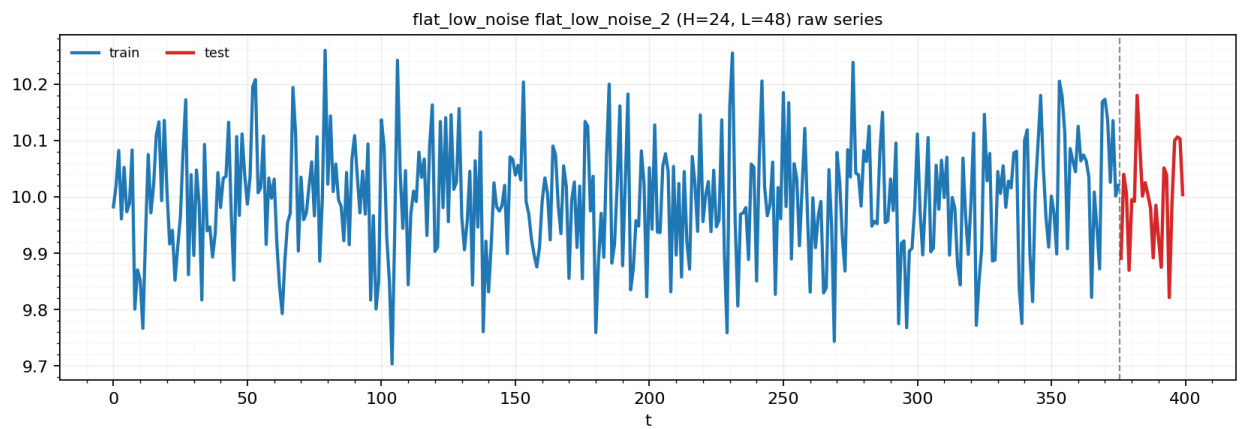


Рисунок 2.2.2. Синтетический ряд *flat_low_noise_2*, разделенный на *test* и *train*.

- *trend_only* (рисунок 2.2.3 и 2.2.4)
 - *trend_slope* = 0.02
 - *season_period* = None
 - *season_amp* = 0.0
 - *noise_std* = 0.1

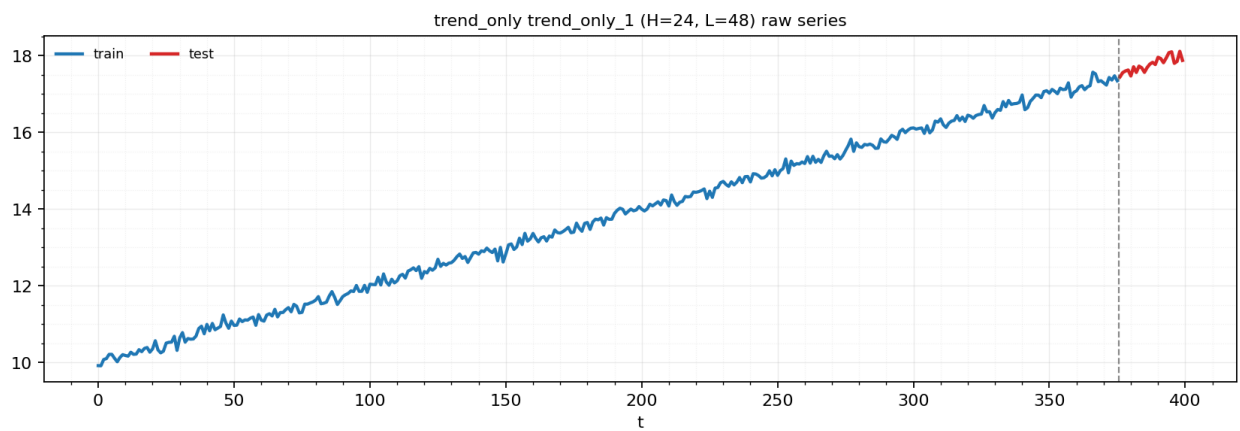


Рисунок 2.2.3. Синтетический ряд *trend_only_1*, разделенный на *test* и *train*.

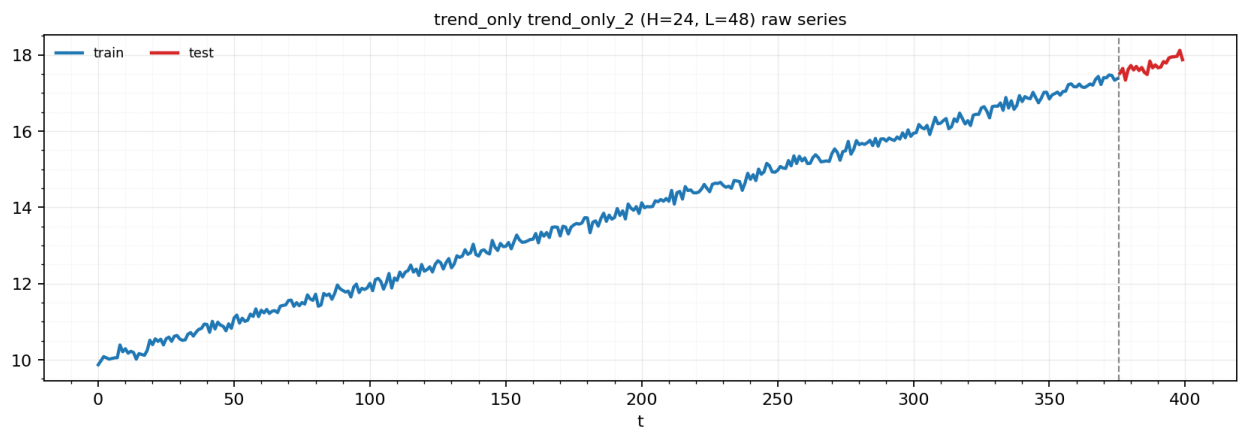


Рисунок 2.2.4. Синтетический ряд *trend_only_2*, разделенный на *test* и *train*.

- *season_only* (рисунок 2.2.5 и 2.2.6)
 - *trend_slope* = 0.0
 - *season_period* = 24
 - *season_amp* = 1.0
 - *noise_std* = 0.1

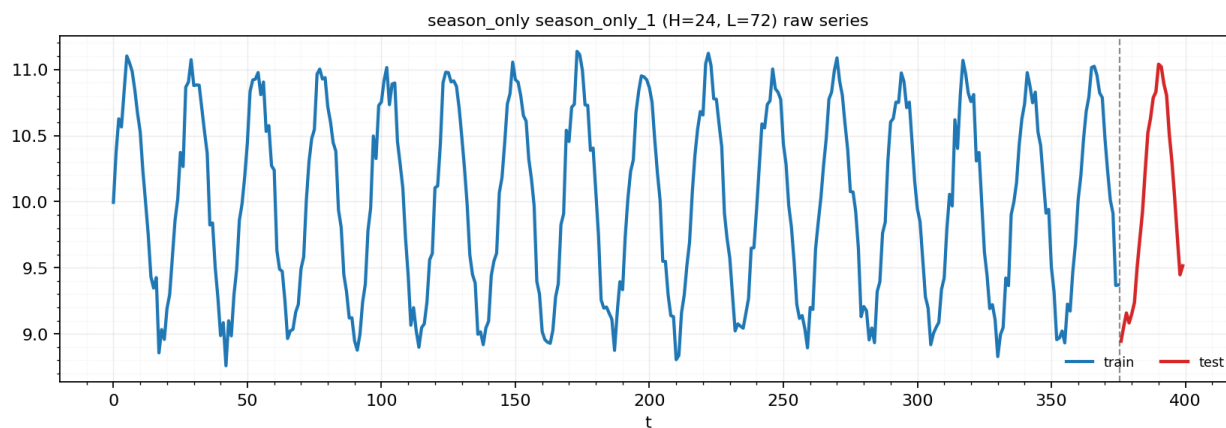


Рисунок 2.2.5. Синтетический ряд *season_only_1*, разделенный на *test* и *train*.

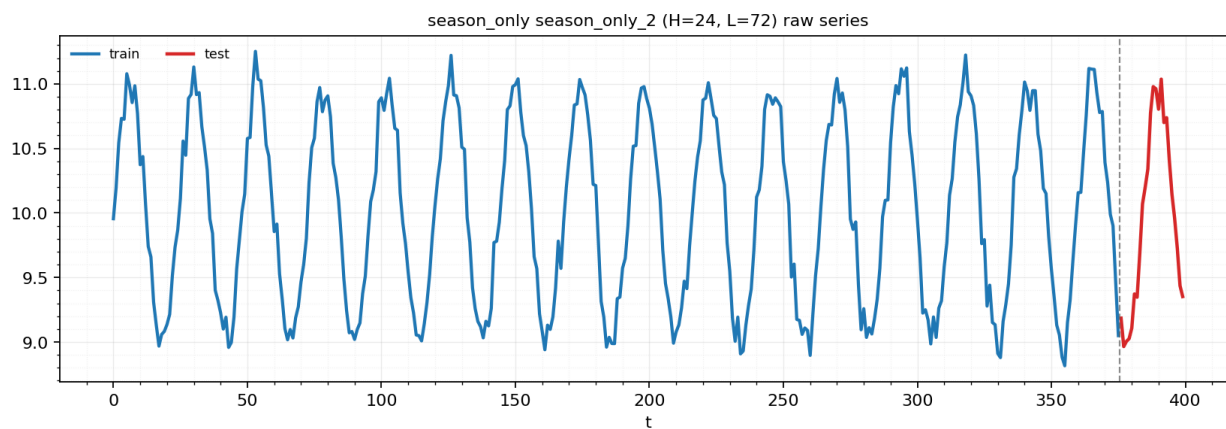


Рисунок 2.2.6. Синтетический ряд *season_only_2*, разделенный на *test* и *train*.

- *trend_season* (рисунок 2.2.7 и 2.2.8)
 - *trend_slope* = 0.02
 - *season_period* = 24
 - *season_amp* = 1.0
 - *noise_std* = 0.1

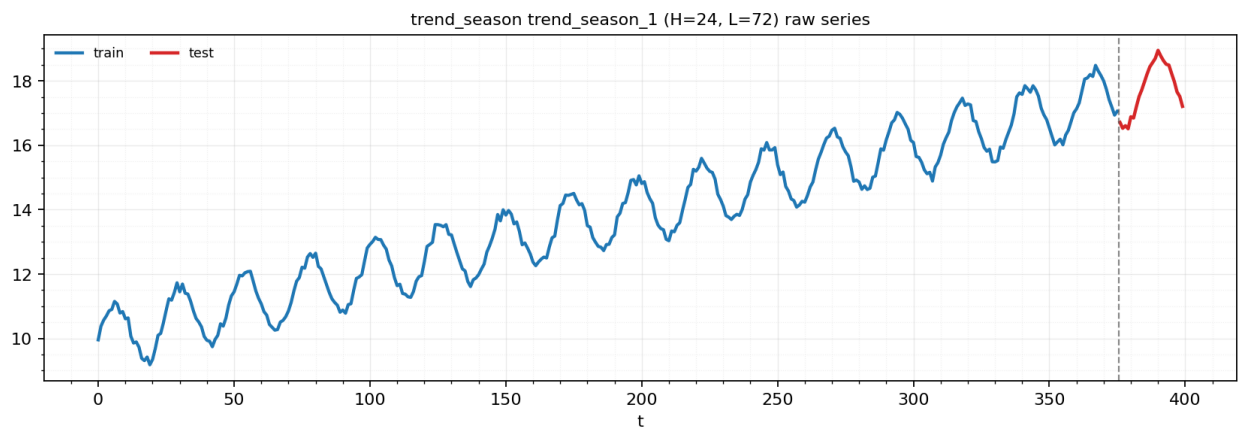


Рисунок 2.2.7. Синтетический ряд *trend_season_1*, разделенный на *test* и *train*.

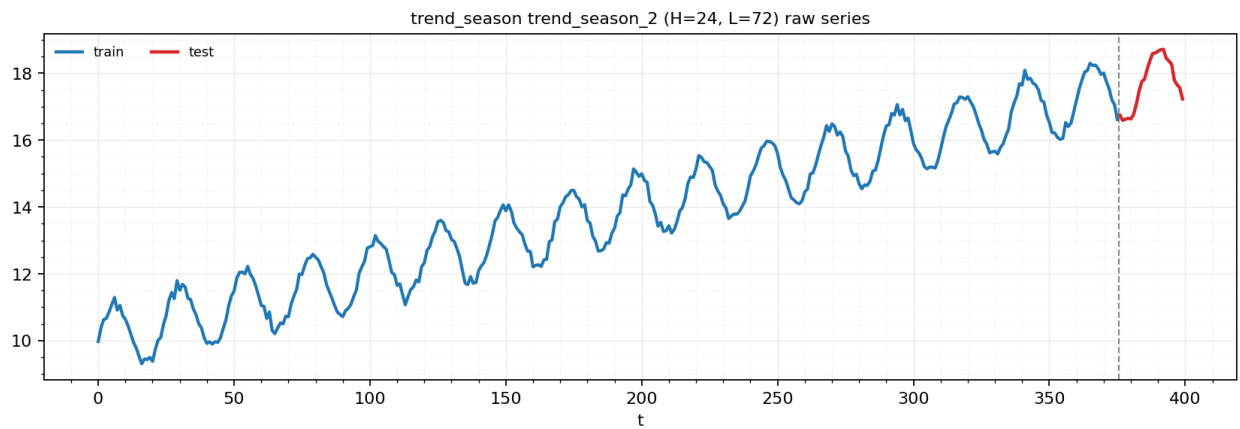


Рисунок 2.2.8. Синтетический ряд *trend_season_2*, разделенный на *test* и *train*.

- *trend_season_high_noise* (рисунок 2.2.9 и 2.2.10)
 - *trend_slope* = 0.02
 - *season_period* = 24
 - *season_amp* = 1.0
 - *noise_std* = 0.5

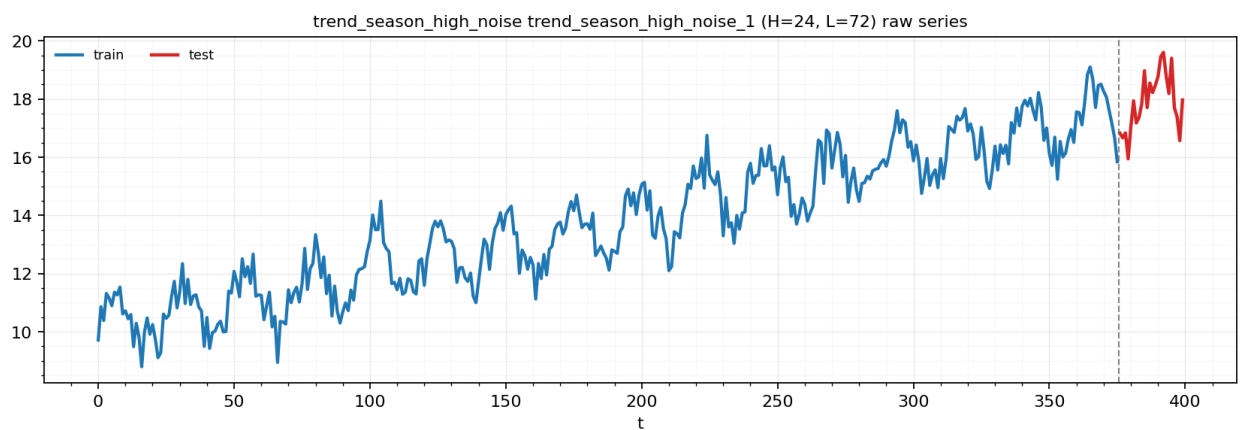


Рисунок 2.2.9. Синтетический ряд *trend_season_high_noise_1*, разделенный на *test* и *train*.

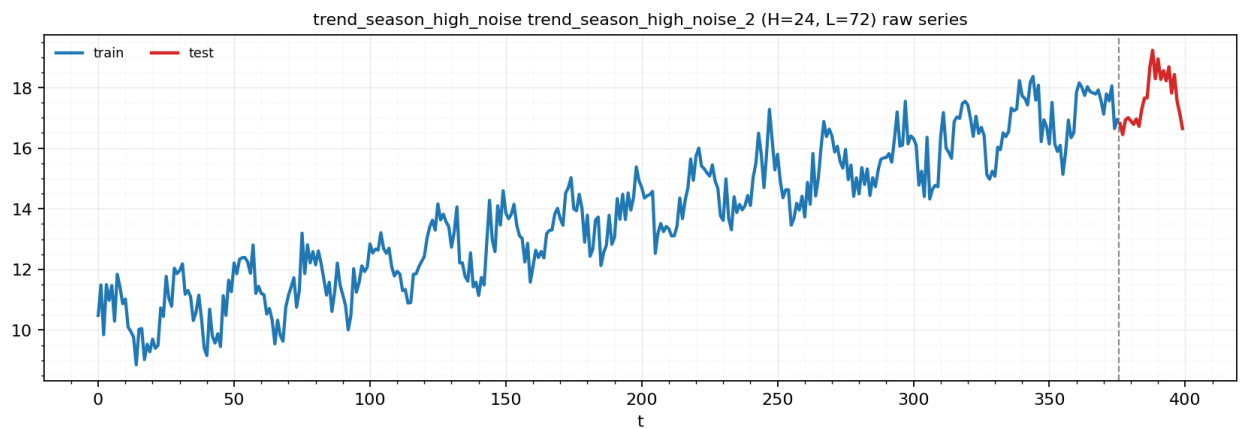


Рисунок 2.2.10. Синтетический ряд *trend_season_high_noise_2*, разделенный на *test* и *train*.

- *season_high_noise* (рисунок 2.2.11 и 2.2.12)
 - *trend_slope* = 0.0
 - *season_period* = 24
 - *season_amp* = 1.0
 - *noise_std* = 0.5

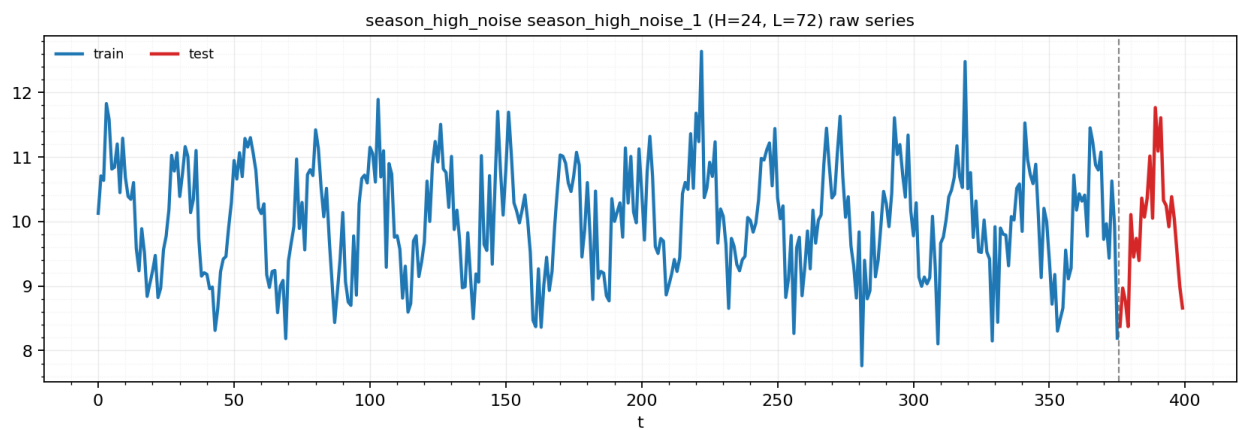


Рисунок 2.2.11. Синтетический ряд *season_high_noise_1*, разделенный на *test* и *train*.

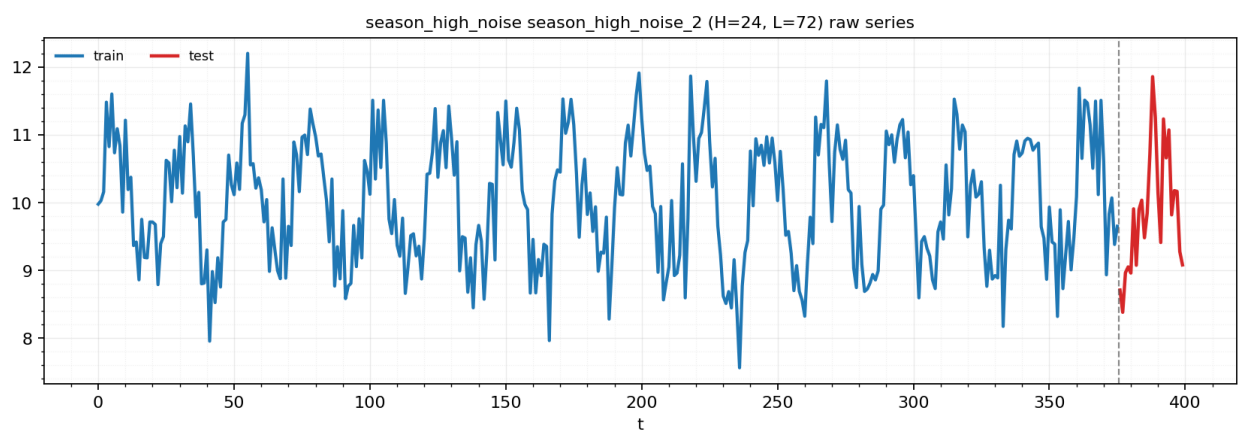


Рисунок 2.2.12. Синтетический ряд *season_high_noise_1*, разделенный на *test* и *train*.

2.2.2. Обучение моделей, подбор параметров ARIMA

Модели были обучены со следующими параметрами обучения:

- горизонт: $horizon = 24$
- $L = best_L(y_{tr}, horizon, per)$, где $per = season_period$ или 1.
- $epochs = 8$
- $batch_size = 64$
- $lr = 5e - 4$
- $weight_decay = 1e - 4$
- $clip = 1.0$

Для каждого ряда были построены графики ACF / PACF, чтобы рассчитать порядки моделей ARIMA для этих рядов. За основу было взято правило:

- $d = 1$ (фиксировано),
- ищем значимые лаги по ACF/PACF только до 24 ($max_lag = 24$),
- ограничиваем $p \leq 3, q \leq 3$,
- p берём по PACF, q по ACF (по максимальному значимому лагу).

Итак, порядки моделей ARIMA для ряда:

- **flat_low_noise_1** (рисунок 2.2.13):

ACF/PACF почти весь в границах $\rightarrow$ ряд близок к белому шуму вокруг уровня $\rightarrow$ ACF/PACF в пределах значимости на лагах $1 \dots 24 \rightarrow p = 0, q = 0 \rightarrow \underline{\text{ARIMA}(0,1,0)}$

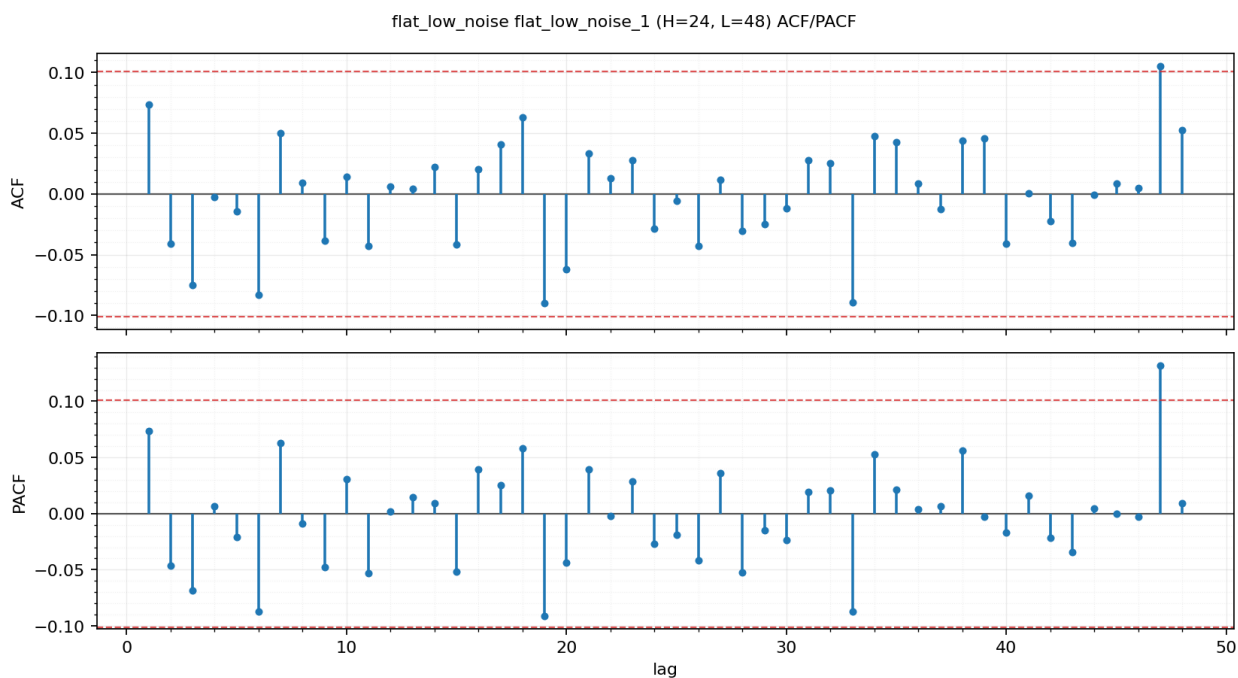


Рисунок 2.2.13. График ACF/PACF (**flat_low_noise_1**) для подбора параметров ARIMA.

- **trend_only_1** (рисунок 2.2.14):

ACF долго остаётся высокой (медленное убывание) → сильная нестационарность, оправдывает $d = 1$; *PACF* выделяет короткую структуру на малых лагах → *PACF*: сильный пик на лаге 1, дальше около нуля → $p \approx 1$; *ACF* “длинная” из-за тренда, но по нашей эвристике q не обязателен → **ARIMA(1,1,0)**

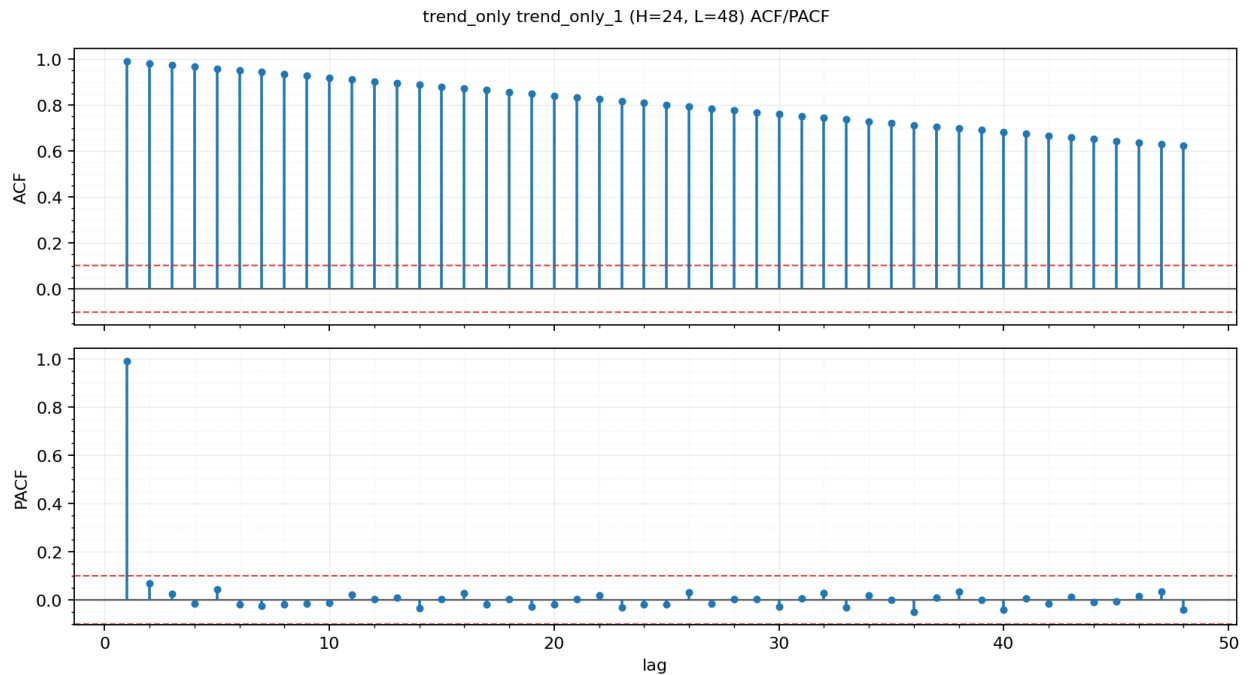


Рисунок 2.2.14. График ACF/PACF (**trend_only_1**) для подбора параметров ARIMA.

- **season_only_1** (рисунок 2.2.15):

ACF выраженно периодична → сезонность с периодом около 24; *PACF* имеет сильный вклад на малых лагах и слабее — на сезонных → *PACF* тоже имеет значимые лаги > 3 → $p = 3$ → **ARIMA(3,1,3)**

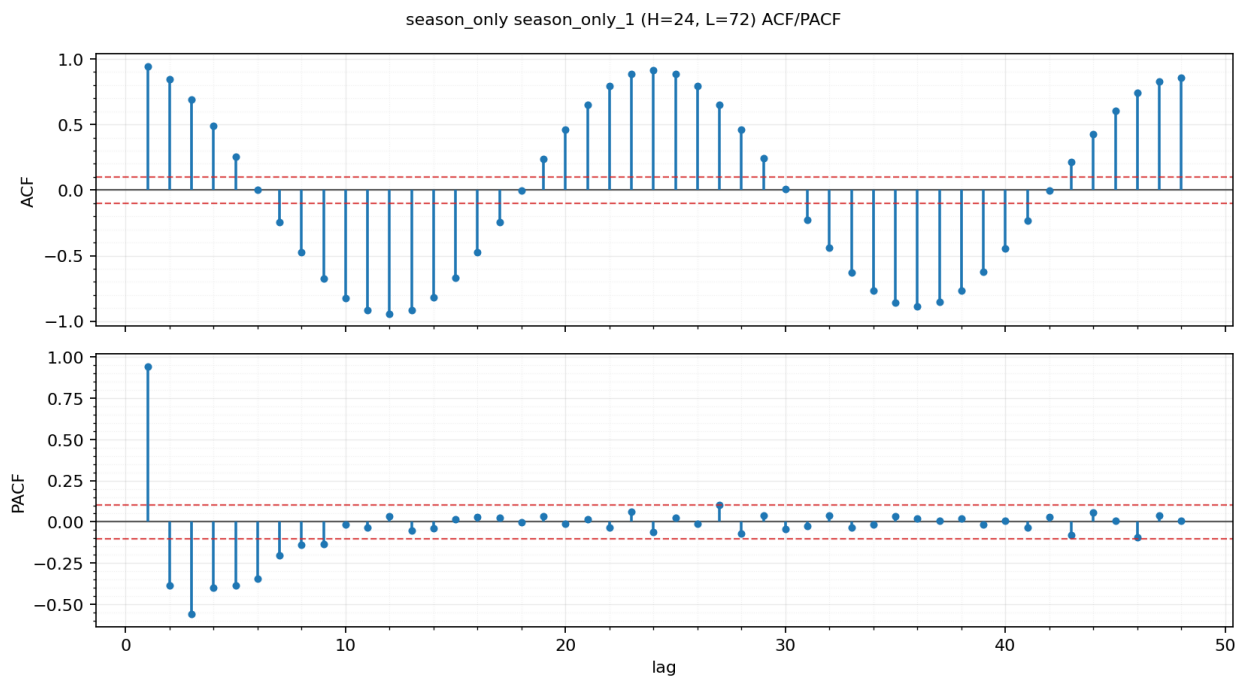


Рисунок 2.2.15. График ACF/PACF (**season_only_1**) для подбора параметров ARIMA.

- **season_high_noise_1 (рисунок 2.2.16)**

тот же сезонный рисунок, но пики размыты шумом (амплитуда корреляций ниже, больше случайных колебаний) → та же сезонность, но шумнее → всё равно есть значимые лаги $> 3 \rightarrow p = q = 3 \rightarrow \underline{\text{ARIMA}(3,1,3)}$

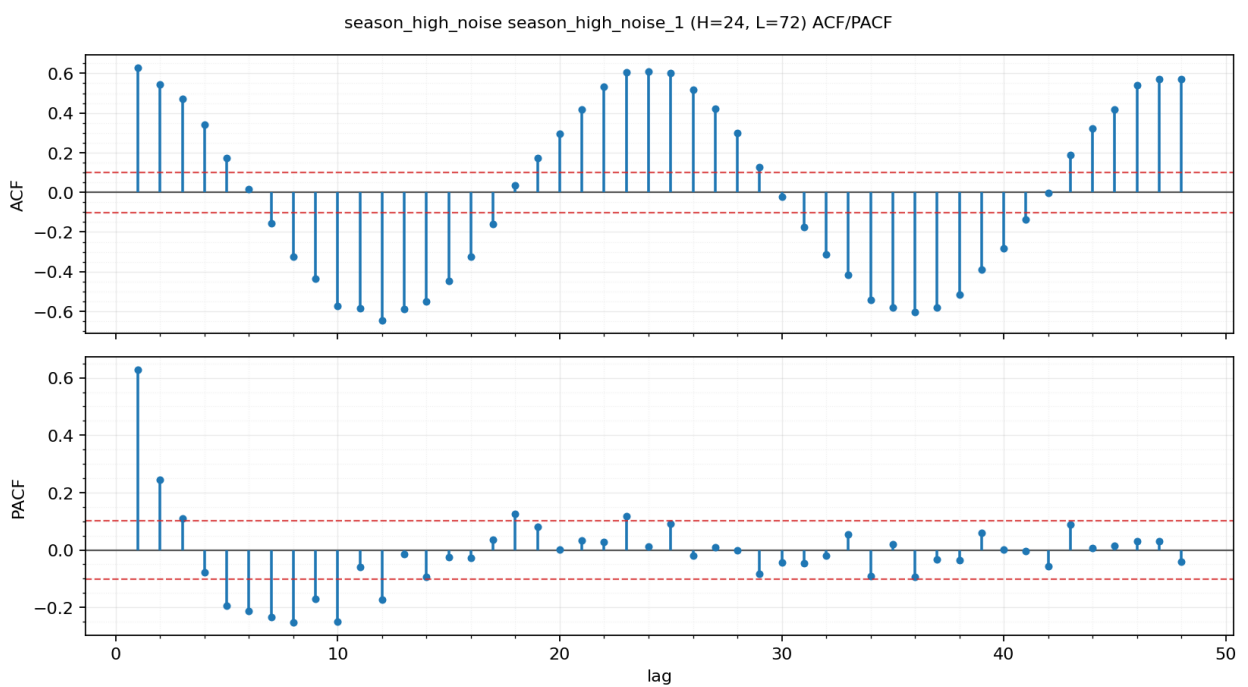


Рисунок 2.2.16. График ACF/PACF (**season_high_noise_1**) для подбора параметров ARIMA.

- **trend_season / trend_season_high_noise (рисунок 2.2.17 и 2.2.18):**

смесь двух эффектов: высокий фон ACF из-за тренда + периодическая структура из-за сезонности; при **high_noise** периодичность менее контрастна → смешение тренда и сезонности даёт значимые лаги по ACF/PACF до 24 → **ARIMA(3,1,3)**

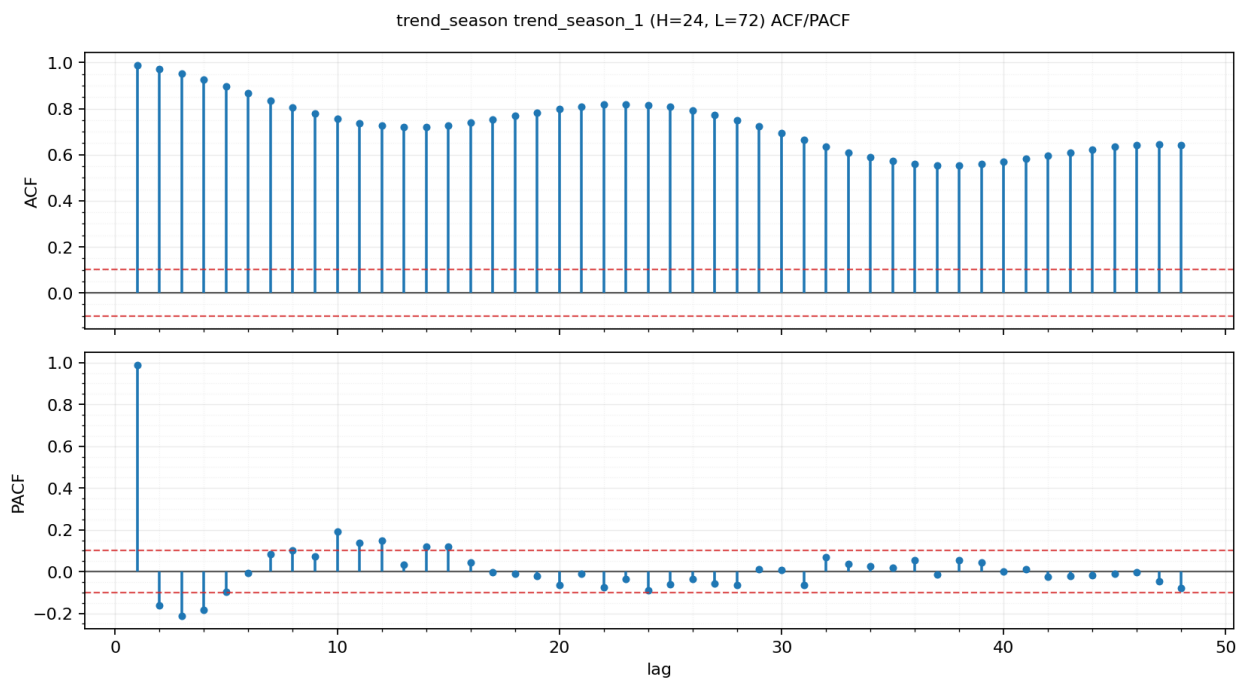


Рисунок 2.2.17. График ACF/PACF (**trend_season_1**) для подбора параметров ARIMA.

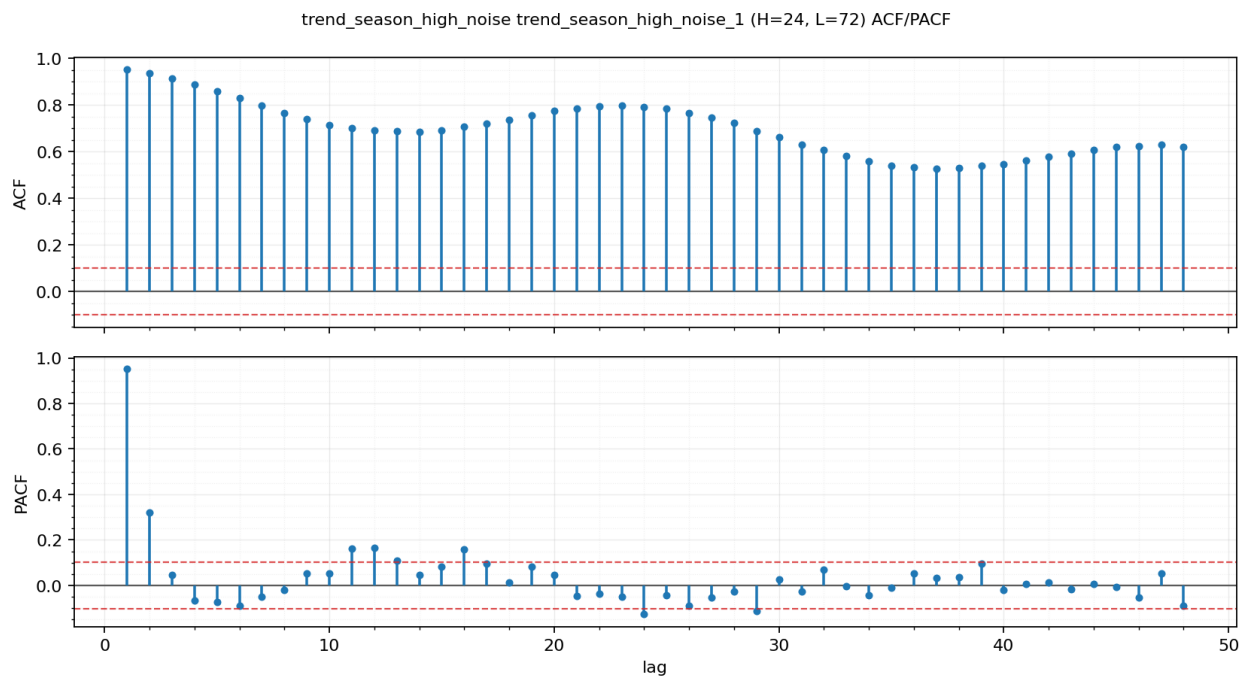


Рисунок 2.2.18. График ACF/PACF (**trend_season_high_noise_1**) для подбора параметров ARIMA.

После подбора параметров для ARIMA приступим к выполнению вейвлет разложения для прогноза нейросетевых моделей. В качестве эксперимента в параметрах *.json* было решено поставить максимальное кол-во компонент D_j равное 5.

Графики разложения на компоненты MODWT для некоторых исходных синтетических рядов показаны на рисунках 2.2.19, 2.2.20, 2.2.21.

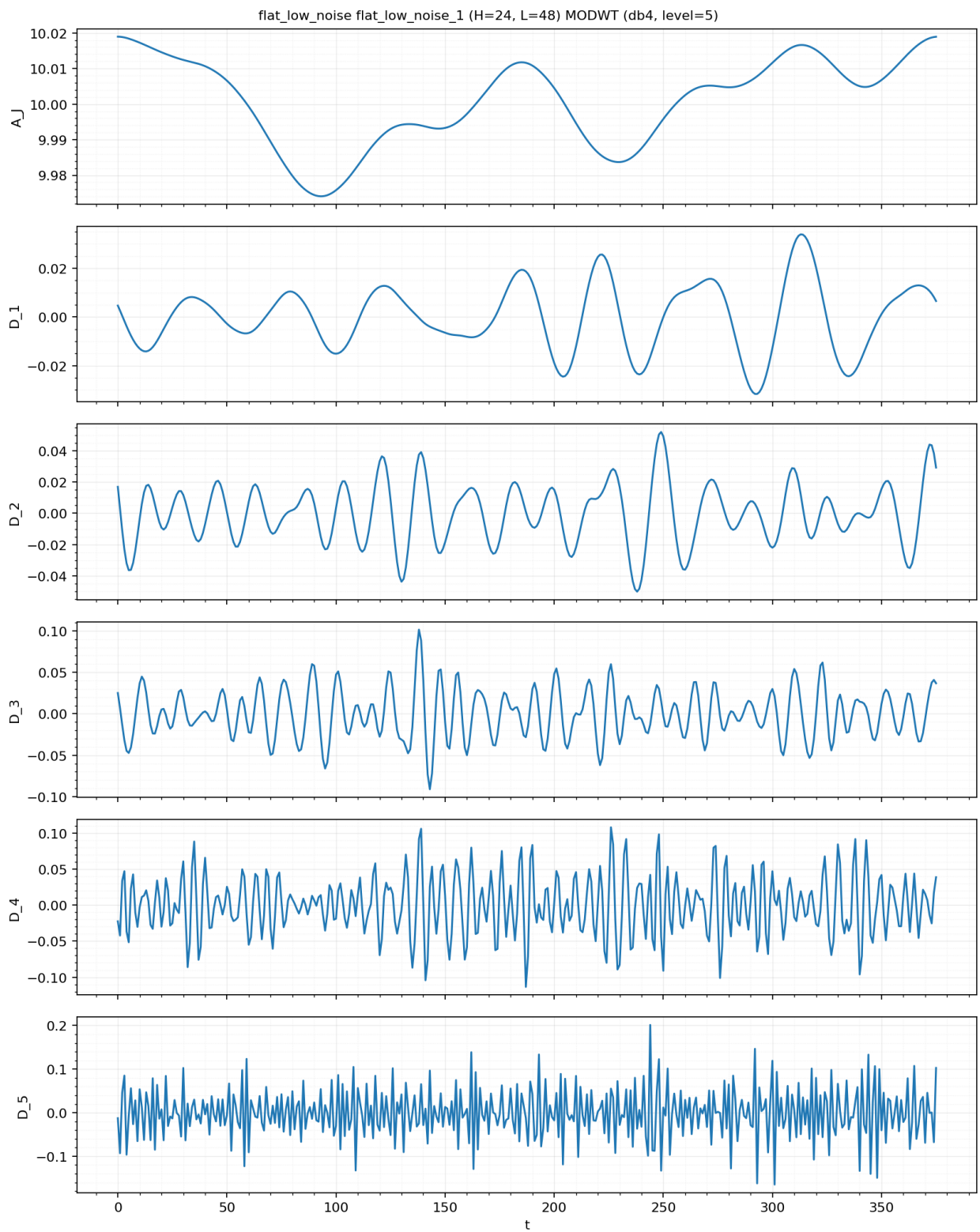


Рисунок 2.2.19. Графики компонент разложения MODWT для синтетического ряда *flat_low_noise_1*.

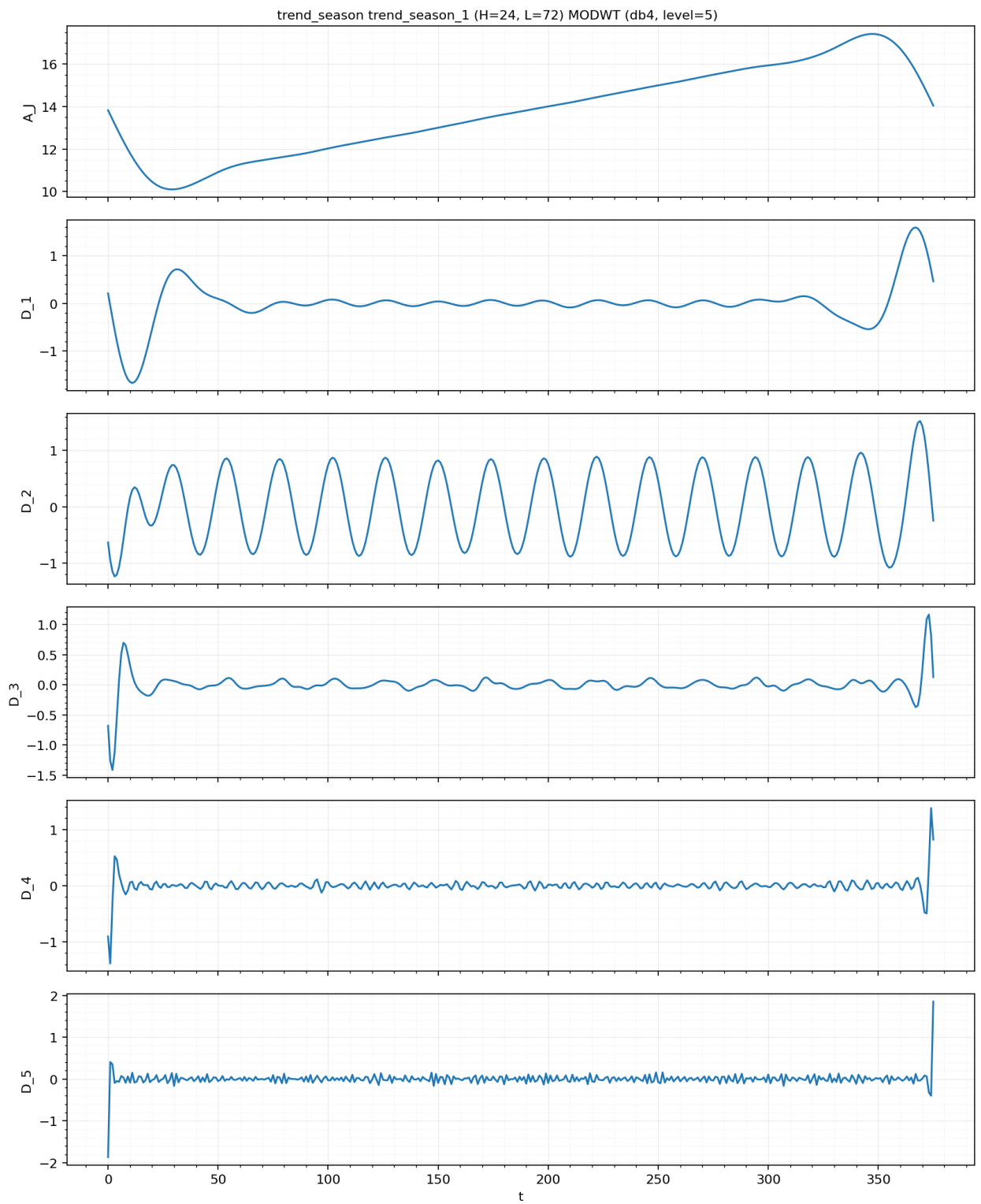


Рисунок 2.2.20. Графики компонент разложения MODWT для синтетического ряда *trend_season_1*.

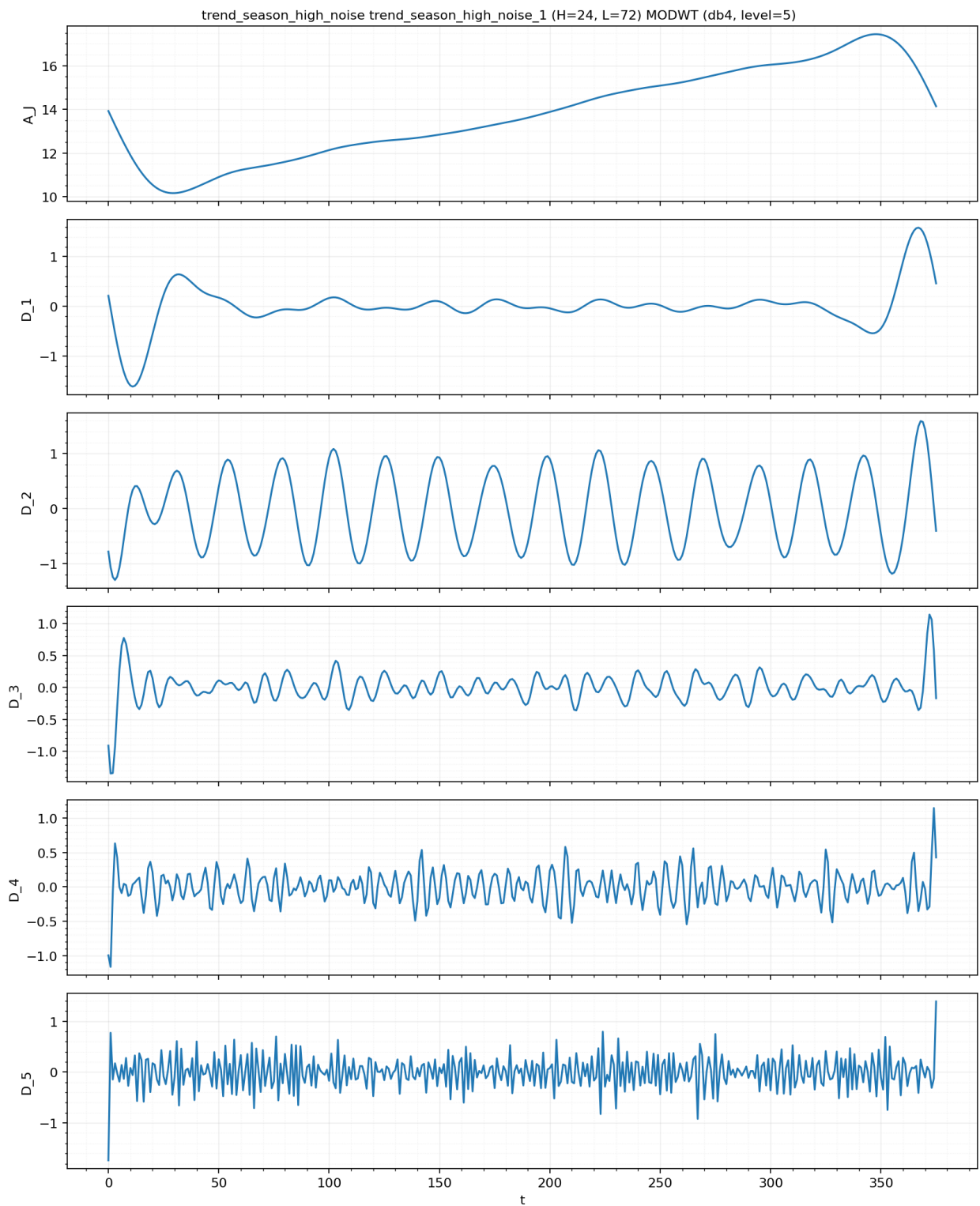


Рисунок 2.2.21. Графики компонент разложения MODWT для синтетического ряда *trend_season_high_noise_1*.

2.2.3. Результаты обучения моделей на синтетических рядах

После применения вейвлет-разложение MODWT на каждую из компонент разложения были натравлены нейросетевые и классические стационарные модели.

Увеличенные графики предсказаний всех реализованных моделей можно увидеть на рисунках 2.2.22 - 2.2.33.

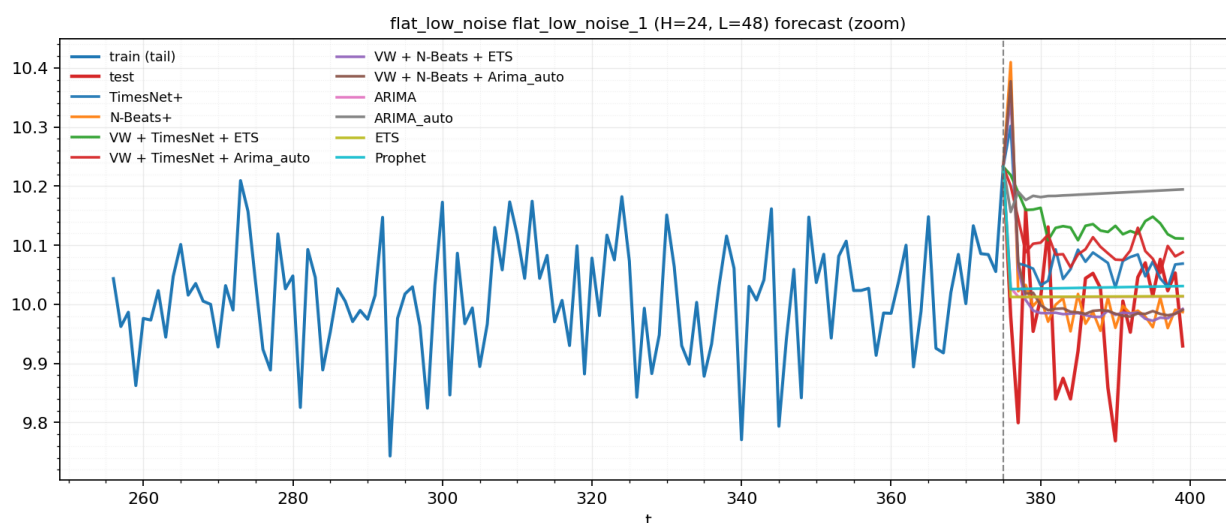


Рисунок 2.2.22. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *flat_low_noise_1*.

Результат по метрикам на *flat_low_noise_1* представлен в таблице 2.2.1.

Таблица 2.2.1. Результат по метрикам на синтетическом ряду *flat_low_noise_1*.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	0.836555	0.840505	0.108148	0.011696
ARIMA_auto	2.088459	2.115946	0.233604	0.054571
ETS	0.829870	0.833827	0.107988	0.011661
Prophet	0.858715	0.863932	0.113912	0.012976
TimesNet+	1.170329	1.180927	0.149854	0.022456
N-Beats+	1.056085	1.062039	0.137006	0.018771
VW_+_TimesNet_+_Arima_auto	1.294616	1.307251	0.159279	0.025370
VW_+_TimesNet_+_ETS	1.617354	1.636114	0.192661	0.037118
VW_+_N-Beats_+_ETS	1.045033	1.050227	0.132477	0.017550
VW_+_N-Beats_+_Arima_auto	1.045484	1.051626	0.136061	0.018513

Обучение и предсказание моделей, результаты на синтетических рядах которых можно увидеть в таблицах 2.2 выполнялось:

- **ARIMA, ARIMA_auto, ETS, Prophet** – на полном синтетическом ряду без использования предсказаний на различных компонентах вейвлет-разложения MODWT

- **TimesNet+** - нейросетевой архитектурой **TimesNet** на всех компонентах вейвлет разложения MODWT, то есть на A_j и D_j , где $j \in \{1,2,3,4,5\}$
- **N-Beats+** - нейросетевой архитектурой **N-Beats** на всех компонентах вейвлет разложения MODWT, то есть на A_j и D_j , где $j \in \{1,2,3,4,5\}$
- **VW + TimesNet + ARIMA_auto** - нейросетевой архитектурой **TimesNet** на компоненте A_j вейвлет-разложения MODWT, а обучение на D_j , где $j \in \{1,2,3,4,5\}$ выполнялось с помощью **ARIMA_auto** (**ARIMA с автоподбором**)
- **VW + TimesNet + ETS** - нейросетевой архитектурой **TimesNet** на компоненте A_j вейвлет-разложения MODWT, а обучение на D_j , где $j \in \{1,2,3,4,5\}$ выполнялось с помощью **ETS**
- **VW + N-Beats + ARIMA_auto** - нейросетевой архитектурой **N-Beats** на компоненте A_j вейвлет-разложения MODWT, а обучение на D_j , где $j \in \{1,2,3,4,5\}$ выполнялось с помощью **ARIMA_auto** (**ARIMA с автоподбором**)
- **VW + N-Beats + ETS** - нейросетевой архитектурой **N-Beats** на компоненте A_j вейвлет-разложения MODWT, а обучение на D_j , где $j \in \{1,2,3,4,5\}$ выполнялось с помощью **ETS**

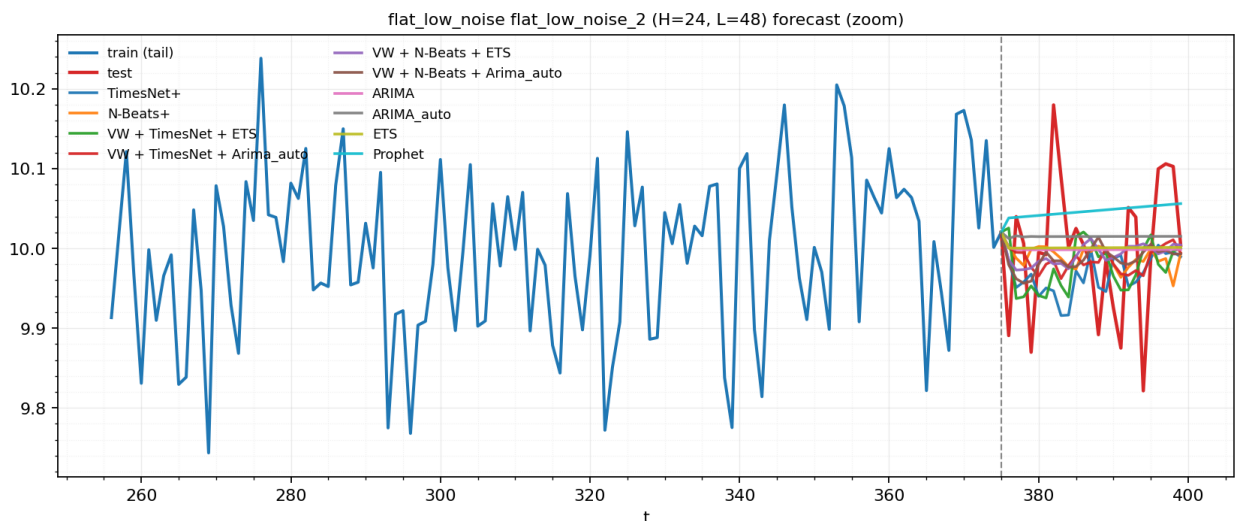


Рисунок 2.2.23. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *flat_low_noise_2*.

Результат по метрикам на *flat_low_noise_2* представлен в таблице 2.2.2.

Таблица 2.2.2. Результат по метрикам на синтетическом ряду *flat_low_noise_2*.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	0.642060	0.642375	0.085027	0.007230
ARIMA_auto	0.662182	0.663544	0.086320	0.007451
ETS	0.638765	0.639220	0.084885	0.007206
Prophet	0.761053	0.764709	0.097350	0.009477

<i>TimesNet+</i>	<i>0.822670</i>	<i>0.820628</i>	<i>0.096815</i>	<i>0.009373</i>
<i>N-Beats+</i>	<i>0.702548</i>	<i>0.701988</i>	<i>0.088662</i>	<i>0.007861</i>
<i>VW_+_TimesNet_+_Arima_auto</i>	<i>0.658869</i>	<i>0.658193</i>	<i>0.082762</i>	<i>0.006850</i>
<i>VW_+_TimesNet_+_ETS</i>	<i>0.802804</i>	<i>0.801407</i>	<i>0.095938</i>	<i>0.009204</i>
<i>VW_+_N-Beats_+_ETS</i>	<i>0.668432</i>	<i>0.668278</i>	<i>0.086688</i>	<i>0.007515</i>
<i>VW_+_N-Beats_+_Arima_auto</i>	<i>0.704451</i>	<i>0.703964</i>	<i>0.086904</i>	<i>0.007552</i>

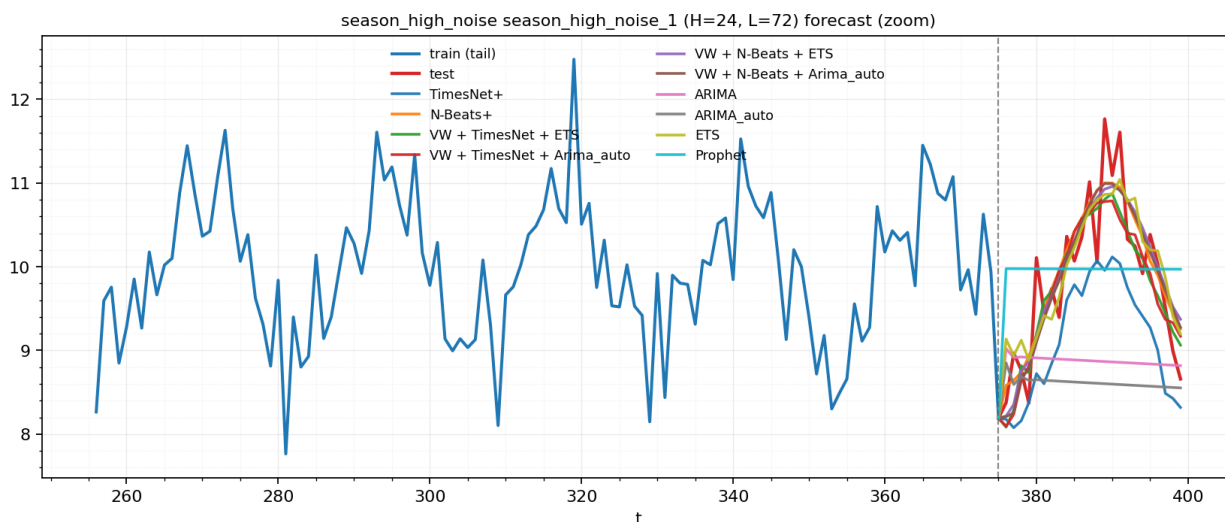


Рисунок 2.2.24. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *season_high_noise_1*.

Результат по метрикам на *season_high_noise_1* представлен в таблице 2.2.3.

Таблица 2.2.3. Результат по метрикам на синтетическом ряду *season_high_noise_1*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	<i>11.865720</i>	<i>11.011854</i>	<i>1.379249</i>	<i>1.902328</i>
<i>ARIMA_auto</i>	<i>14.128892</i>	<i>12.933650</i>	<i>1.586317</i>	<i>2.516401</i>
<i>ETS</i>	<i>4.077759</i>	<i>4.101361</i>	<i>0.474430</i>	<i>0.225084</i>
<i>Prophet</i>	<i>7.317218</i>	<i>7.453695</i>	<i>0.914931</i>	<i>0.837098</i>
<i>TimesNet+</i>	<i>7.817819</i>	<i>7.817819</i>	<i>0.879595</i>	<i>0.773688</i>
<i>N-Beats+</i>	<i>3.694401</i>	<i>3.713599</i>	<i>0.450041</i>	<i>0.202537</i>
<i>VW_+_TimesNet_+_Arima_auto</i>	<i>4.137991</i>	<i>4.088963</i>	<i>0.503489</i>	<i>0.253502</i>
<i>VW_+_TimesNet_+_ETS</i>	<i>3.650136</i>	<i>3.601622</i>	<i>0.459195</i>	<i>0.210860</i>
<i>VW_+_N-Beats_+_ETS</i>	<i>3.922336</i>	<i>3.930914</i>	<i>0.473843</i>	<i>0.224527</i>
<i>VW_+_N-Beats_+_Arima_auto</i>	<i>3.929391</i>	<i>3.926171</i>	<i>0.470079</i>	<i>0.220974</i>

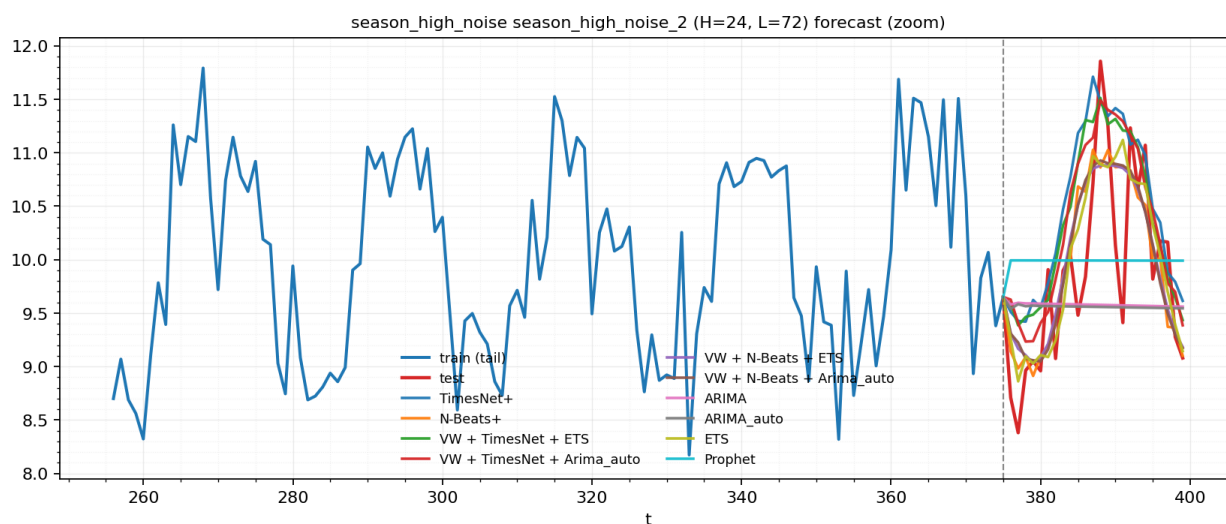


Рисунок 2.2.25. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *season_high_noise_2*.

Результат по метрикам на *season_high_noise_2* представлен в таблице 2.2.4.

Таблица 2.2.4. Результат по метрикам на синтетическом ряду *season_high_noise_2*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	7.628257	7.424214	0.935993	0.876082
ARIMA_auto	7.648907	7.429518	0.941362	0.886163
ETS	4.392476	4.472230	0.585188	0.342445
Prophet	7.317510	7.419477	0.888493	0.789420
TimesNet+	6.869995	7.236861	0.852859	0.727369
N-Beats+	4.368829	4.442695	0.587221	0.344829
VW_+_TimesNet_+_Arima_auto	5.876639	6.145952	0.745971	0.556473
VW_+_TimesNet_+_ETS	6.093863	6.385576	0.763457	0.582867
VW_+_N-Beats_+_ETS	4.618860	4.701917	0.595552	0.354683
VW_+_N-Beats_+_Arima_auto	4.621963	4.706317	0.601466	0.361761

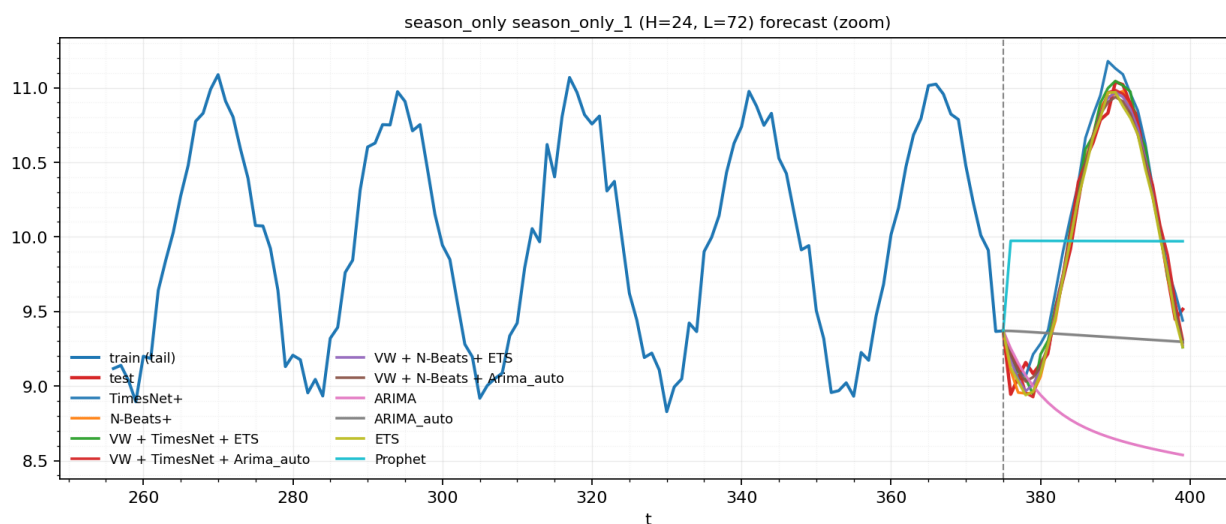


Рисунок 2.2.26. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *season_only_1*.

Результат по метрикам на *season_only_1* представлен в таблице 2.2.5.

Таблица 2.2.5. Результат по метрикам на синтетическом ряду *season_only_1*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	13.436836	12.324209	1.508205	2.274682
ARIMA_auto	8.059397	7.636083	0.976033	0.952639
ETS	0.796059	0.793121	0.101818	0.010367
Prophet	6.312529	6.323611	0.700451	0.490631
TimesNet+	1.258303	1.268393	0.146777	0.021544
N-Beats+	0.744563	0.742307	0.093029	0.008654
VW_+_TimesNet_+_Arima_auto	0.872336	0.873493	0.110474	0.012204
VW_+_TimesNet_+_ETS	0.859795	0.861606	0.111366	0.012402
VW_+_N-Beats_+_ETS	0.664810	0.663817	0.090912	0.008265
VW_+_N-Beats_+_Arima_auto	0.666604	0.666519	0.091541	0.008380

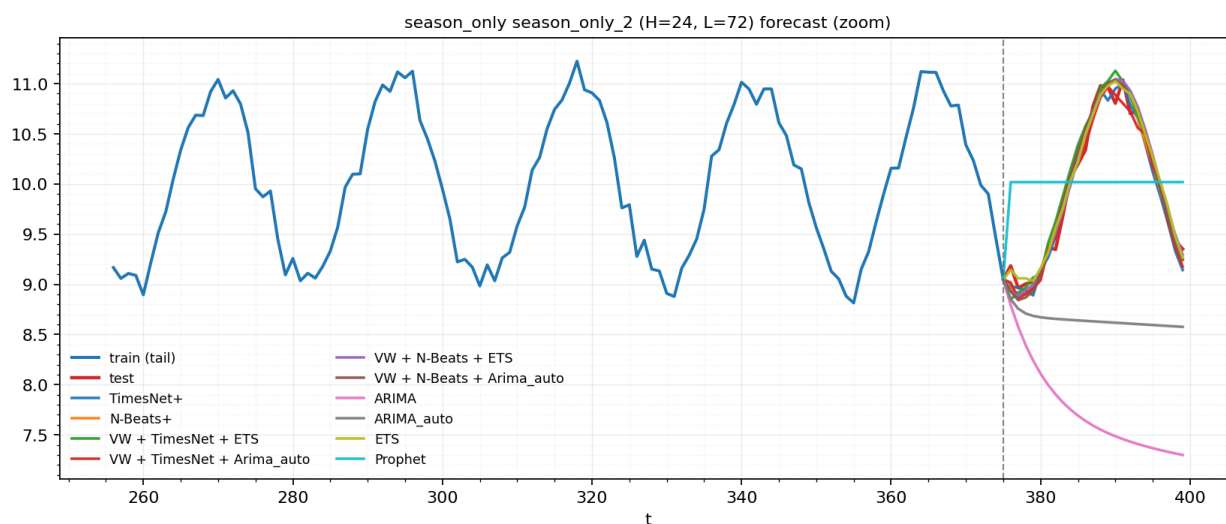


Рисунок 2.2.27. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *season_only_2*.

Результат по метрикам на *season_only_2* представлен в таблице 2.2.6.

Таблица 2.2.6. Результат по метрикам на синтетическом ряду *season_only_2*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	25.200972	21.938072	2.466829	6.085245
ARIMA_auto	14.016508	12.874925	1.517973	2.304242
ETS	0.817562	0.821187	0.101259	0.010253
Prophet	6.270134	6.327025	0.702060	0.492889
TimesNet+	0.882230	0.879872	0.104741	0.010971
N-Beats+	0.929066	0.931508	0.118902	0.014138
VW_+_TimesNet_+_Arima_auto	0.911062	0.907472	0.105828	0.011200
VW_+_TimesNet_+_ETS	1.074253	1.078551	0.146648	0.021506
VW_+_N-Beats_+_ETS	0.914075	0.915725	0.115134	0.013256
VW_+_N-Beats_+_Arima_auto	1.012931	1.014319	0.124594	0.015524

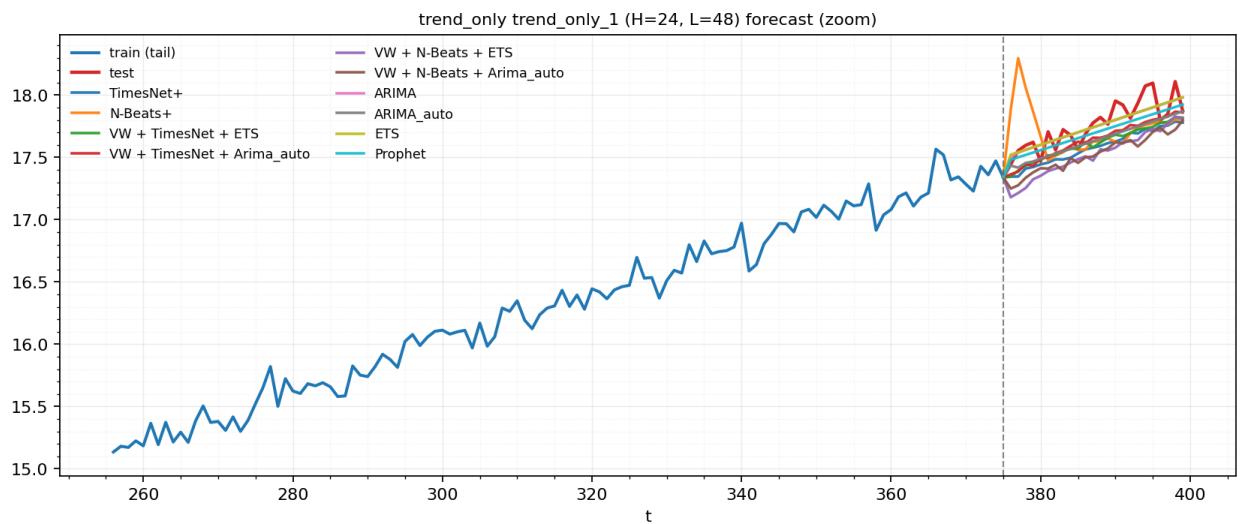


Рисунок 2.2.28. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_only_1*.

Результат по метрикам на *trend_only_1* представлен в таблице 2.2.7.

Таблица 2.2.7. Результат по метрикам на синтетическом ряду *trend_only_1*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	0.470511	0.470053	0.099961	0.009992
ARIMA_auto	0.738911	0.734860	0.161109	0.025956
ETS	0.473289	0.472894	0.100151	0.010030
Prophet	0.551199	0.549368	0.118852	0.014126
TimesNet+	1.043645	1.036692	0.210142	0.044160
N-Beats+	1.238910	1.241038	0.274336	0.075260
VW_+_TimesNet_+_Arima_auto	0.744216	0.740373	0.157710	0.024873
VW_+_TimesNet_+_ETS	0.859750	0.854601	0.181181	0.032827
VW_+_N-Beats_+_ETS	1.363194	1.352424	0.260214	0.067711
VW_+_N-Beats_+_Arima_auto	1.336405	1.325827	0.258629	0.066889

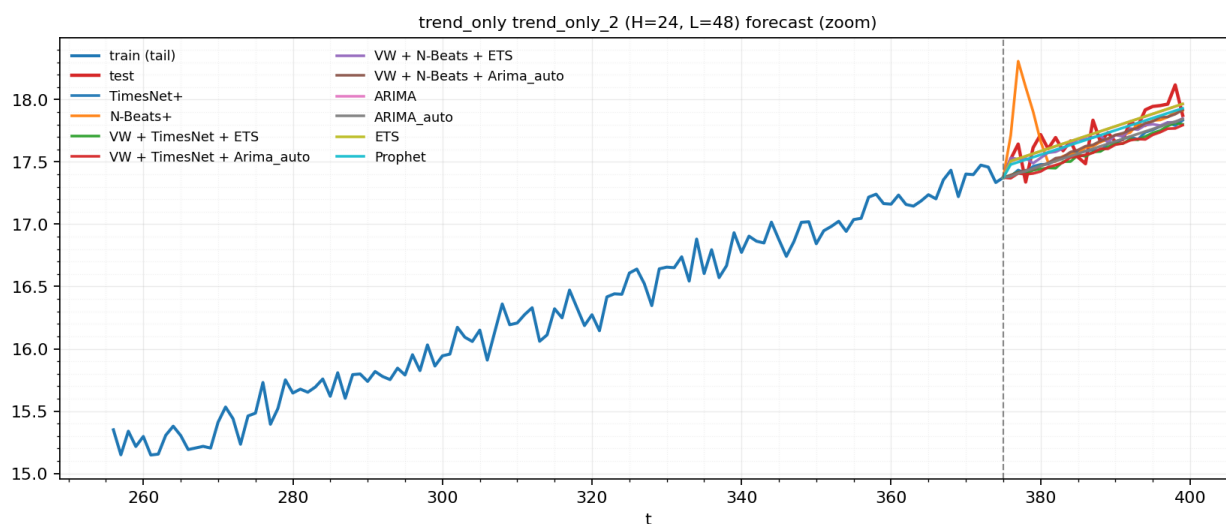


Рисунок 2.2.29. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_only_2*.

Результат по метрикам на *trend_only_2* представлен в таблице 2.2.8.

Таблица 2.2.8. Результат по метрикам на синтетическом ряду *trend_only_2*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	0.475122	0.475641	0.103651	0.010744
ARIMA_auto	0.718752	0.715461	0.149497	0.022349
ETS	0.474675	0.475229	0.103774	0.010769
Prophet	0.501094	0.500787	0.104917	0.011008
TimesNet+	0.731585	0.728228	0.150622	0.022687
N-Beats+	0.877073	0.883450	0.240196	0.057694
VW_+_TimesNet_+_Arima_auto	0.867801	0.862912	0.177851	0.031631
VW_+_TimesNet_+_ETS	0.829911	0.825532	0.168207	0.028293
VW_+_N-Beats_+_ETS	0.584915	0.583778	0.126511	0.016005
VW_+_N-Beats_+_Arima_auto	0.620622	0.618515	0.128845	0.016601

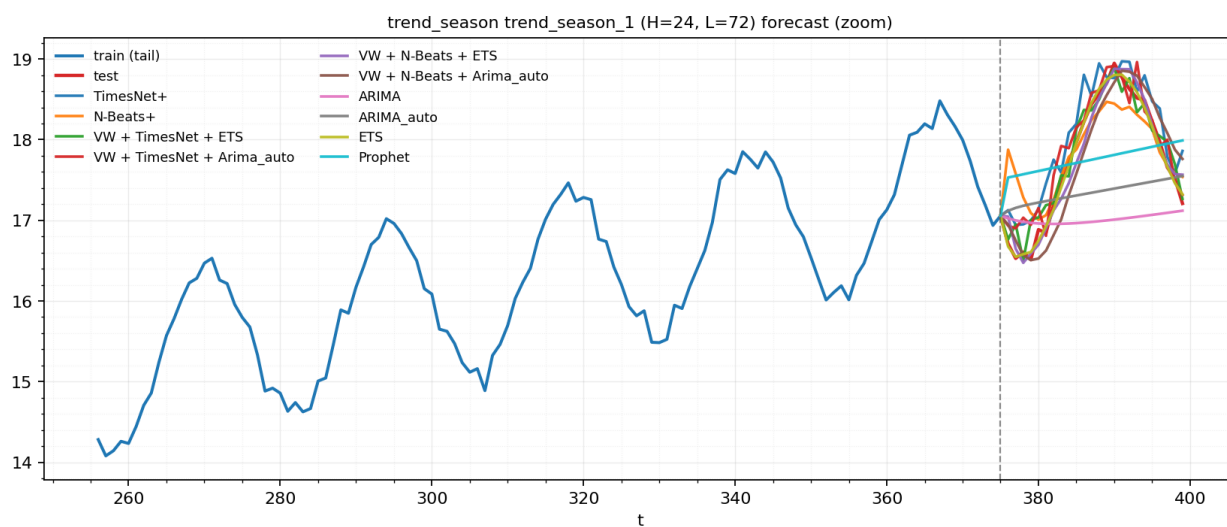


Рисунок 2.2.30. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_season_1*

Результат по метрикам на *trend_season_1* представлен в таблице 2.2.9.

Таблица 2.2.9. Результат по метрикам на синтетическом ряду *trend_season_1*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	5.120800	4.953092	1.079501	1.165323
ARIMA_auto	3.966768	3.888523	0.825992	0.682263
ETS	0.347141	0.346958	0.072911	0.005316
Prophet	3.570157	3.578107	0.707299	0.500273
TimesNet+	1.766336	1.787086	0.359950	0.129564
N-Beats+	1.686828	1.707418	0.419241	0.175763
VW_+_TimesNet_+_Arima_auto	1.110489	1.119779	0.245792	0.060414
VW_+_TimesNet_+_ETS	0.859852	0.864611	0.194554	0.037851
VW_+_N-Beats_+_ETS	0.807019	0.807175	0.175813	0.030910
VW_+_N-Beats_+_Arima_auto	1.567357	1.566031	0.307988	0.094857

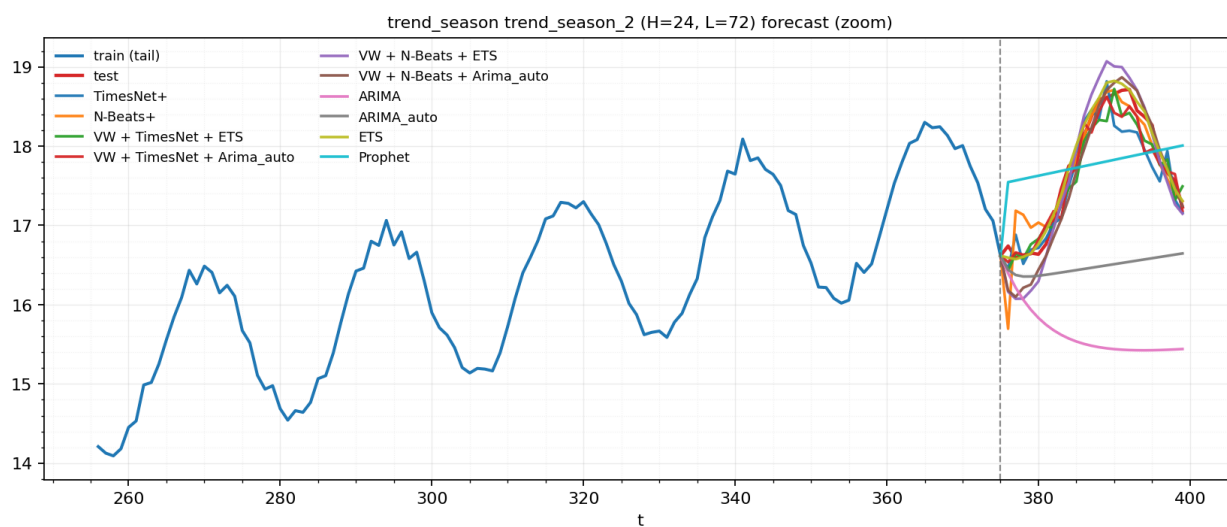


Рисунок 2.2.31. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_season_2*

Результат по метрикам на *trend_season_2* представлен в таблице 2.2.10.

Таблица 2.2.10. Результат по метрикам на синтетическом ряду *trend_season_2*.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	12.514492	11.638625	2.318877	5.377191
ARIMA_auto	7.120748	6.803974	1.422296	2.022925
ETS	0.509742	0.510841	0.107162	0.011484
Prophet	3.462113	3.480792	0.680201	0.462673
TimesNet+	1.344236	1.335248	0.285909	0.081744
N-Beats+	1.145072	1.142312	0.305738	0.093476
VW_+_TimesNet_+_Arima_auto	0.879018	0.875003	0.197811	0.039129
VW_+_TimesNet_+_ETS	1.097742	1.093293	0.216282	0.046778
VW_+_N-Beats_+_ETS	1.507360	1.500890	0.305198	0.093146
VW_+_N-Beats_+_Arima_auto	1.180386	1.171209	0.254134	0.064584

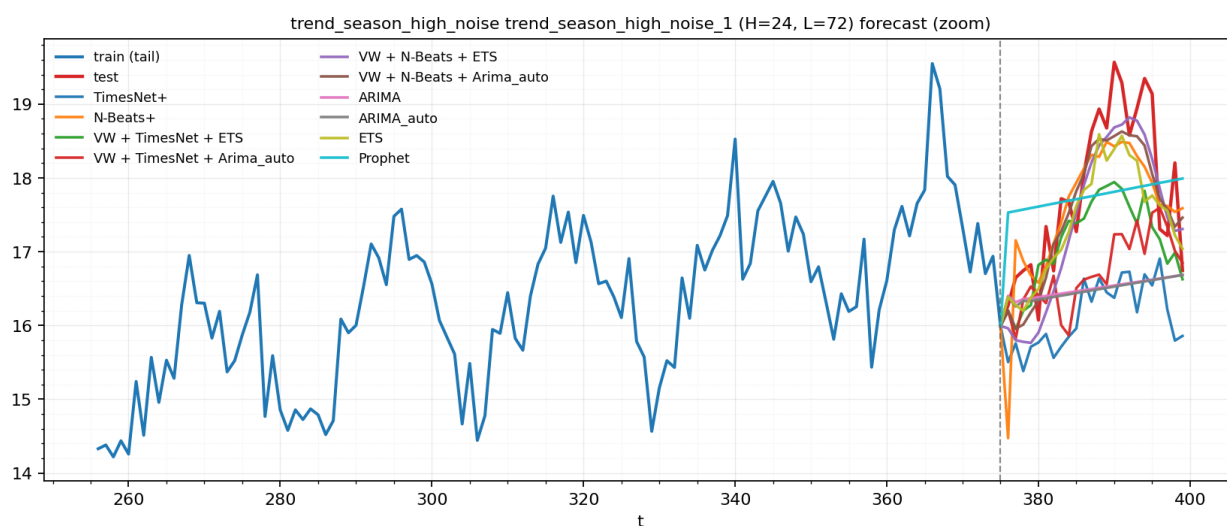


Рисунок 2.2.32. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_season_high_noise_1*

Результат по метрикам на *trend_season_high_noise_1* представлен в таблице 2.2.11.

Таблица 2.2.11. Результат по метрикам на синтетическом ряду *trend_season_high_noise_1*.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	9.080515	8.582188	1.810956	3.279561
ARIMA_auto	9.542449	9.000803	1.882844	3.545102
ETS	2.356727	2.343414	0.543239	0.295109
Prophet	4.239536	4.242980	0.908992	0.826267
TimesNet+	6.232668	6.003897	1.219147	1.486320
N-Beats+	3.060538	2.980803	0.780690	0.609477
VW_+_TimesNet_+_Arima_auto	6.579469	6.331744	1.293264	1.672531
VW_+_TimesNet_+_ETS	7.859877	7.487894	1.572457	2.472622
VW_+_N-Beats_+_ETS	2.752962	2.748248	0.572140	0.327344
VW_+_N-Beats_+_Arima_auto	2.849025	2.837588	0.581073	0.337645

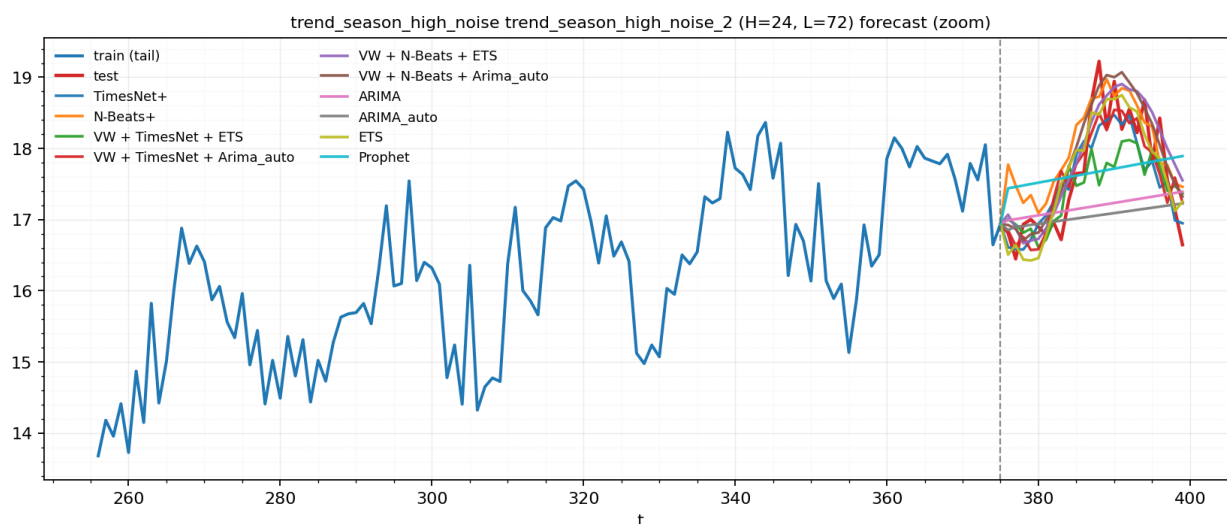


Рисунок 2.2.33. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_season_high_noise_2*

Результат по метрикам на *trend_season_high_noise_2* представлен в таблице 2.2.12.

Таблица 2.2.12. Результат по метрикам на синтетическом ряду *trend_season_high_noise_2*.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	3.922380	3.821047	0.891596	0.794943
ARIMA_auto	4.145259	4.011928	0.974209	0.949083
ETS	1.951594	1.951635	0.410019	0.168116
Prophet	3.723458	3.723383	0.759353	0.576616
TimesNet+	1.902403	1.888094	0.451589	0.203933
N-Beats+	2.901600	2.947042	0.577199	0.333158
VW_+_TimesNet_+_Arima_auto	1.948254	1.946868	0.423297	0.179181
VW_+_TimesNet_+_ETS	2.431984	2.397821	0.593411	0.352137
VW_+_N-Beats_+_ETS	2.075400	2.097240	0.433286	0.187737
VW_+_N-Beats_+_Arima_auto	2.067213	2.092227	0.432960	0.187455

В среднем по всем синтетическим профилям лучшей и самой стабильной моделью оказался **ETS** (минимальные sMAPE/MAPE и RMSE/MSE), потому его аддитивная тренд + сезонная структура совпадает с генератором синтетики и он устойчив при малом объеме данных, а среди гибридов лидируют **VW + N-Beats + ETS** и **VW + N-Beats + ARIMA_auto**, которые удерживают низкую абсолютную ошибку за счёт разделения низко- и высокочастотных компонент. При этом **ARIMA/ARIMA_auto** без сезонной части резко деградируют на рядах с периодом 24 (вплоть до $RMSE > 1$ и очень больших sMAPE/MAPE), тогда как нейросетевые гибриды и ETS сохраняют приемлемое качество даже при высоком шуме.

2.3. Тестирование моделей на наборе данных МЗ

2.3.1. Описание набора данных

МЗ — классический бенчмарк из **3003 реальных временных рядов** из областей *micro, industry, macro, finance, demographic, other*

Ряды имеют частоты *yearly (645 рядов), quarterly (756 рядов), monthly (1428 рядов), other (174 ряда)*.

В постановке МЗ прогноз делается на фиксированные горизонты, а качество считается по отложенному хвосту ряда — именно такие горизонты мы будем брать при обучении:

- **Yearly:** горизонт $h = 6$
- **Quarterly:** $h = 8$
- **Monthly:** $h = 18$
- **Other:** $h = 8$

Чтобы обеспечить достаточную историю, в МЗ задавались минимальные длины рядов (типично: **14** для *yearly*, **16** для *quarterly*, **48** для *monthly*, **60** для *other*); в Монэше также приводятся характерные диапазоны длин по частотам (например, *monthly* до 144 наблюдений).

2.3.2. Обучение на МЗ

Для ускорения обучения, я решил исключить из рассмотрения прогноз гибридных модели, которые на всех компонентах вейвлета делали предсказания нейросетевой архитектурой (TimesNet+, N-beats+) - оценивались гибридные модели семейства **VW: VW + TimesNet + ETS, VW + TimesNet + ARIMA_auto, VW + N-Beats + ETS, VW + N-Beats + ARIMA_auto**.

Для каждой категории (**yearly, quarterly, monthly**) случайно выбиралось **по 100 рядов** (*pick = random, seed = 42*) и для каждого ряда использовался стандартный разрез МЗ: модель обучалась на тренировочной части y_{tr} , а качество считалось на отложенном хвосте длиной $H = M3_H[category]$; сезонный период задавался как $P = M3_P[category]$.

Длина входного окна подбиралась автоматически: $L = best_L(y_{tr}, H, P)$.

Для всех гибридов применялось вейвлет-разложение **MODWT** с параметрами:

- *wavelet* = "db4"
- *boundary* = "wrap"
- *level* = 3 (для **quarterly** принудительно использовался *level* = 2, для **monthly** — *level* = 3)

Обучение нейросетевой части выполнялось **20 эпох** при:

- *batch_size* = 64
- *lr* = $3e - 4$
- *weight_decay* = $2e - 4$

- $clip = 0.5$

Рассмотрим обучение на примере ряда T1079 из категории monthly, график которого изображен на рисунке 2.3.1.

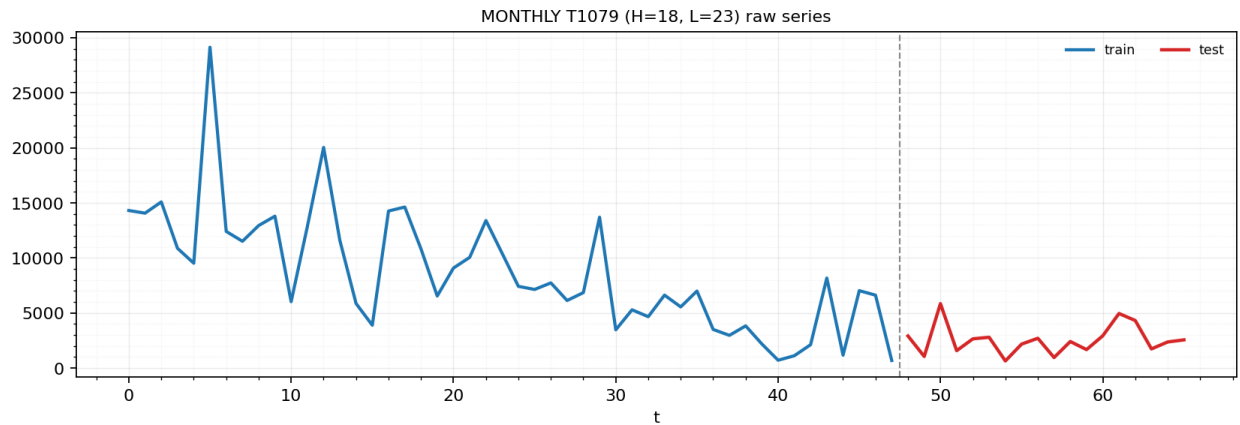


Рисунок 2.3.1. График T1079 из категории monthly.

На графике ACF / PACF данного ряда (рисунок 2.3.2) для подбора параметров ARIMA видно выраженное изменение уровня (нисходящий дрейф и скачки), поэтому для стационарности требуется **первое дифференцирование: $d = 1$** .

На PACF значимо выделяются лаги **1**, а также **3–4** – это типичный признак AR-компоненты с обрывом около $p \rightarrow \mathbf{p = 4}$.

ACF же убывает плавно и остаётся положительно значимой на ряде первых лагов, что больше похоже на AR-процесс, а не $MA(q) \rightarrow \mathbf{q = 0}$.

Сезонный SARIMA-компонент здесь не обязателен: на ACF нет выраженного сезонного пика на лаге **12** (для месячных рядов это было бы главным индикатором сезонности).

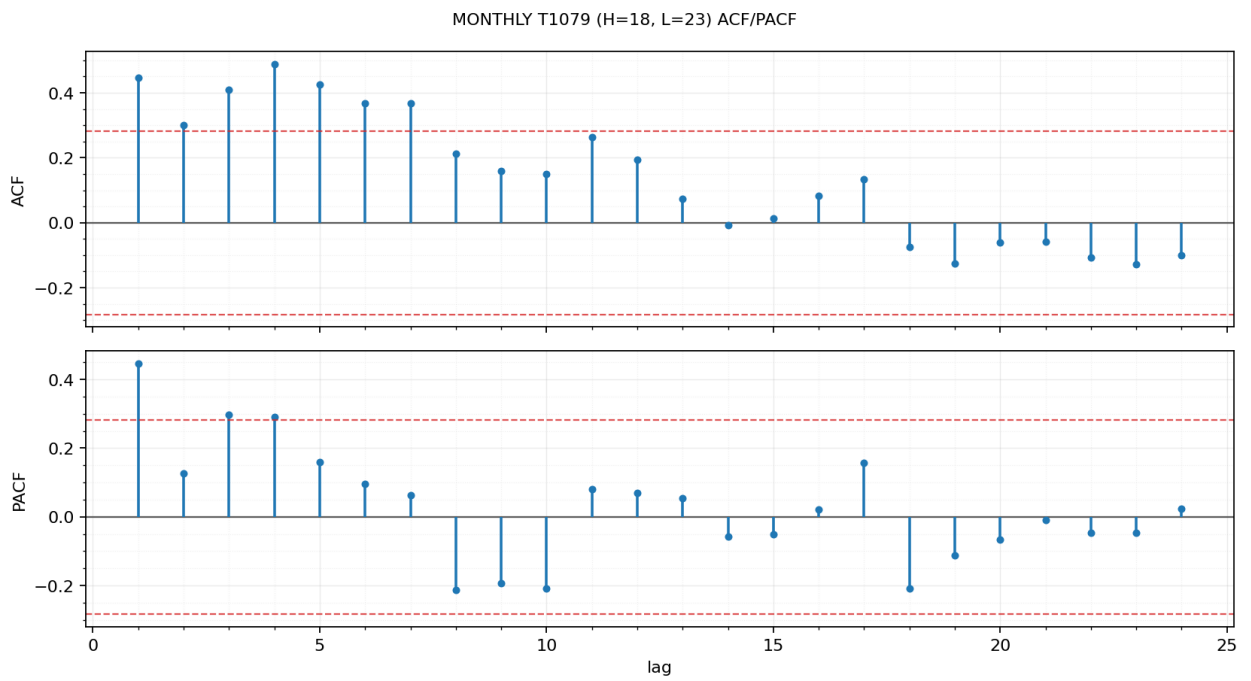


Рисунок 2.3.2. График ACF/PACF ряда T1079 для подбора параметров классической модели ARIMA.

Максимальное ограничение на кол-во D_j на данном наборе данных равно 3. График вейвлет-разложения MODWT на компоненты A_J и D_1, D_2, D_3 для ряда T1079 из категории monthly показан на рисунке 2.3.3.

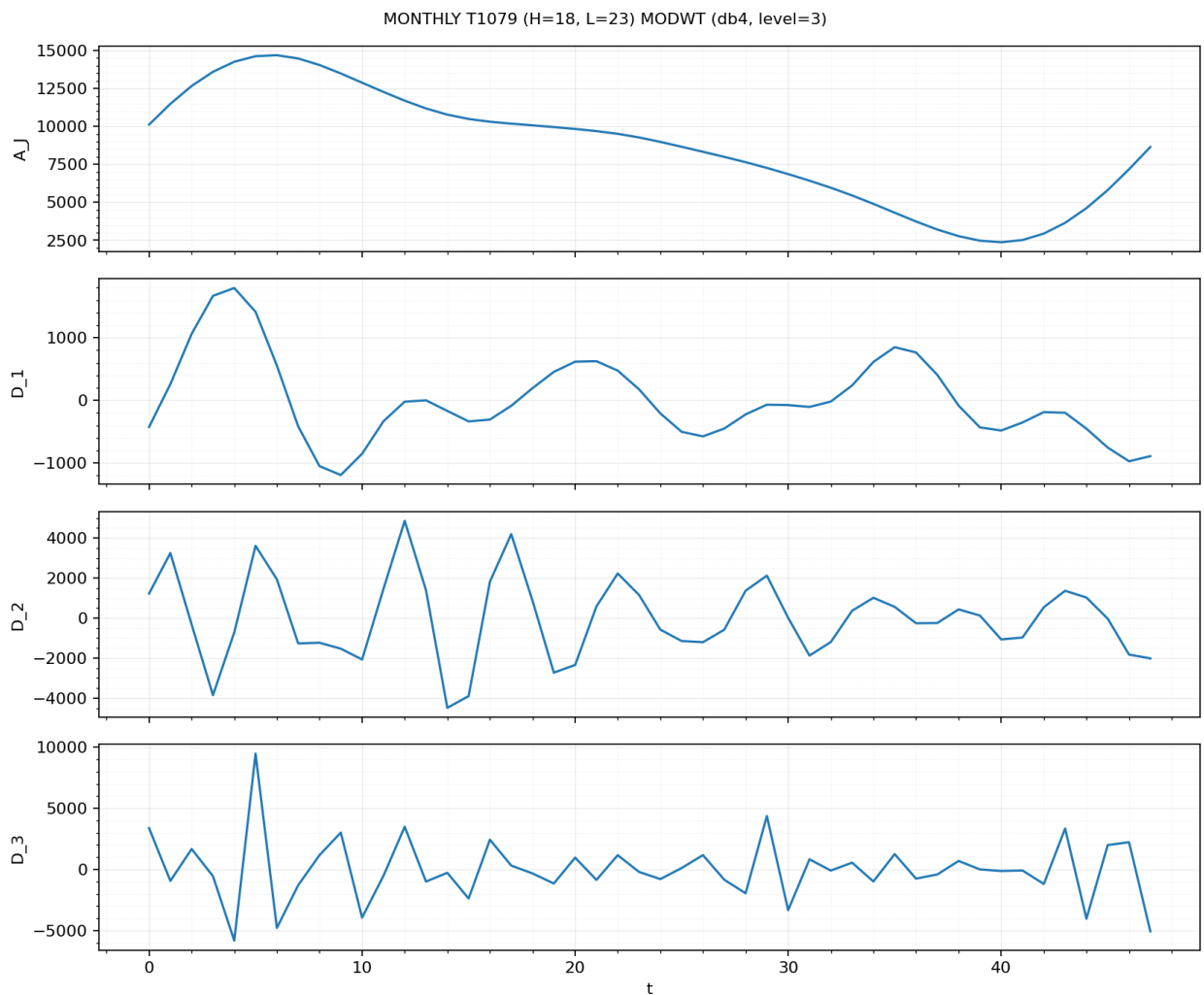


Рисунок 2.3.3. График вейвлет-разложения MODWT на компоненты для ряда T1079 из категории monthly.

2.3.3. Результаты обучения и оценка предсказаний на наборе данных МЗ

Теперь построим прогноз на каждой из компонент: на A_J будем использовать нейросетевые архитектуры TimesNet и N-beats, а на D_j классические статистические модели ARIMA (с автоподбором), ETS.

На рисунке 2.3.4 показан график компонент разложения ряда на тренд+сезонность и остатки с помощью MODWT, а также предсказание дальнейшего поведения ряда на компонентах с помощью нейросетевых архитектур и статистических моделей.

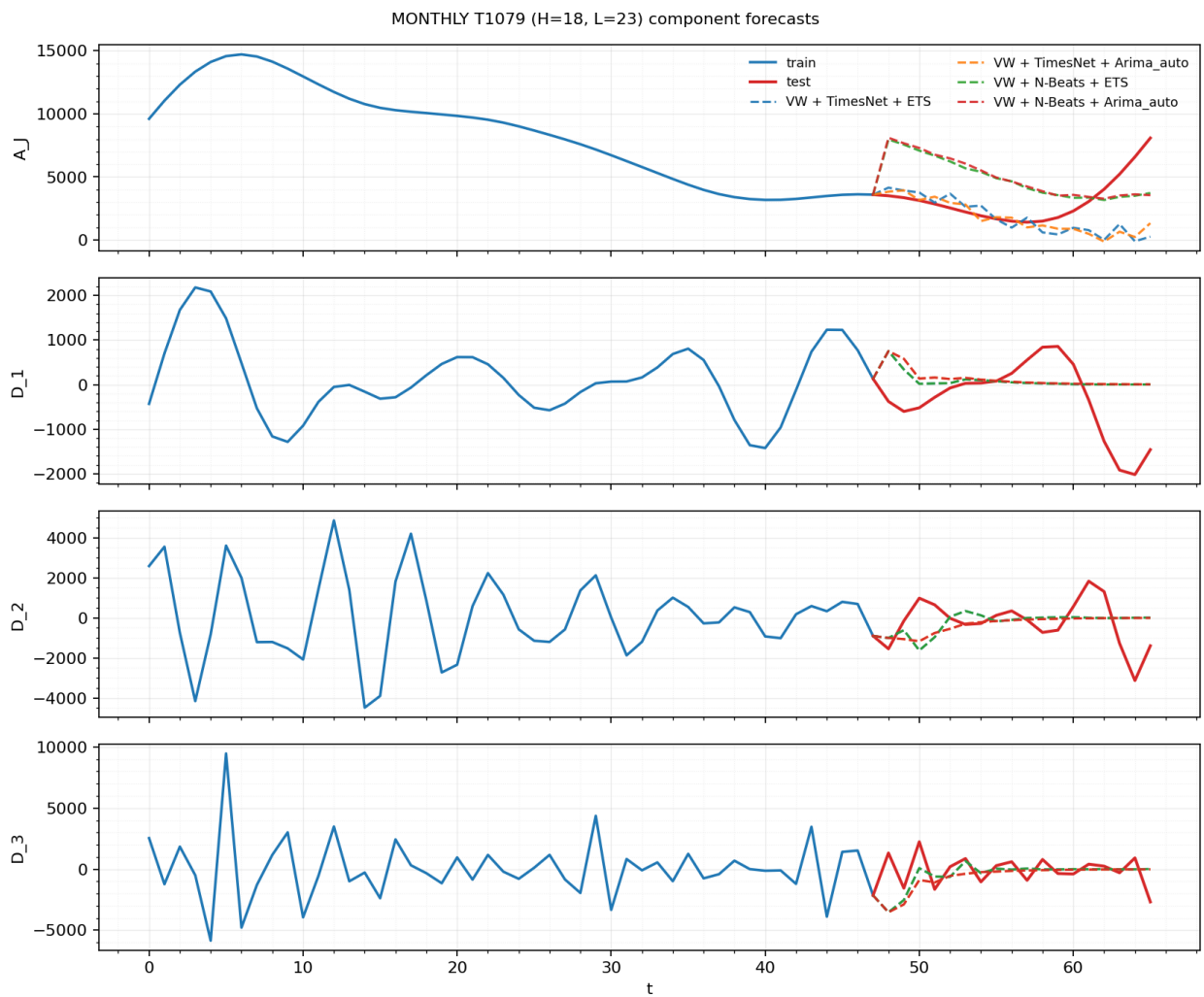


Рисунок 2.3.4. График компонент разложения ряда T1079 на тренд+сезонность и остатки с помощью MODWT, а также предсказание дальнейшего поведения ряда на компонентах с помощью нейросетевых архитектур и статистических моделей.

Суммируя предсказания на компонентах, получаем итоговое предсказание поведение ряда T1079 (рисунок 2.3.5).

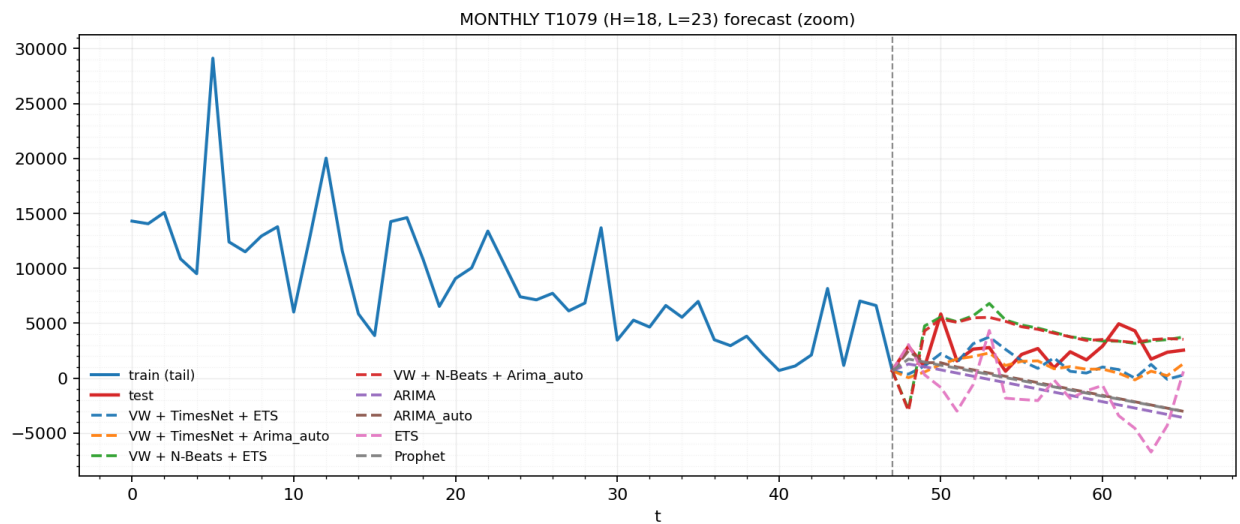


Рисунок 2.3.5. Итоговое предсказание поведения ряда T1079, сравнение с предсказаниями классических статистических моделей.

Еще несколько предсказаний для других категорий рядов на МЗ показаны на рисунках 2.3.6 – 2.3.9.

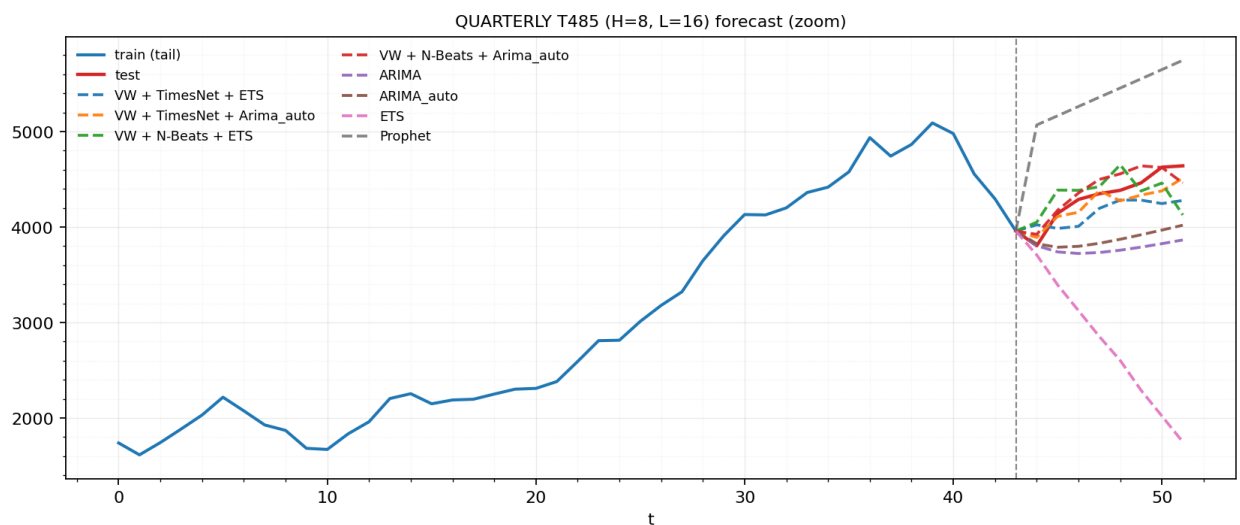


Рисунок 2.3.6. Предсказания реализованных гибридных и классических моделей на ряде из набора МЗ (*Quarterly T485*).

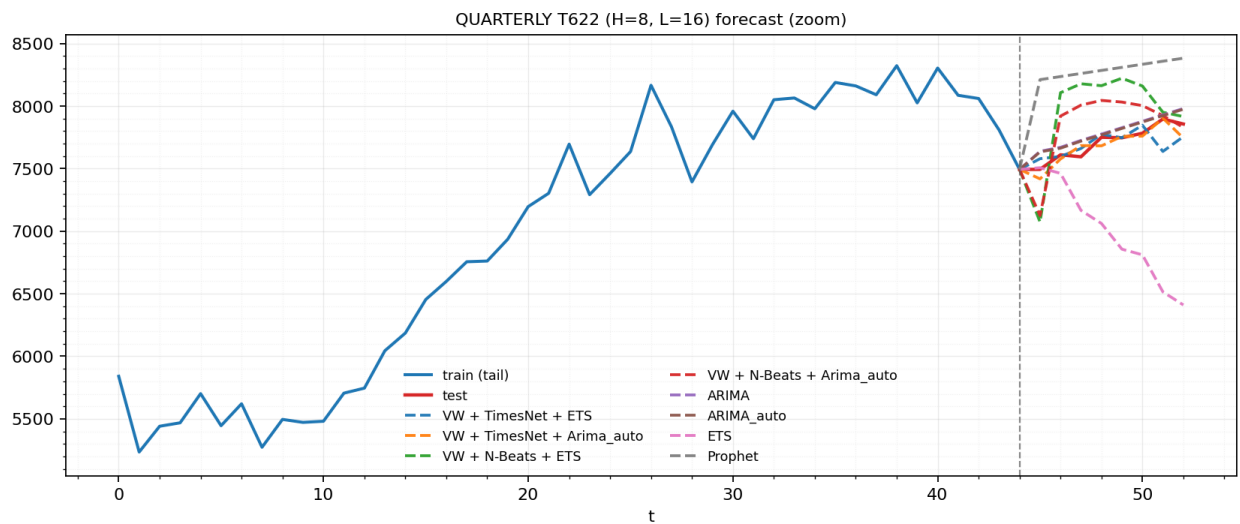


Рисунок 2.3.7. Предсказания реализованных гибридных и классических моделей на ряде из набора МЗ (*Quarterly T622*).

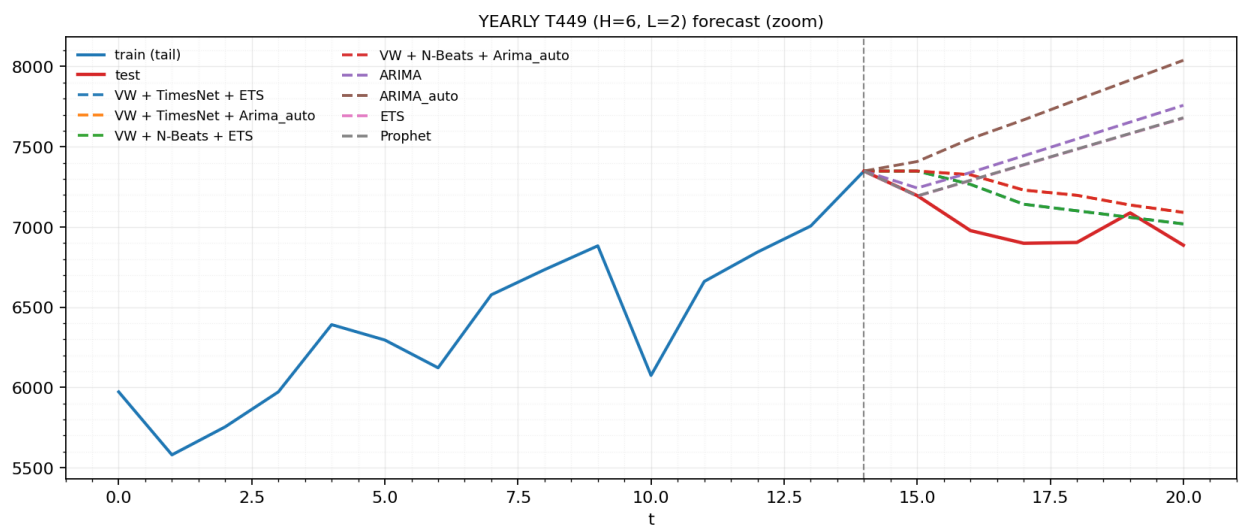


Рисунок 2.3.8. Предсказания реализованных гибридных и классических моделей на ряде из набора МЗ (*Yearly T449*).

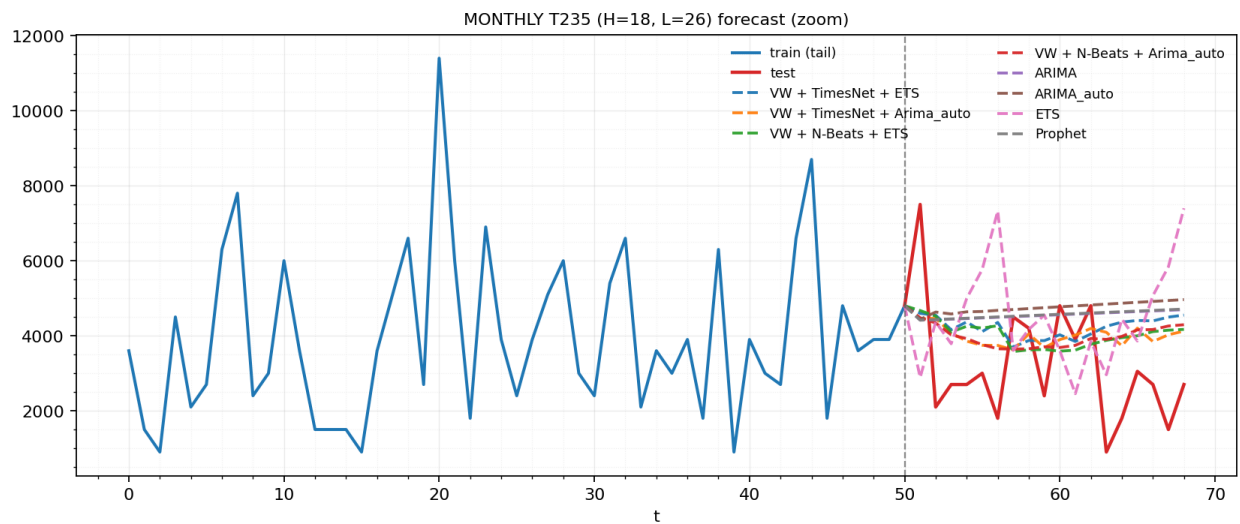


Рисунок 2.3.9. Предсказания реализованных гибридных и классических моделей на ряде из набора M3 (Monthly T235).

Теперь проведем общую оценку предсказаний на рядах по метрикам (*sMAPE*, *MAPE*, *RMSE*, *MSE*).

Итог по всем категориям (усредненные показатели метрик на подвыборке рядов):

sMAPE:

1. **VW + N-Beats + ETS — 22.520 (лучший)**
2. **VW + N-Beats + ARIMA_auto — 22.524**
3. **VW + TimesNet + ARIMA_auto — 22.542**
4. **VW + TimesNet + ETS — 23.195**
5. ETS — 23.992
6. Prophet — 24.804
7. ARIMA_auto — 25.380
8. ARIMA — 25.492

MAPE:

1. **VW + TimesNet + ETS — 30.722 (лучший)**
2. ETS — 31.214
3. VW + TimesNet + ARIMA_auto — 30.882
4. Prophet — 32.105
5. VW + N-Beats + ETS — 32.258
6. VW + N-Beats + ARIMA_auto — 33.574
7. ARIMA — 40.738
8. ARIMA_auto — 42.123

RMSE:

1. **ETS — 1197.523 (лучший по абсолютной ошибке)**
2. VW + TimesNet + ETS — 1211.092

3. VW + TimesNet + ARIMA_auto — 1215.865
4. VW + N-Beats + ETS — 1228.265
5. Prophet — 1247.606
6. VW + N-Beats + ARIMA_auto — 1248.927
7. ARIMA — 1412.073
8. ARIMA_auto — 1418.484

MSE:

1. **VW + N-Beats + ETS — 2 682 435.843 (лучший)**
2. VW + TimesNet + ETS — 2 740 591.470
3. Prophet — 2 923 764.847
4. VW + N-Beats + ARIMA_auto — 2 919 561.325
5. VW + TimesNet + ARIMA_auto — 2 928 699.177
6. ETS — 3 151 806.436
7. ARIMA — 5 219 305.607
8. ARIMA_auto — 5 353 539.604

Итак, по относительной ошибке (sMAPE) лидируют **VW-гибриды (~22.5)**, по RMSE лучше всего **ETS (1197.5)**, а по MSE лучший **VW + N-Beats + ETS (2.68 млн)**. ARIMA/ARIMA_auto — худшие по всем абсолютным метрикам.

Итог по категориям (усредненные показатели метрик на подвыборке рядов определенной категории):

Monthly (самая сложная категория)

- **sMAPE:**
 - $VW + N - Beats + ETS = 28.327$ (лучший)
 - $VW + TimesNet + ARIMA_auto = 30.985$
 - $ETS = 32.407$
 - $ARIMA_auto = 41.704$ (худший)
- **MAPE:**
 - $ETS = 38.743$ (лучший)
 - $VW + TimesNet + ETS = 43.353$
 - $ARIMA_auto = 73.237$ (худший)
- **RMSE:**
 - $ETS = 1207.997$ (лучший)
 - $VW + N - Beats + ETS = 1309.319$
 - $ARIMA_auto = 2084.502$ (худший)
- **MSE:**
 - $VW + N - Beats + ETS = 2\,845\,086$ (лучший)
 - $ARIMA_auto = 9\,844\,017$ (худший)

Ряды из категории *monthly* дают сильную вариативность и неоднородность масштаба, поэтому *ARIMA_auto* деградирует сильнее всего; *ETS* и *VW*-гибриды удерживают ошибку заметно ниже.

Quarterly

- **sMAPE:**
 - $VW + TimesNet + ARIMA_auto = 10.101$ (лучший)
 - $ARIMA_auto = 10.133$
 - $VW + TimesNet + ETS = 10.858$
 - $ETS = 15.124$
 - $Prophet = 16.110$ (худший)
- **MAPE:**
 - $ARIMA_auto = 11.197$ (лучший)
 - $VW + TimesNet + ARIMA_auto = 11.494$
 - $VW + TimesNet + ETS = 12.704$
 - остальные выше
- **RMSE:**
 - $ARIMA_auto = 613.074$ (лучший)
 - $VW + TimesNet + ARIMA_auto = 644.639$
 - $VW + TimesNet + ETS = 687.910$
 - остальные выше
- **MSE:**
 - $ARIMA_auto = 719\,860$ (лучший)
 - $VW + TimesNet + ARIMA_auto = 867\,308$
 - остальные выше

На quarterly автоподбор ARIMA оказывается удачным (короткие ряды + умеренная сезонность), поэтому ARIMA_auto даёт минимум по RMSE/MSE; VW с TimesNet почти догоняет.

Yearly

- **sMAPE:**
 - $ARIMA_auto = 24.304$ (лучший)
 - $ETS = 24.444$
 - $VW + TimesNet + ETS = 26.157$
 - $VW + TimesNet + ARIMA_auto = 26.541$ (худший)
- **MAPE:**
 - $Prophet = 28.458$ (лучший)
 - $VW + N - Beats + ETS = 35.822$
 - остальные выше
- **RMSE:**
 - $Prophet = 1205.473$ (лучший)
 - $VW + N - Beats + ETS = 1523.722$
 - $ETS = 1544.991$
 - $ARIMA_auto = 1557.877$
 - остальные выше
- **MSE:**
 - $Prophet = 2\,265\,596$ (лучший)
 - остальные выше ($\approx 3.8 - 5.5$ млн).

Годовые ряды короткие и часто с низкой регулярной сезонностью; Prophet в среднем лучше отрабатывает уровень/тренд на yearly (особенно по RMSE/MSE), а ARIMA_auto выигрывает по sMAPE (относительная ошибка) за счёт удачной локальной динамики.

2.4. Тестирование моделей на наборе данных M4

2.4.1. Описание набора данных

Набор M4 — крупный эталонный корпус для сравнения методов прогнозирования временных рядов, опубликованный в рамках **M4 Forecasting Competition (2018)** под руководством S. Makridakis и соавторов. Он содержит **100 000 реальных унивариативных рядов** и задаёт единый протокол тестирования по фиксированному горизонту прогноза для каждой частоты.

Всего в M4 **6 частот** (сильный дисбаланс в пользу месячных рядов):

- **Hourly** (414 рядов, длина прогноза 48)
- **Daily** (4227 рядов, длина прогноза 14)
- **Weekly** (359 рядов, длина прогноза 13)
- **Monthly** (48000 рядов, длина прогноза 18)
- **Quarterly** (24000 рядов, длина прогноза 8)
- **Yearly** (23000 рядов, длина прогноза 6)

Для обеспечения достаточной истории в соревновании вводились минимальные длины рядов: 13 (yearly), 16 (quarterly), 42 (monthly), 80 (weekly), 93 (daily), 700 (hourly) наблюдений.

Каждый ряд имеет идентификатор, частоту и доменную метку (Micro, Industry, Macro, Finance, Demographic, Other), что позволяет анализировать переносимость моделей между областями.

Особенности M4 (практически важные)

1. **Разные масштабы времени** (часовые–годовые) → модели должны работать и на короткой, и на длинной памяти.
2. **Дисбаланс классов**: почти половина набора — **monthly (48 000)**.
3. **Только унивариативные данные** (без экзогенных признаков) → сравнение методов “в чистом виде”.

2.4.2. Симуляция rollout на quarterly рядах

Чтобы оценить поведение моделей в реальном сценарии многократного прогноза, применялась *rollout*-симуляция: прогноз строится не «одним шагом», а **последовательным разворачиванием на H шагов вперёд**.

Процесс выглядит так:

1. Берётся последнее доступное окно длиной L из обучающей части.
2. Модель предсказывает следующий шаг (или блок шагов), после чего предсказание добавляется в конец окна, а самое старое значение из окна удаляется.
3. Шаги 1–2 повторяются, пока не будет получен прогноз на весь горизонт H .
4. В гибридных схемах *rollout* проводится для компонент (аппроксимация/детали) отдельно, затем выполняется обратная реконструкция в исходный ряд.

Рассмотрим ряд Q2143 из категории quarterly. Его график представлен на рисунке 2.4.2.1.

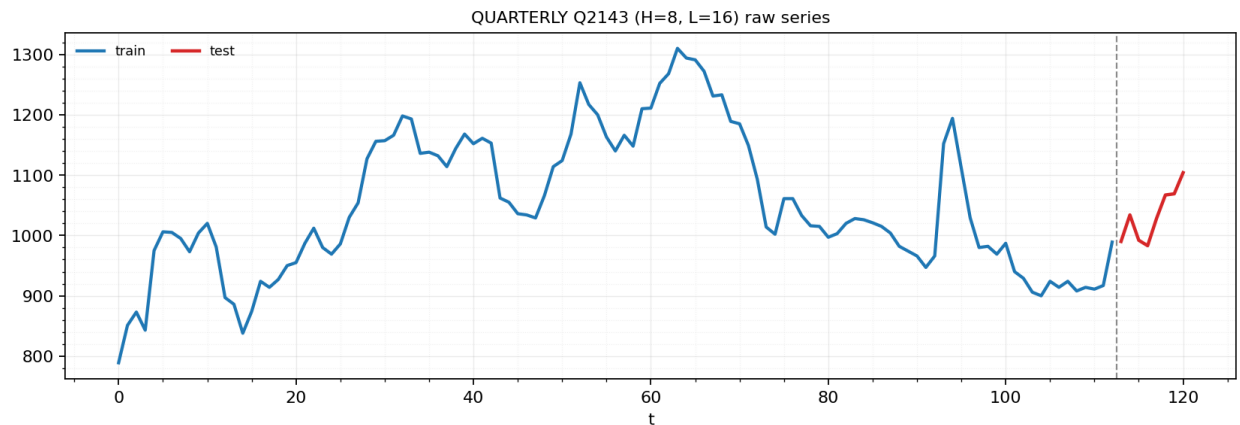


Рисунок 2.4.2.1. График ряда Q2143 из категории quarterly.

С помощью вейвлет-разложения MODWT разобьем ряд на 5 компонент: A_j и D_j , где $j \in \{1, 2, 3, 4\}$, а после построим *rollout*-симуляцию на каждой из компонент.

Обучать мы будем гибридные модели, аналогичные моделям, которые мы обучали на МЗ (см. главу 2.3.2), однако обучаться будем на 40 эпохах (вместо 20 на МЗ).

На рисунке 2.4.2.2 показана декомпозиция ряда Q2143, а также *rollout*-симуляция на каждой из компонент.

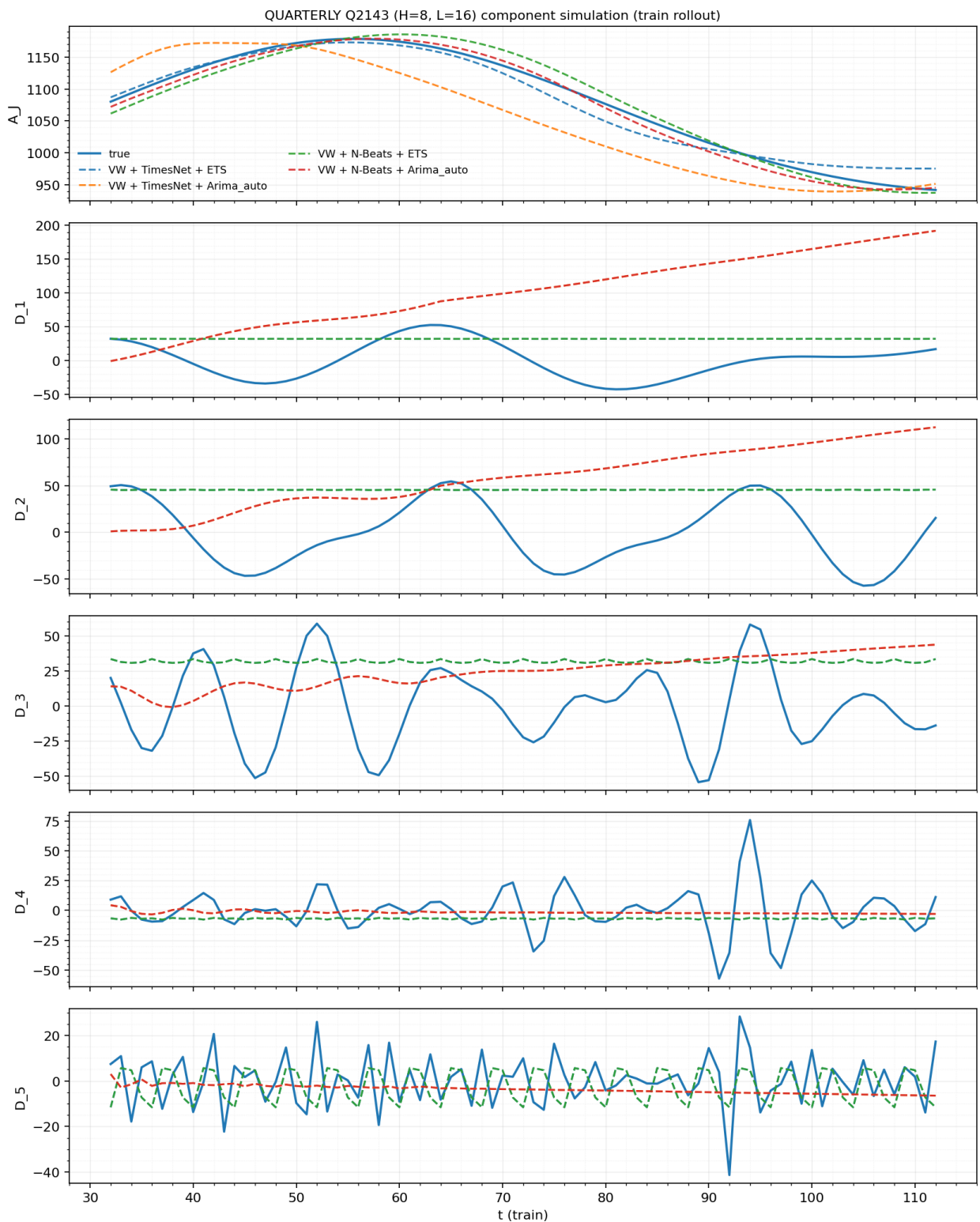


Рисунок 2.4.2.2. Декомпозиция ряда Q2143 и *rollout*-симуляция на каждой из компонент.

Анализируя *rollout*-симуляцию на каждой из компонент декомпозиции ряда Q2143, заметно, что аппроксимирующая компонента воспроизводится лучше всего: все гибриды корректно ловят **общую дугообразную динамику** (рост → пик → спад). Ошибка в основном проявляется как **сдвиг по уровню** и небольшая **недооценка/переоценка** на участке после пика.

А вот на средних и высоких частотах **ETS** «зажимает» детали в почти константу → теряются волны/фаза, **ARIMA_auto** даёт нарастающий тренд в деталях → появляется дрейф при rollout.

Пошагово суммируя *rollout*-прогнозов на компонентах вейвлет-разложения, построим *rollout*-симуляцию на исходном ряде Q2143 (рисунок 2.4.2.3).

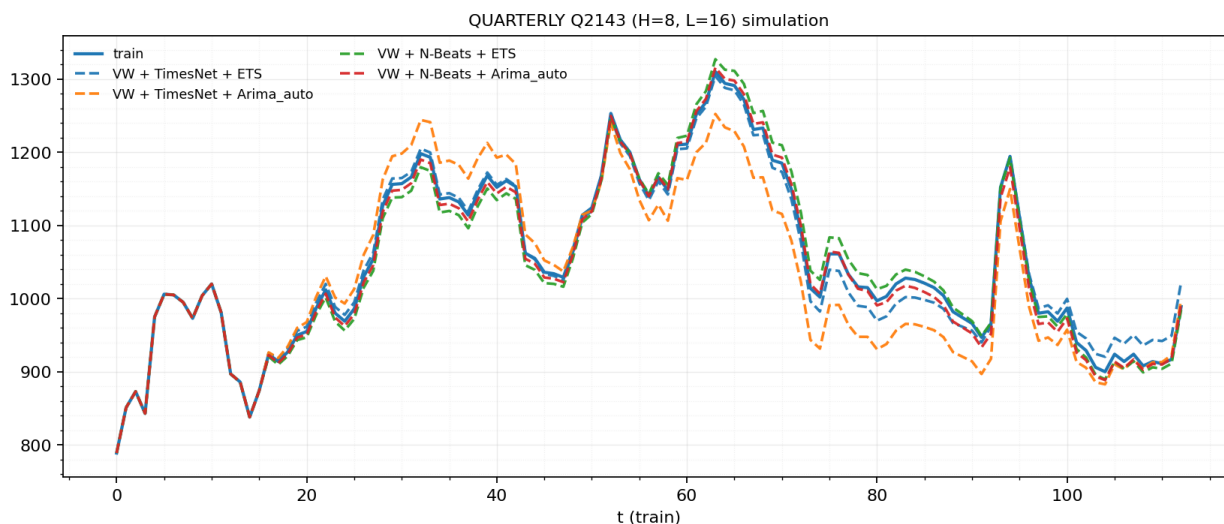


Рисунок 2.4.2.3. Rollout-симуляция исходного ряда Q2143.

Из рисунка 2.4.2.3 заметно, что все гибридные модели держат общий уровень и крупные перемены, потому что основную энергию несёт A_J .

Различия между методами обусловлены именно тем, как предсказываются детали:

- В гибридных моделях, где детали прогнозируются **ETS**, более ровная траектория: итоговый ряд ближе к тренду, меньше мелких колебаний.
- В гибридных моделях, где детали прогнозируются **ARIMA_auto** чаще виднеется **смещение вниз/дрейф** на концах участков: это эффект накопления ошибки в D_j при рекурсивном прогоне (то, что видно на графиках компонент как дрейф деталей).
- Пики/выбросы в районе ~90–95 по времени воспроизводятся частично, но не идеально: они лежат в высокочастотных $D_4 - D_5$, которые в *rollout* предсказываются хуже всего.

Метрики оценивания прогноза на ряде Q2143 показаны в таблице 2.4.2.1:

Таблица 2.4.2.1. Результат по метрикам на ряде Q2143 из набора данных M4.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	4.163	4.078	57.691	3328.258

ARIMA_auto	4.148	4.071	56.811	3227.499
ETS	4.122	3.985	61.494	3781.485
Prophet	23.623	20.915	233.586	54562.220
VW_+_TimesNet_+_Arima_auto	4.717	4.797	52.398	2745.568
VW_+_TimesNet_+_ETS	3.915	3.804	56.642	3208.341
VW_+_N-Beats_+_ETS	4.242	4.067	66.252	4389.262
VW_+_N-Beats_+_Arima_auto	4.162	4.074	57.027	3252.027

По ряду Q2143 лучшие результаты дали гибриды **VW + TimesNet**:

- минимум по $sMAPE/MAPE$ у $VW + TimesNet + ETS$ ($sMAPE=3.915$, $MAPE=3.804$)
- минимум по $RMSE/MSE$ у $VW + TimesNet + ARIMA_auto$ ($RMSE=52.398$, $MSE=2745.568$)

Классические **ARIMA/ARIMA_auto/ETS** показывают сопоставимый, но чуть хуже уровень ошибок, тогда как **Prophet** существенно проигрывает ($sMAPE=23.623$, $RMSE=233.586$), что указывает на неудачную подгонку трендовой модели для данного ряда.

Графики других *rollout*-симуляций на рядах категории quarterly для набора данных M4 показаны на рисунках 2.4.2.4-2.4.2.7.

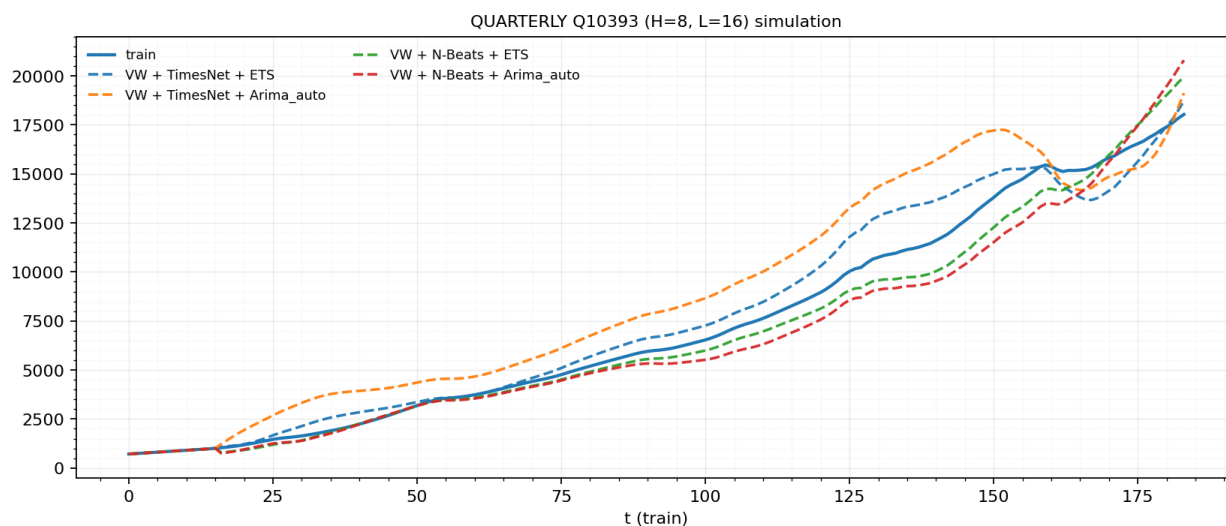


Рисунок 2.4.2.4. Rollout-симуляция исходного ряда Q10393.

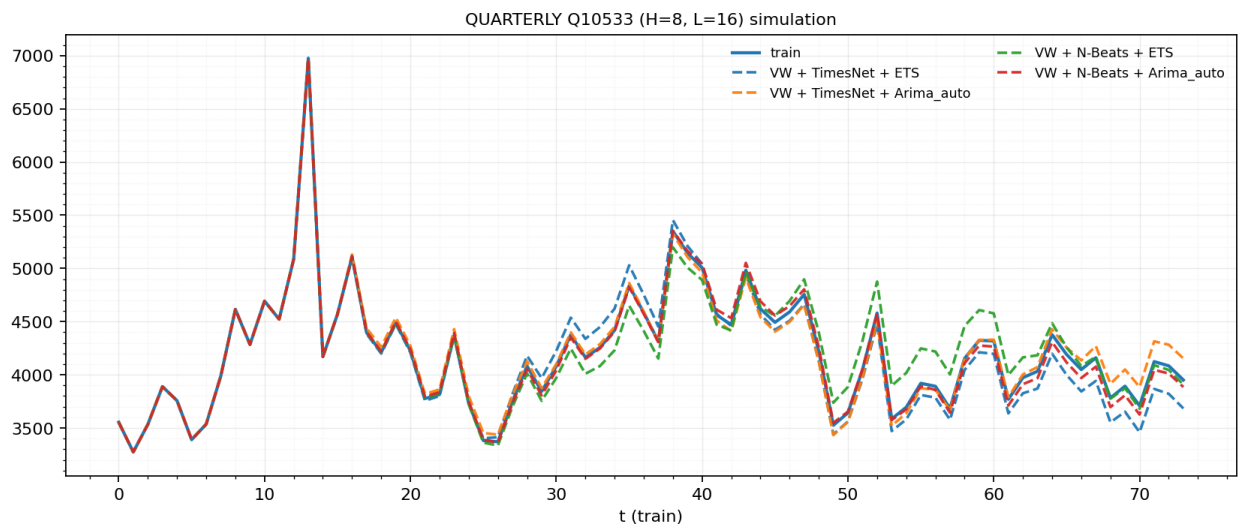


Рисунок 2.4.2.5. Rollout-симуляция исходного ряда Q10533.

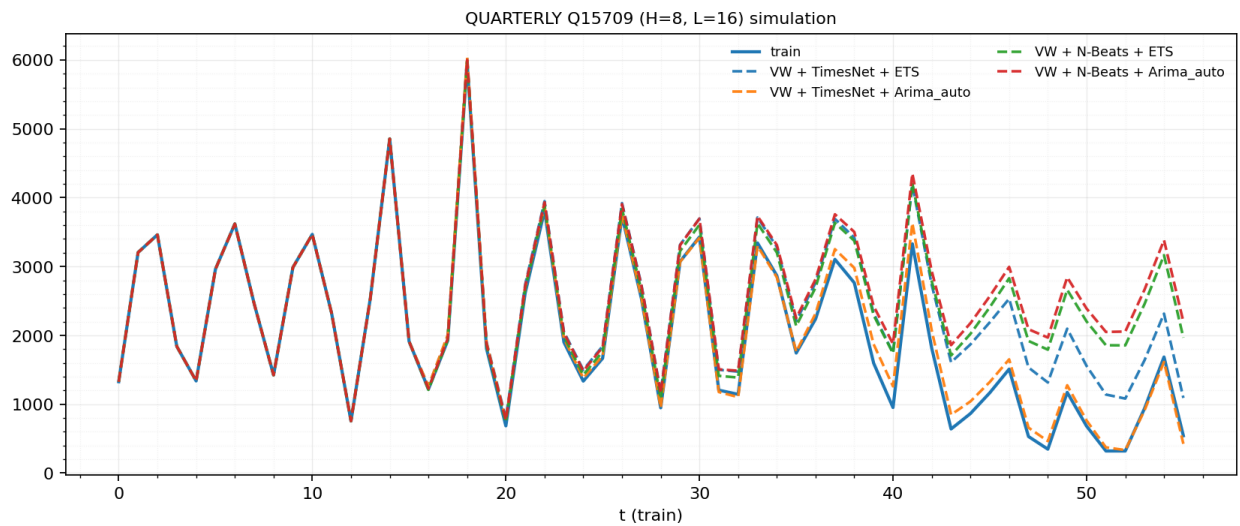


Рисунок 2.4.2.6. Rollout-симуляция исходного ряда Q15709

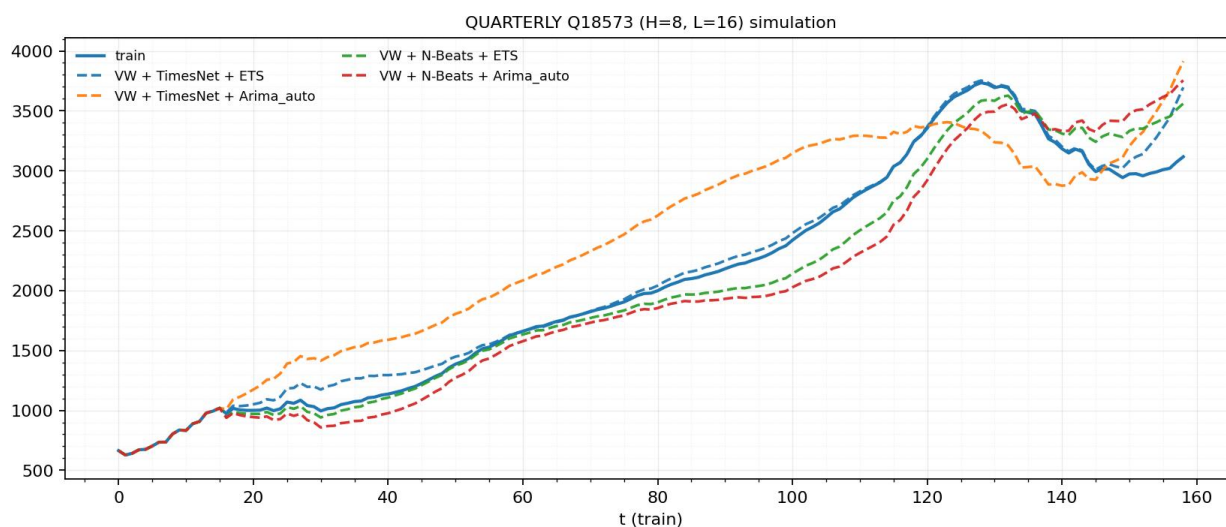


Рисунок 2.4.2.7. Rollout-симуляция исходного ряда Q18573

2.4.3. Результаты обучения и оценка предсказаний на наборе данных M4

После рассмотрения и оценки каскадного прогнозирования (*rollout* симуляции) на рядах квартальной частоты, проведем оценку на больших объемах данных для целого набора, агрегировав результаты по категории рядов.

Для каждой частоты выбиралось **по 500 рядов** (кроме Hourly и Weekly, так как рядов в этих категориях меньше запрашиваемого количества) в качестве соблюдения вычислительных ограничений при сохранении репрезентативности. Помимо гибридных архитектур в обучении участвовали и чистые модели N-Beats, TimesNet без предварительного разложения на компоненты MODWT.

Каждый ряд приводился к формату одна колонка значений + частота, после чего выполнялся первичный анализ: считался тест Дики – Фуллера для оценки стационарности и необходимости дифференцирования, строились графики ACF/PACF.

В таблице 2.4.3.1 представлены результаты по метрикам всех моделей, участвовавших в обучении на подвыборке из 500 рядов годовой частоты.

Таблица 2.4.3.1. Результаты моделей по метрикам, участвовавших в обучении на подвыборке из 500 рядов годовой частоты.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	14.661	17.633	812.069	1256886.441
ARIMA_auto	14.768	18.005	828.427	1378057.503
ETS	17.766	22.642	909.607	1406149.290
Prophet	22.442	35.474	1384.235	6352256.513
TimesNet (без разложения на компоненты MODWT)	15.684	17.479	1002.506	1564040.871
N-Beats (без разложения на компоненты MODWT)	26.547	31.019	1670.704	5305433.269

<i>компоненты MODWT)</i>				
VW + TimesNet + Arima_auto	13.518	15.004	764.001	614658.015
VW_+_TimesNet_+_ETS	16.861	19.930	1048.345	1574910.266
VW + N-Beats + ETS	20.903	24.969	1275.731	2765657.926
VW + N-Beats + Arima_auto	22.720	27.851	1360.148	3280777.038

Из результатов предсказания на годовой частоте лучшей моделью по абсолютным и относительным метрикам стала гибридная модель, сочетающая в себе агрегацию предсказаний нейросетевой архитектуры TimesNet на компоненте A_j разложения MODWT и предсказаний ARIMA с автоподбором параметров на остаточных компонентах D_j – **VW + TimesNet + Arima_auto**. Однако при рассмотрении метрик заметно, что классические модели справляются с предсказанием даже лучше нейросетевых и это понятно: **данные слишком короткие**, поэтому нейросетям (особенно N-Beats) тяжело – они проигрывают.

На рисунках 2.4.3.1 – 2.4.3.2 можно увидеть несколько результатов каскадного прогнозирования через построение графиков *rollout* симуляции.

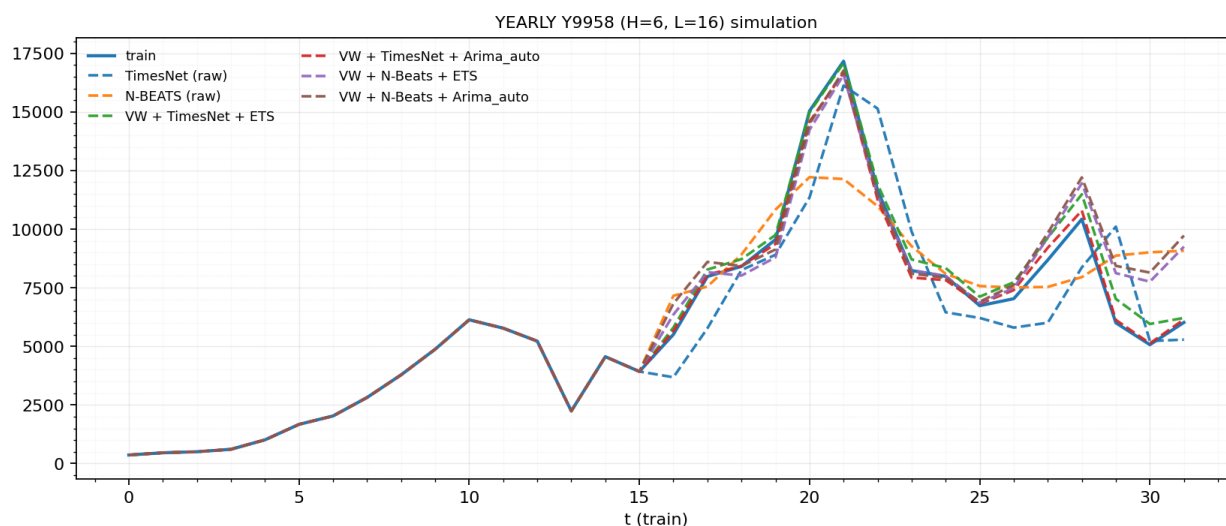


Рисунок 2.4.3.1. Rollout-симуляция исходного ряда Y9958

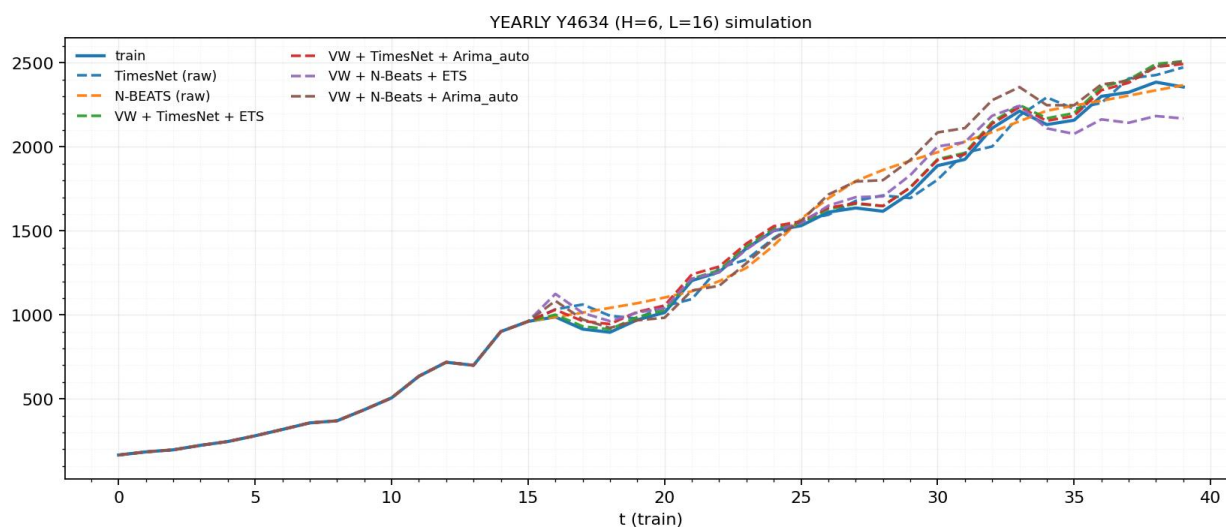


Рисунок 2.4.3.2. Rollout-симуляция исходного ряда Y4634

В таблице 2.4.3.2 представлены результаты по метрикам всех моделей, участвовавших в обучении на подвыборке из 500 рядов квартальной частоты.

Таблица 2.4.3.2. Результаты моделей по метрикам, участвовавших в обучении на подвыборке из 500 рядов квартальной частоты.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	14.385	19.538	538.908	475019.327
ARIMA_auto	14.017	19.588	519.906	449277.862
ETS	13.150	15.343	505.145	427108.616
Prophet	30.967	134.202	1104.468	2808740.241
TimesNet (без разложения на компоненты MODWT)	14.866	30.138	463.858	377755.477
N-Beats (без разложения на компоненты MODWT)	16.492	24.987	599.845	800123.456
VW + TimesNet + Arima_auto	15.600	24.728	570.466	566371.986
VW + TimesNet + ETS	13.695	21.981	570.981	662457.969
VW + N-Beats + ETS	16.203	23.456	872.346	760987.645
VW + N-Beats + Arima_auto	15.845	22.134	848.809	720476.722

Из результатов предсказания на квартальной частоте лучшей моделью **по относительным метрикам** стала **ETS**, похоже, так как она лучше всего ловит гладкую динамику – тренд и слабую сезонность, а **по абсолютным TimesNet** без разложения на компоненты MODWT, чем показал лучшее попадание в абсолютные значения на крупных уровнях ряда, но хуже на участках, где значения малы (MAPE/sMAPE выше ETS).

На рисунках 2.4.3.3 – 2.4.3.4 можно увидеть несколько результатов каскадного прогнозирования через построение графиков *rollout* симуляции на квартальной частоте.

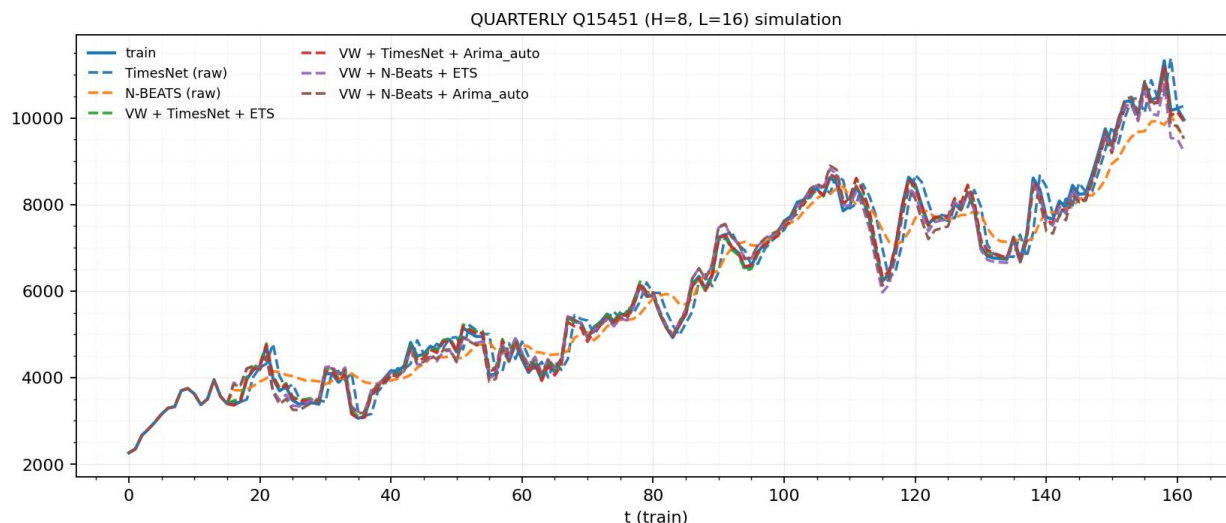


Рисунок 2.4.3.3. Rollout-симуляция исходного ряда Q15451

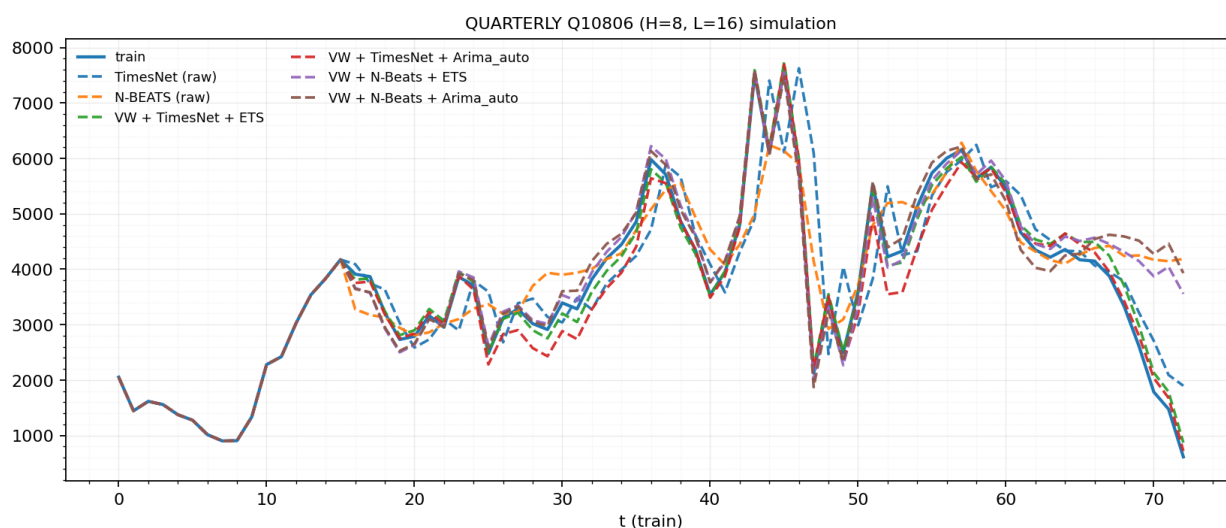


Рисунок 2.4.3.4. Rollout-симуляция исходного ряда Q10806

В таблице 2.4.3.3 представлены результаты по метрикам всех моделей, участвовавших в обучении на подвыборке из 500 рядов дневной частоты.

Таблица 2.4.3.3. Результаты моделей по метрикам, участвовавших в обучении на подвыборке из 500 рядов дневной частоты.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	3.098	3.002	170.192	46951.625
ARIMA_auto	3.076	2.980	168.258	45809.830

<i>ETS</i>	3.085	2.994	165.867	43210.975
<i>Prophet</i>	12.456	16.541	963.783	3390573.745
<i>TimesNet (без разложения на компоненты MODWT)</i>	3.809	3.641	185.841	49132.251
<i>N-Beats (без разложения на компоненты MODWT)</i>	3.901	3.804	236.408	87354.811
<i>VW + TimesNet + Arima_auto</i>	2.133	2.146	128.943	28479.300
<i>VW_+ TimesNet_+ ETS</i>	2.048	2.062	111.606	17823.037
<i>VW + N-Beats + ETS</i>	2.697	2.659	168.891	52215.917
<i>VW + N-Beats + Arima_auto</i>	2.666	2.632	169.989	62485.030

У дневных рядов обычно есть тренд, календарные эффекты, локальные всплески и шум, именно на таких рядах заметно, что разложение исходного сигнала на компоненты MODWT – лучшее решение для построения краткосрочного прогноза. Лучшей моделью по относительным и абсолютным метрикам стала **гибридная архитектура**, работающая в связке с **TimesNet** и **ETS**, однако гибриды N-Beats тоже показали себя одними из лучших моделей, незначительно уступая **TimesNet**.

На рисунках 2.4.3.5 – 2.4.3.6 можно увидеть несколько результатов каскадного прогнозирования через построение графиков *rollout* симуляции на дневной частоте.

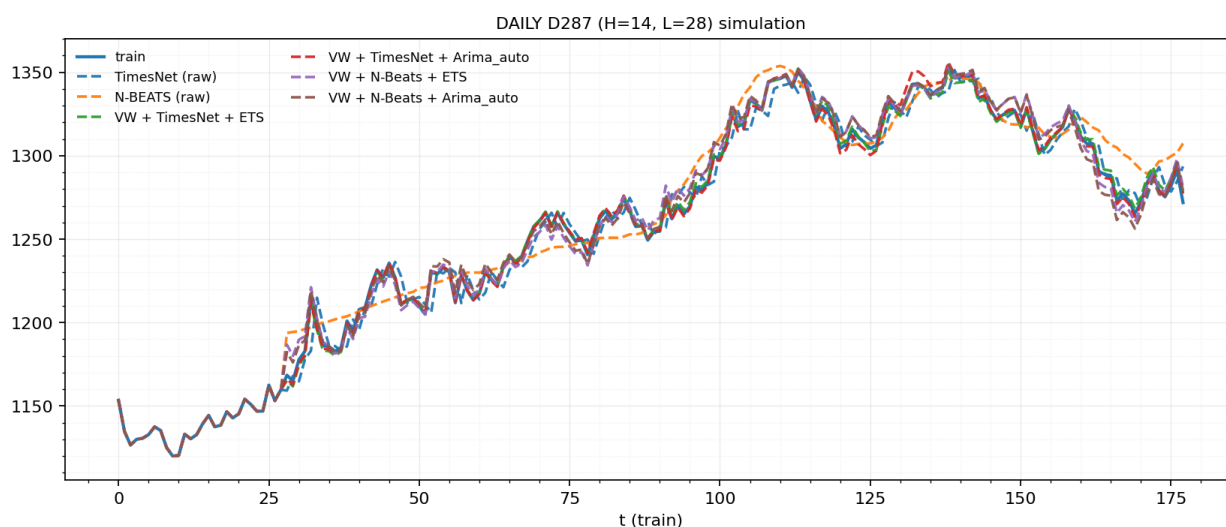


Рисунок 2.4.3.5. Rollout-симуляция исходного ряда D287

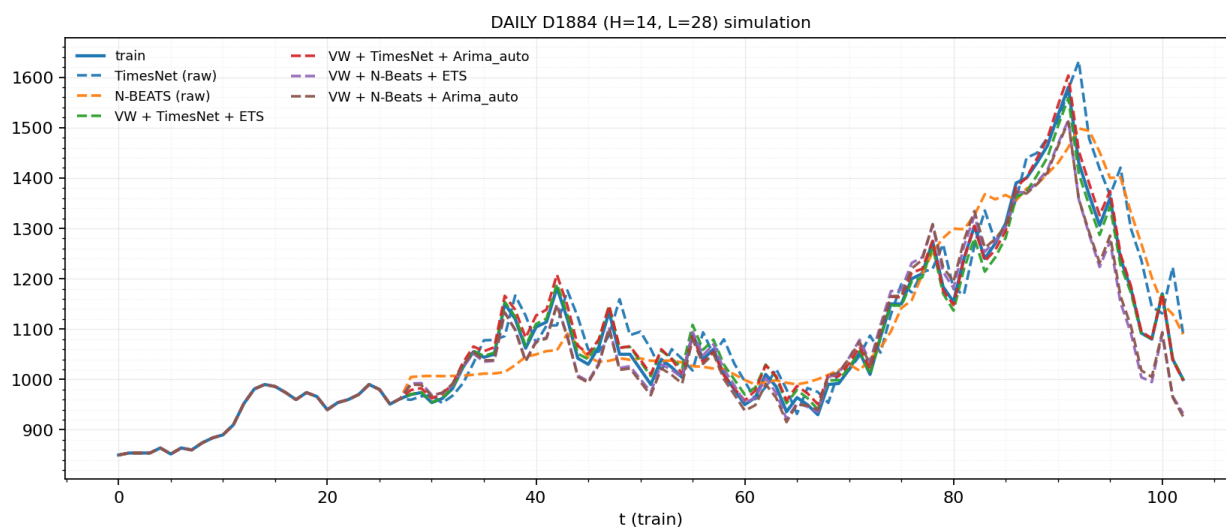


Рисунок 2.4.3.6. Rollout-симуляция исходного ряда D1884

В таблице 2.4.3.4 представлены результаты по метрикам всех моделей, участвовавших в обучении на подвыборке из 414 рядов часовой частоты.

Таблица 2.4.3.4. Результаты моделей по метрикам, участвовавших в обучении на подвыборке из 414 рядов часовой частоты.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	52.702	106.829	165.931	97704.384
ARIMA_auto	63.957	56.709	175.236	110850.691
ETS	22.378	28.346	112.389	44273.202
Prophet	41.096	123.264	141.818	76124.493
TimesNet (без разложения на компоненты MODWT)	21.660	24.137	105.620	45670.413
N-Beats (без разложения на компоненты MODWT)	23.893	31.424	119.419	59030.867
VW + TimesNet + Arima_auto	38.392	93.072	138.562	76642.275
VW + TimesNet + ETS	39.562	97.596	141.284	80571.886
VW + N-Beats + ETS	37.928	82.950	142.263	80473.400
VW + N-Beats + Arima_auto	37.615	87.304	139.438	76392.550

На часовых данных доминируют высокочастотные компоненты (внутрисуточная периодика), резкие пики и возможные фазовые сдвиги, поэтому разложение и последующая реконструкция могут вносить фазовые ошибки и сглаживание амплитуды, особенно на экстремумах - а именно они сильнее всего наказываются метриками. Именно поэтому применение гибридов здесь неоптимально, для часовых рядов наилучшие значения ошибок демонстрирует **TimesNet без разложения на компоненты MODWT**, обеспечивая минимальные *sMAPE*, *RMSE* и *MSE*, тогда как ETS выступает сильным статистическим базисом с сопоставимыми результатами.

На рисунках 2.4.3.7 – 2.4.3.8 можно увидеть несколько результатов каскадного прогнозирования через построение графиков *rollout* симуляции на часовой частоте.

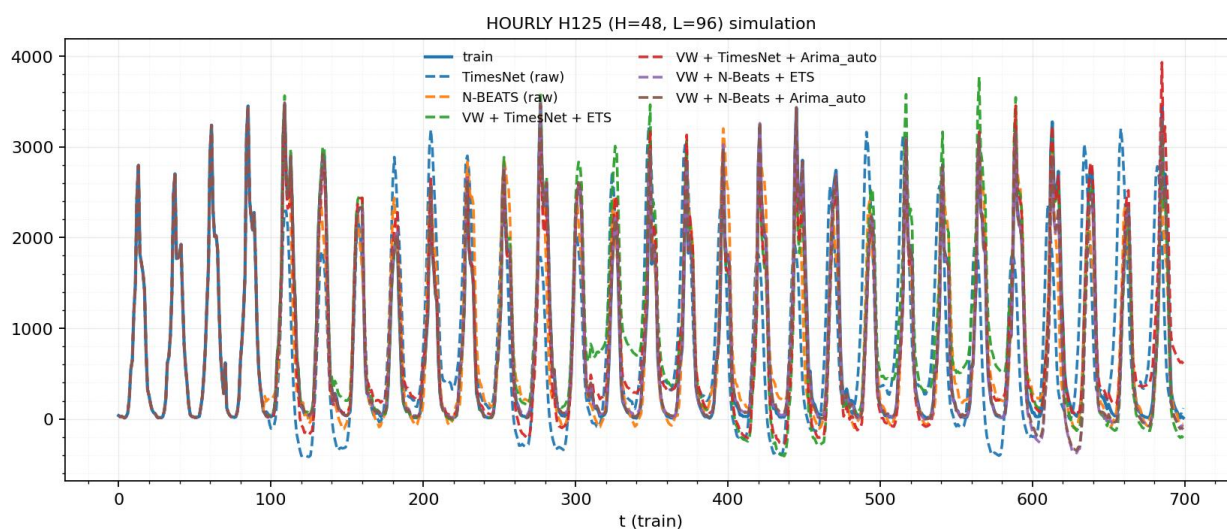


Рисунок 2.4.3.7. Rollout-симуляция исходного ряда H125

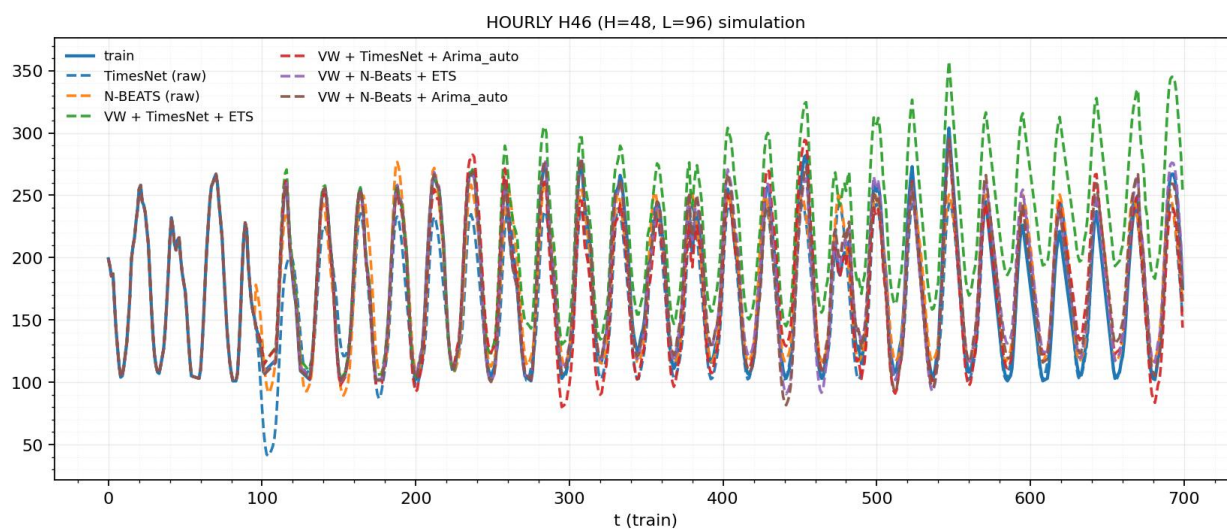


Рисунок 2.4.3.8. Rollout-симуляция исходного ряда H46

В таблице 2.4.3.5 представлены результаты по метрикам всех моделей, участвовавших в обучении на подвыборке из 500 рядов месячной частоты.

Таблица 2.4.3.5. Результаты моделей по метрикам, участвовавших в обучении на подвыборке из 500 рядов месячной частоты.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	16.216	19.935	710.185	872794.251
ARIMA_auto	16.824	19.296	791.994	1219394.770
ETS	16.159	19.333	676.599	662432.589
Prophet	24.029	34.121	1003.488	1595866.281
TimesNet (без разложения на компоненты MODWT)	22.387	30.063	1031.227	1952895.726
N-Beats (без разложения на компоненты MODWT)	23.301	34.896	1070.374	1783919.086
VW + TimesNet + Arima_auto	22.324	31.159	1068.190	2187207.953
VW_+_TimesNet_+_ETS	22.046	31.509	1039.062	2017867.408
VW + N-Beats + ETS	22.780	34.290	986.320	1510823.575
VW + N-Beats + Arima_auto	22.568	33.717	961.668	1442035.710

На месячной частоте ряды обычно короче, а основная структура (тренд/слабая сезонность) хорошо описывается ETS или ARIMA, поэтому разложение MODWT в такой ситуации может **дробить полезный низкочастотный сигнал на компоненты без добавочной информативности**, усложняя задачу модели, поэтому классические модели здесь одержали победу.

На рисунках 2.4.3.9 – 2.4.3.10 можно увидеть несколько результатов каскадного прогнозирования через построение графиков *rollout* симуляции на месячной частоте.

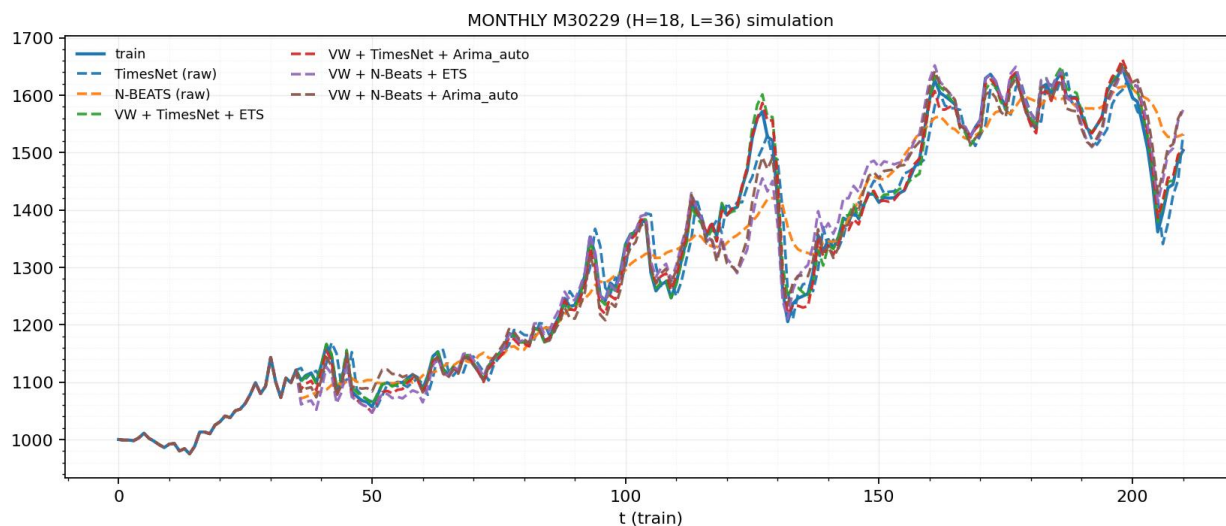


Рисунок 2.4.3.9. Rollout-симуляция исходного ряда M30229

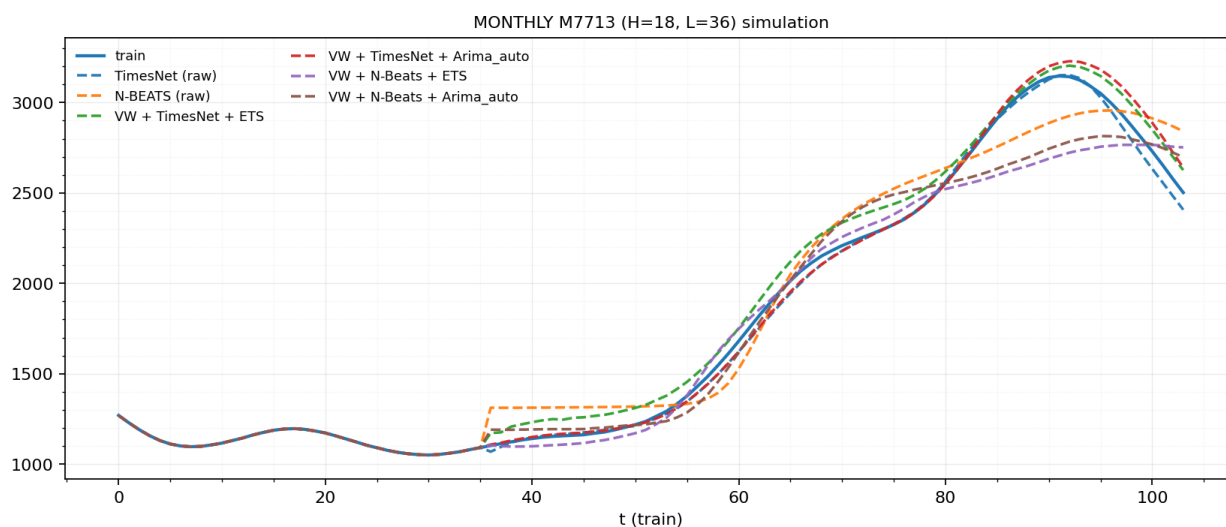


Рисунок 2.4.3.10. Rollout-симуляция исходного ряда M7713

В таблице 2.4.3.6 представлены результаты последней категории рядов из набора данных M4 по метрикам всех моделей, участвовавших в обучении на подвыборке из 359 рядов.

Таблица 2.4.3.6. Результаты моделей по метрикам, участвовавших в обучении на подвыборке из 359 рядов недельной частоты.

Модель	sMAPE	MAPE	RMSE	MSE
ARIMA	6.124	5.738	206.829	57390.930
ARIMA_auto	6.727	6.577	230.887	80590.405
ETS	5.374	5.952	224.529	107378.782
Prophet	10.324	9.789	305.469	114676.762
TimesNet (без разложения на компоненты MODWT)	6.396	7.235	263.742	137370.954
N-Beats (без разложения на компоненты MODWT)	5.161	5.524	195.809	51010.664
VW + TimesNet + Arima_auto	5.424	5.607	220.663	59217.798
VW + TimesNet + ETS	5.511	5.675	228.813	65970.071
VW + N-Beats + ETS	3.965	4.130	149.353	35069.643
VW + N-Beats + Arima_auto	4.627	4.867	175.984	48037.365

Недельные ряды часто содержат одновременно низкочастотную структуру (тренд/медленные изменения) и умеренные колебания, поэтому разложение MODWT помогает развести компоненты, после чего **ETS стабильно описывает гладкую часть**, а **N-Beats** лучше подхватывает остаточные нелинейности – поэтому данная модель оказалась лучшей как по относительным, так и по абсолютным метрикам.

На рисунках 2.4.3.11 – 2.4.3.12 можно увидеть несколько результатов каскадного прогнозирования через построение графиков *rollout* симуляции на недельной частоте.

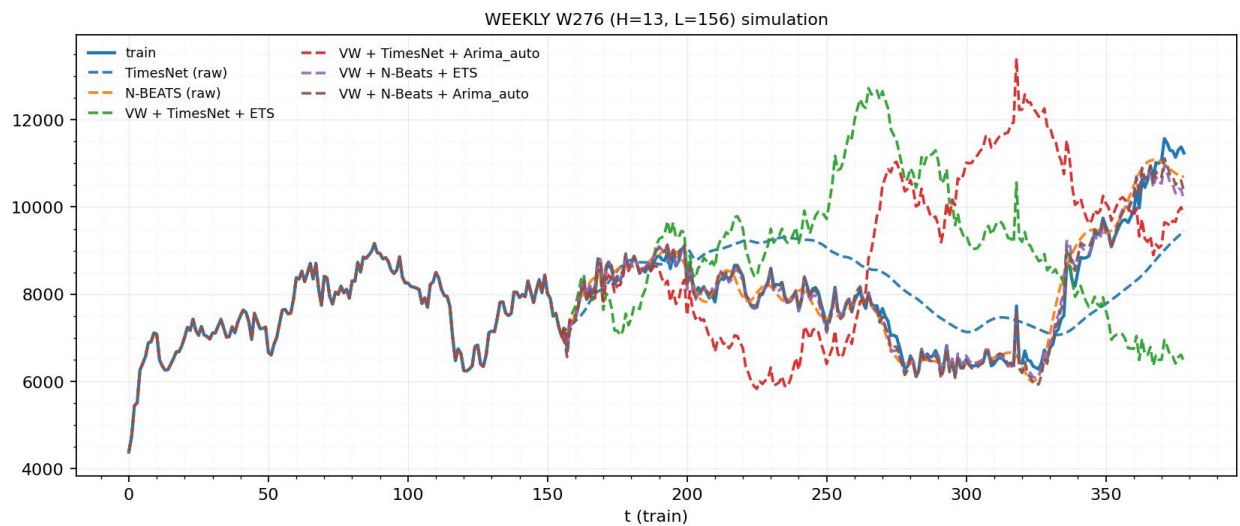


Рисунок 2.4.3.11. Rollout-симуляция исходного ряда W276

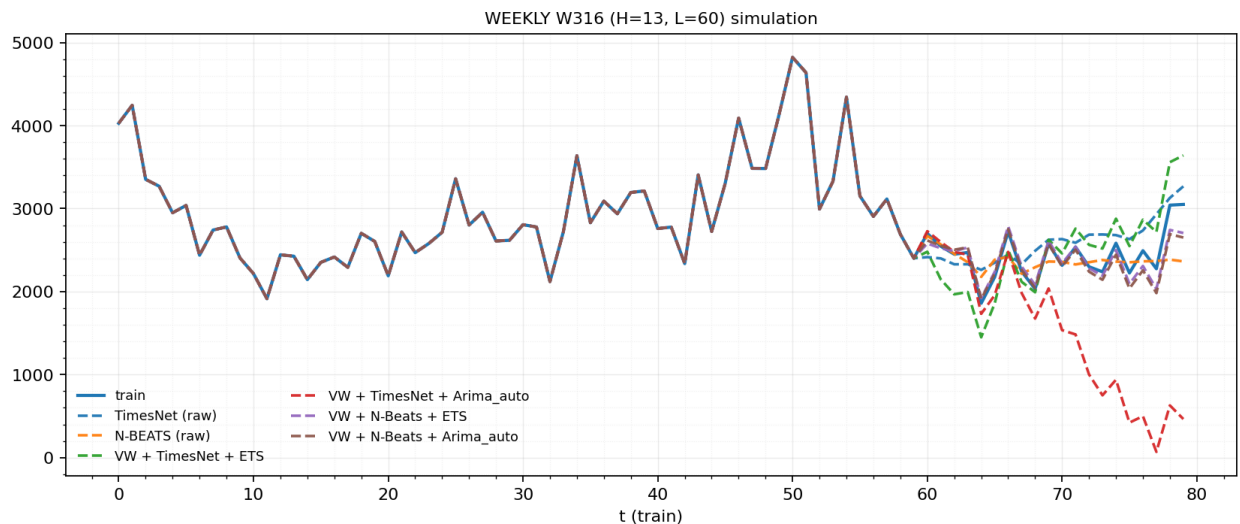


Рисунок 2.4.3.12. Rollout-симуляция исходного ряда W316

По результатам обучения и оценки на наборе M4 можно сделать вывод, что выбор оптимальной модели существенно зависит **от частоты и длины временных рядов**: единой архитектуры, стабильно превосходящей остальные во всех категориях, не наблюдается.

На **низкочастотных и коротких** рядах (годовых и месячных) наилучшие значения ошибок демонстрируют **классические статистические подходы (ETS/ARIMA)**, что объясняется ограниченным числом наблюдений и, как следствие, недостаточной информативностью данных для устойчивого обучения нейросетевых моделей.

На **высокочастотных** часовых рядах, напротив, лидирует **TimesNet без предварительного разложения**, тогда как использование разложения MODWT приводит

к ухудшению метрик, вероятно из-за фазовых и амплитудных искажений при разложении и последующей реконструкции сигнала, особенно в областях резких пиков.

В то же время для **дневных и недельных рядов гибридные архитектуры с предварительным разложением** оказываются наиболее эффективными: разбиение исходного сигнала на компоненты позволяет разделить гладкую низкочастотную динамику и более сложные остаточные колебания, после чего разные модели (ETS/ARIMA и TimesNet или N-BEATS) более точно аппроксимируют соответствующие части процесса, обеспечивая снижение как относительных (sMAPE/MAPE), так и абсолютных (RMSE/MSE) ошибок.

2.5. Тестирование моделей на рядах со сложной сезонностью

2.5.1. Описание набора данных

Чтобы оправдать сложность построенных моделей, проведем эксперимент на сценариях с многопериодной сезонностью, где генерируются синтетические временные ряды, в которых сезонная часть состоит **как минимум из двух компонент**.

В качестве структуры для генерации профилей со сложной сезонностью был построен класс *SynthProfile*, где сезонность описывается как основной компонентой, так и набором дополнительных компонент в виде троек (*период, амплитуда, фаза*).

Генерация ряда выполняется по формуле:

- Задаётся временная ось $t = 0, 1, \dots, T - 1$;
- Выстраивается линейный тренд, добавляется гауссов шум, а сезонность — сумма синусоид:

$$season(t) = \sum_k a_k \sin \left(2\pi \frac{t}{p_k} + \varphi_k \right)$$

В наборе профилей присутствуют четыре сценария сложной сезонности, отличающиеся комбинациями периодов (включая взаимно простые и имеющие общий делитель), уровнем шума и наличием тренда:

- **p12 + p20** (НОК = 60), без тренда, умеренный шум;
- **p7 + p15** (НОК = 105), с небольшим трендом, повышенный шум;

Для корректного задания контекстного окна при многопериодной сезонности используется идея покрытия общего цикла, то есть для профиля вычисляется **НОК всех сезонных периодов**, и этот НОК применяется как ориентир при подборе длины *lookback*.

2.5.2. Рассмотрение эвристики краткосрочного прогнозирования на компонентах MODWT в условиях многопериодной сезонности

Для каждого сценария сложной сезонности было сгенерировано по 500 временных рядов, а обучение моделей проводилось в течение 24 эпох. Число детальных компонент MODWT D_j было ограничено тремя — это значение выбрано эмпирически. В качестве граничного условия использовалось циклическое продолжение, что позволило снизить краевые артефакты при разложении синтетических сезонных сигналов.

Сравнивать результаты краткосрочного предсказания будем среди классических статистических моделей — ARIMA, ARIMA (с автоподбором), ETS, Prophet, чистых нейросетевых структур - TimesNet и N-Beats, декомпозиционных гибридов — MODWT+ TimesNet + ARIMA, MODWT+ TimesNet + ETS, MODWT+ N-Beats + ARIMA, MODWT+ N-Beats + ETS.

На рисунке 2.5.2.1 изображено предсказание на компонентах одного из сгенерированных рядов со сложной сезонностью, где паттерн повторяется каждые 60 точек.

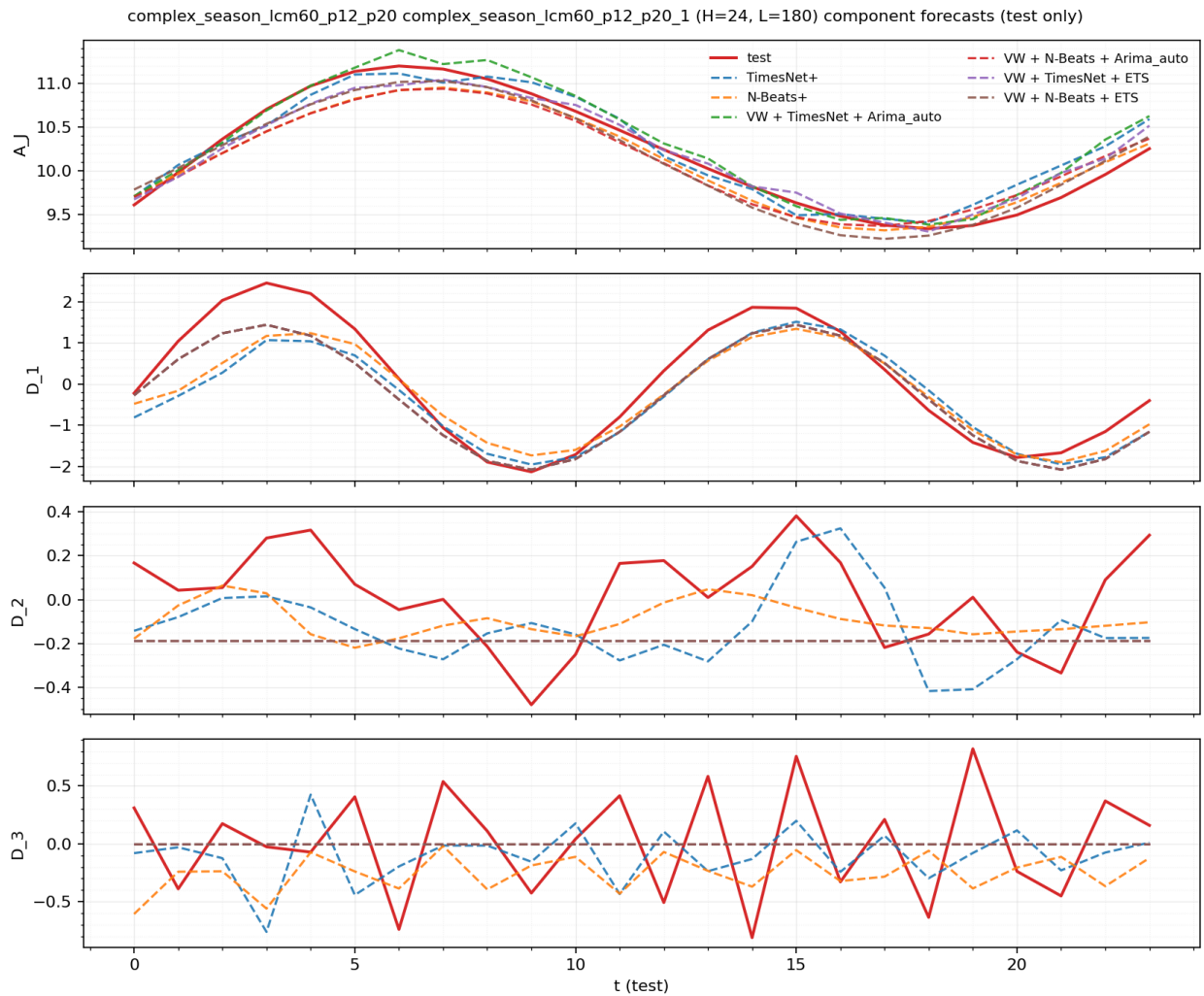


Рисунок 2.5.2.1. Краткосрочное предсказание на компонентах одного из сгенерированных рядов со сложной сезонностью, где паттерн повторяется каждые 60 точек.

Из рисунка 2.5.2.1 видно, что на тестовом горизонте модели достаточно уверенно восстанавливают низко- и среднечастотную структуру сигнала. Так, например, форма компоненты D_1 (основной сезонной волны) воспроизводится корректно по фазе и общей динамике. Вместе с тем для аппроксимационной компоненты A наблюдается систематическое смещение прогноза вверх (завышение уровня и амплитуды), наиболее выраженное у предсказывающей нейросетевой архитектуры TimesNet. Для более высокочастотных компонент прогнозы статистическими моделями заметно сглажены и тяготеют к нулю, в результате чего резкие локальные колебания и выбросы, присутствующие в истинных компонентах, моделями в основном не воспроизводятся и в последствии при реконструкции прогноза имеют самую низкую значимость.

На рисунке 2.5.2.2 отображающего предсказание на компонентах ряда со сложной сезонностью (повторение паттерна через 105 точек), повышенным шумом и небольшим трендом.

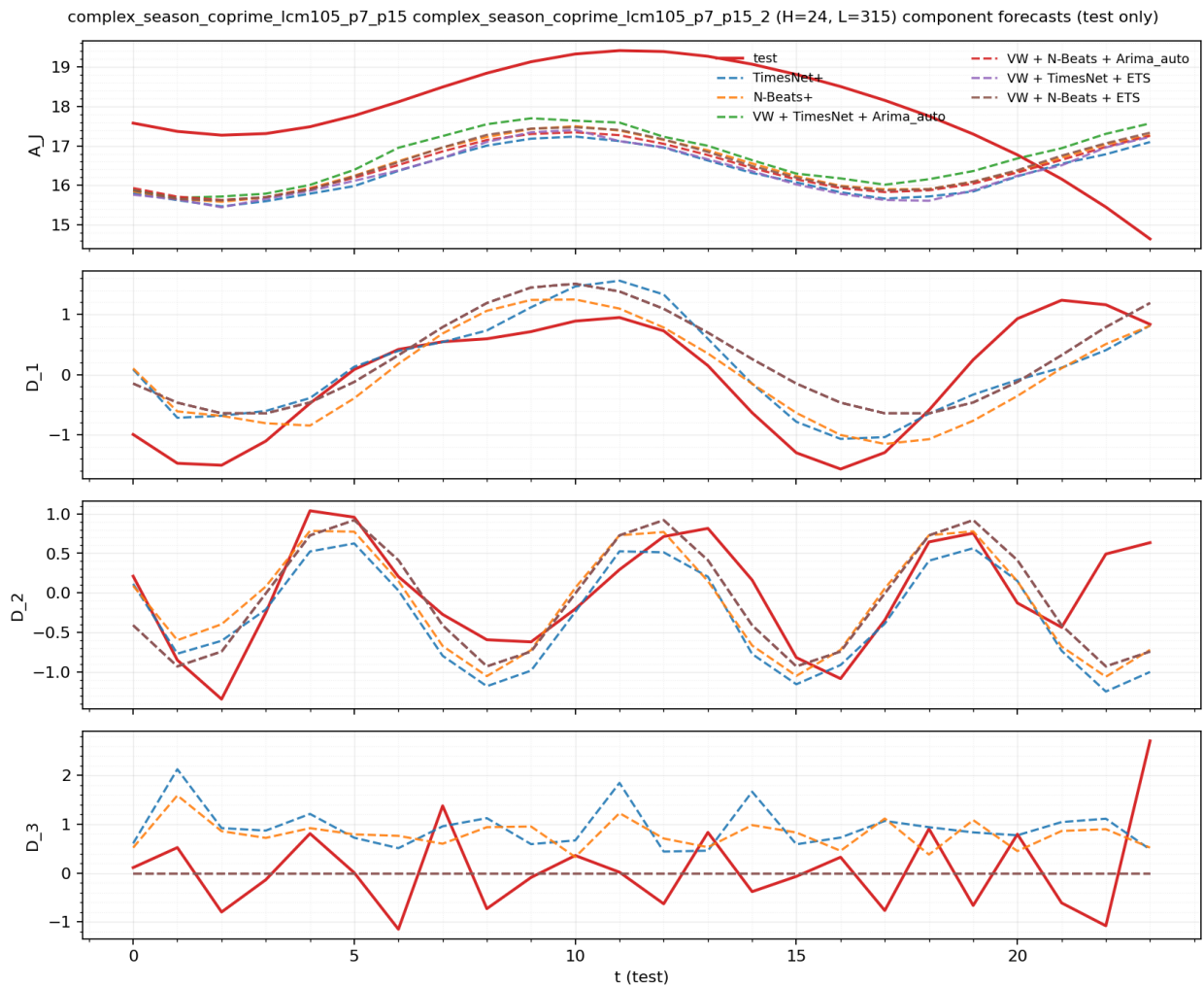


Рисунок 2.5.2.2. Краткосрочное предсказание на компонентах одного из сгенерированных рядов со сложной сезонностью, где паттерн повторяется каждые 105 точек.

Из рисунка 2.5.2.2 видно, что на профиле с взаимно простыми периодами ситуация несколько осложняется по сравнению с предыдущей ситуацией. Теперь основная ошибка прогноза связана с аппроксимационной компонентой A : все модели дают сглаженную траекторию с заниженной амплитудой и не воспроизводят выраженный максимум и последующее резкое снижение в конце горизонта. Компоненты D_1 и D_2 в целом передают форму, при этом экстремумы часто сглаживаются. Высокочастотная компонента предсказывается слабо: модели либо стремятся к нулю, либо демонстрируют смещение, не воспроизводя локальные выбросы, включая финальный пик.

Таким образом, в условиях многопериодной сезонности модели в целом уверенно восстанавливают **низко- и среднечастотную структуру** (вклад компонент $D_1 - D_2$), тогда как **основные ошибки прогноза** чаще всего связаны с аппроксимационной компонентой A (систематическое смещение уровня или недобор амплитуды, особенно при длинном

совместном цикле и наличии тренда) и с высокочастотными остатками, которые по своей природе ближе к шуму и потому сглаживаются – что и является эвристикой на рядах со сложной сезонностью.

2.5.3. Результаты обучения на рядах со сложной сезонностью

Если гибридным архитектурам трудно получать качественное предсказание на рядах со сложной сезонностью, то как будут себя вести классические статистические модели и нейросетевые архитектуры без разложения?

В таблице 2.5.3.1 приведены усредненные результаты на выборке из 500 рядов с НОК = 60.

Таблица 2.5.3.1. Усредненные результаты моделей по метрикам, участвовавших в обучении на подвыборке из 500 рядов с НОК = 60.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	21.1802	22.5048	2.5078	6.3159
ARIMA_auto	20.6365	21.8431	2.4225	5.8812
ETS	15.5499	17.5680	1.7954	3.5315
Prophet	18.8324	20.2349	2.1795	4.7528
TimesNet (без разложения на компоненты MODWT)	7.3432	7.6421	0.8236	0.6787
N-Beats (без разложения на компоненты MODWT)	6.7896	7.0241	0.7760	0.6043
VW + TimesNet + Arima_auto	6.2190	6.6163	0.8514	0.7314
VW_ + TimesNet_ + ETS	6.1780	6.3732	0.8472	0.7177
VW + N-Beats + ETS	6.3280	6.6712	0.8510	0.7242
VW + N-Beats + Arima_auto	6.2280	6.4722	0.8363	0.6995

По таблице 2.5.3.1 видно, что нейросетевые и декомпозиционные подходы существенно превосходят классические модели ARIMA/ETS/Prophet по всем метрикам качества.

Абсолютная ошибка у классических методов остаётся на уровне 1.8–2.5, тогда как у нейросетевых и гибридных решений — порядка 0.78–0.85. Среди моделей без декомпозиции на компоненты MODWT наилучшие значения RMSE и MSE демонстрирует N-Beats, что указывает на способность нейросетевой архитектуры эффективно восстанавливать структуру сигнала при сравнительно коротком общем цикле. При этом декомпозиционные MODWT-гибриды дают более низкие значения относительных ошибок (*sMAPE/MAPE*), и это ожидаемо, ведь декомпозиция + статистика на деталях помогает подправить относительные метрики (в процентах).

В таблице 2.5.3.2 приведены усредненные результаты на выборке из 500 рядов с НОК = 105.

Таблица 2.5.3.2. Усредненные результаты моделей по метрикам, участвовавших в обучении на подвыборке из 500 рядов с НОК = 105.

Модель	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
ARIMA	25.0001	21.4701	2.7817	7.9298
ARIMA_auto	25.6806	21.9780	2.8433	8.2536
ETS	15.0421	13.5639	1.6982	2.9915
Prophet	13.8267	14.0810	1.6475	2.7648
TimesNet (без разложения на компоненты MODWT)	8.3323	8.1832	1.0051	1.0234
N-Beats (без разложения на компоненты MODWT)	7.3963	7.2040	0.8994	0.8537
VW + TimesNet + Arima_auto	6.1233	6.0964	0.7719	0.6114
VW_+ TimesNet_+ ETS	7.7637	7.8657	0.9309	0.8728
VW + N-Beats + ETS	7.0451	6.9905	0.8527	0.7426
VW + N-Beats + Arima_auto	6.3439	6.2106	0.7944	0.6697

По таблице 2.5.3.2 видно, что при усложнении многопериодной сезонности декомпозиционные гибриды начинают давать устойчивое преимущество над моделями без разложения. Лучшие значения по всем рассматриваемым метрикам демонстрирует гибридный **MODWT + TimesNet + ARIMA**, а близкие результаты показывает **MODWT + N-Beats + ARIMA**. Чистые нейросетевые архитектуры уступают гибридам, хотя и существенно превосходят классические модели, для которых абсолютные ошибки остаются заметно выше. Таким образом, при большем общем цикле разделение сигнала на низко- и высокочастотные компоненты и отдельное моделирование деталей статистическими методами улучшает как относительные, так и абсолютные показатели качества прогноза, что оправдывает применение метода «разделяй и властвуй».

Заключение

В рамках выполненной НИР были систематизированы и сопоставлены классические и современные подходы к краткосрочному прогнозированию одномерных временных рядов: модели семейства ARIMA/ETS/Prophet, а также нейросетевые архитектуры TimesNet и N-BEATS, дополненные идеей многомасштабной декомпозиции с помощью MODWT.

Практическая часть была направлена на проверку того, в каких условиях разложение ряда на компоненты действительно повышает качество прогноза: сначала на контролируемых синтетических профилях, затем на реальных рядах M3 и далее - на rollout-симуляциях и агрегированной оценке по частотным категориям набора M4.

По результатам экспериментов **на синтетических рядах** видно, что при правильном совпадении предположений модели с природой данных ETS оказывается наиболее стабильным и точным базовым методом (особенно по абсолютным ошибкам), а среди гибридов наиболее уверенно проявляют себя схемы семейства **MODWT с N-BEATS и ETS/ARIMA_auto** за счёт разделения низко и высокочастотных компонент. При этом **ARIMA/ARIMA_auto** без корректной сезонной части могут резко деградировать на профилях с выраженной периодичностью, тогда как ETS и нейросетевые гибриды сохраняют приемлемое качество даже при высоком шуме.

На реальных данных M3 (по 100 рядов на yearly/quarterly/monthly) при вычислительных ограничениях основное сравнение проводилось между классическими подходами и MODWT-гибридами, что позволило подтвердить неоднозначность «универсального лидера» и зависимость результата от выбора метрики: по относительным ошибкам гибридные схемы чаще оказываются предпочтительнее, тогда как по абсолютным метрикам ETS остаётся сильным конкурентным ориентиром.

В агрегированных итогах наблюдается важная закономерность: разные метрики выбирают разных лидеров —

- по sMAPE лучше всего выступает **VW + N – Beats + ETS (22.520)**,
- по MAPE — **VW + TimesNet + ETS (30.722)**,
- по RMSE минимальное значение сохраняет **ETS (1197.523)**,

что подчёркивает пользу гибридов именно для относительных ошибок при сохранении сильной конкурентоспособности ETS по абсолютной шкале.

Дополнительные эксперименты на данных M4, агрегированные по частотным категориям, показали, что эффективность разложения MODWT существенно зависит от частоты — на **дневных и недельных рядах гибридные архитектуры** дают наибольший выигрыш, на **месячных и годовых рядах** чаще выигрывают **классические ETS/ARIMA** из-за малой длины рядов, а на **часовых** данных разложение в текущей постановке приводит к ухудшению качества, вероятно из-за фазовых и амплитудных искажений при декомпозиции и реконструкции сигнала на высоких частотах.

На примере quarterly-ряда M4 (Q2143) показано, что гибриды **MODWT + TimesNet** дают наилучшие показатели:

- $sMAPE/MAPE = 3.915 / 3.804$ (лучший с ETS)

- $RMSE/MSE = 52.398 / 2745.568$ (лучший с ARIMA (с автоподбором))

Исходя из полученных результатов, можно сформулировать следующие практические рекомендации выбора модели:

1. Если целевая метрика — **относительная ошибка ($sMAPE/MAPE$)** и рассматриваются **дневные или недельные ряды**, целесообразно применять **MODWT-гибриды** (в частности, связки с **TimesNet/ N-BEATS и ETS**), поскольку они стабильно снижают ошибки за счёт разделения компонент.
2. Если данные относятся к месячной или годовой частоте (короткие ряды), предпочтительнее классические ETS/ARIMA, так как нейросетевые и гибридные модели в этих условиях чаще не реализуют преимущества из-за ограниченного объёма наблюдений.
3. Для **часовых рядов** в рассматриваемой постановке оптимальным является использование **TimesNet без разложения**, а применение MODWT-гибридов требует отдельной адаптации (например, иной схемы декомпозиции или процедур согласования фаз), поскольку в текущем виде приводит к росту ошибок.

Список литературы

1. Box G.E.P. Time series analysis: forecasting and control / G.E.P. Box, G.M. Jenkins, G.C. Reinsel, G.M. Ljung. – 5th ed. – Hoboken, NJ: John Wiley & Sons, 2015. – 712 p.
2. Brockwell P.J. Introduction to time series and forecasting / P.J. Brockwell, R.A. Davis. – 3rd ed. – Cham: Springer, 2016. – 425 p.
3. Makridakis S. The M4 Competition: Results, findings, conclusion and way forward / S. Makridakis, E. Spiliotis, V. Assimakopoulos // International Journal of Forecasting. – 2018. – Vol. 34, no. 4. – P. 802–808. – DOI: 10.1016/j.ijforecast.2018.06.001.
4. Makridakis S. The M3-Competition: results, conclusions and implications / S. Makridakis, M. Hibon // International Journal of Forecasting. – 2000. – Vol. 16, no. 4. – P. 451–476. – DOI: 10.1016/S0169-2070(00)00057-1.
5. Percival D.B. Wavelet methods for time series analysis / D.B. Percival, A.T. Walden. – Cambridge: Cambridge University Press, 2000. – 594 p.
6. Zhu L. MODWT-ARMA model for time series prediction / L. Zhu, Y. Wang, Q. Fan // Applied Mathematical Modelling. – 2014. – Vol. 38, no. 5–6. – P. 1859–1865. – DOI: 10.1016/j.apm.2013.10.002.
7. Adib A. On the reliability of a novel MODWT-based hybrid ARIMA-artificial intelligence approach to forecast daily Snow Depth (Case study: The western part of the Rocky Mountains in the U.S.A) / A. Adib, A. Zaerpour, M. Lotfirad // Cold Regions Science and Technology. – 2021. – Vol. 189. – Art. 103342. – DOI: 10.1016/j.coldregions.2021.103342.
8. Li Y. Multi-step ahead wind speed forecasting approach coupling maximal overlap discrete wavelet transform, improved grey wolf optimization algorithm and long short-term memory / Y. Li, T. Peng, C. Zhang, W. Sun, L. Hua, C. Ji, N.M. Shahzad // Renewable Energy. – 2022. – Vol. 196. – P. 1115–1126. – DOI: 10.1016/j.renene.2022.07.016.
9. Hyndman R.J. Another look at measures of forecast accuracy / R.J. Hyndman, A.B. Koehler // International Journal of Forecasting. – 2006. – Vol. 22, no. 4. – P. 679–688. – DOI: 10.1016/j.ijforecast.2006.03.001.
10. Hyndman R.J. Automatic time series forecasting: the forecast package for R / R.J. Hyndman, Y. Khandakar // Journal of Statistical Software. – 2008. – Vol. 27, no. 3. – P. 1–22. – DOI: 10.18637/jss.v027.i03.
11. Hyndman R.J. A state space framework for automatic forecasting using exponential smoothing methods / R.J. Hyndman, A.B. Koehler, R.D. Snyder, S. Grose // International Journal of Forecasting. – 2002. – Vol. 18, no. 3. – P. 439–454. – DOI: 10.1016/S0169-2070(01)00110-8.
12. Holt C.C. Forecasting seasonals and trends by exponentially weighted moving averages / C.C. Holt // International Journal of Forecasting. – 2004. – Vol. 20, no. 1. – P. 5–10. – DOI: 10.1016/j.ijforecast.2003.09.015.
13. Winters P.R. Forecasting sales by exponentially weighted moving averages / P.R. Winters // Management Science. – 1960. – Vol. 6, no. 3. – P. 324–342.
14. Taylor S.J. Forecasting at scale / S.J. Taylor, B. Letham // The American Statistician. – 2018. – Vol. 72, no. 1. – P. 37–45. – DOI: 10.1080/00031305.2017.1380080.
15. Mallat S.G. A theory for multiresolution signal decomposition: the wavelet representation / S.G. Mallat // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 1989. – Vol. 11, no. 7. – P. 674–693. – DOI: 10.1109/34.192463.
16. Daubechies I. Ten lectures on wavelets / I. Daubechies. – Philadelphia: SIAM, 1992. – 357 p.
17. Kedadouche M. A comparative study between Empirical Wavelet Transforms and Empirical Mode Decomposition Methods: Application to bearing defect diagnosis / M.

- Kedadouche, M. Thomas, A. Tahan // *Mechanical Systems and Signal Processing*. – 2016. – DOI: 10.1016/j.ymssp.2016.02.049.
18. Li Y. Comparison of empirical modal decomposition class techniques applied in noise cancellation for building heating consumption prediction based on time-frequency analysis / Y. Li, N. Zhu, Y. Hou // *Energy and Buildings*. – 2023. – Vol. 284. – Art. 112853. – DOI: 10.1016/j.enbuild.2023.112853.
 19. Gao R. Time series forecasting based on echo state network and empirical wavelet transformation / R. Gao, L. Du, O. Duru, K.F. Yuen // *Applied Soft Computing*. – 2021. – Vol. 102. – Art. 107111. – DOI: 10.1016/j.asoc.2021.107111.
 20. Zhang G.P. Time series forecasting using a hybrid ARIMA and neural network model / G.P. Zhang // *Neurocomputing*. – 2003. – Vol. 50. – P. 159–175. – DOI: 10.1016/S0925-2312(01)00702-0.
 21. Montero-Manso P. FFORMA: Feature-based forecast model averaging / P. Montero-Manso, G. Athanasopoulos, R.J. Hyndman, T.S. Talagala // *International Journal of Forecasting*. – 2020. – Vol. 36, no. 1. – P. 86–92. – DOI: 10.1016/j.ijforecast.2019.02.011.
 22. Smyl S. A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting / S. Smyl // *International Journal of Forecasting*. – 2020. – Vol. 36, no. 1. – P. 75–85. – DOI: 10.1016/j.ijforecast.2019.03.017.
 23. Oreshkin B. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting [Электронный ресурс] / B. Oreshkin, D. Carпов, N. Chapados, Y. Bengio. – 2019. – Режим доступа: <https://arxiv.org/abs/1905.10437> (дата обращения: 25.02.2026), свободный.
 24. Olivares K.G. Neural basis expansion analysis with exogenous variables: Forecasting electricity prices with NBEATSx / K.G. Olivares, C. Challu, G. Marcjasz, R. Weron, A. Dubrawski // *International Journal of Forecasting*. – 2023. – DOI: 10.1016/j.ijforecast.2022.03.001.
 25. Van Belle J. Improving forecast stability using deep learning / J. Van Belle, R. Crevits, W. Verbeke // *International Journal of Forecasting*. – 2023. – Vol. 39, no. 3. – P. 1333–1350. – DOI: 10.1016/j.ijforecast.2022.06.007.
 26. Nie Y. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers [Электронный ресурс] / Y. Nie, N. Nguyen, P. Sinthong, J. Kalagnanam. – 2022. – arXiv:2211.14730 [cs.LG]. – Режим доступа: <https://arxiv.org/abs/2211.14730> (дата обращения: 25.02.2026), свободный.
 27. Wu H. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis [Электронный ресурс] / H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long. – 2022. – Режим доступа: <https://arxiv.org/abs/2210.02186> (дата обращения: 25.02.2026), свободный.
 28. Huang Y. TimesNet-PM2.5: Interpretable TimesNet for Disentangling Intraproduct and Interperiod Variations in PM2.5 Prediction / Y. Huang, Z. Zhou, Z. Wang, X. Zhi, X. Liu // *Atmosphere*. – 2023. – Vol. 14, no. 11. – Art. 1604. – DOI: 10.3390/atmos14111604.
 29. Wu H. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting [Электронный ресурс] / H. Wu, J. Xu, J. Wang, M. Long. – 2021. – arXiv:2106.13008 [cs.LG]. – Режим доступа: <https://arxiv.org/abs/2106.13008> (дата обращения: 25.02.2026), свободный.
 30. Zhou H. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting [Электронный ресурс] / H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang // *Proceedings of the AAAI Conference on Artificial Intelligence*. – 2021. – (AAAI-21). – Режим доступа: <https://cdn.aaai.org/ojs/17325/17325-13-20819-1-2-20210518.pdf> (дата обращения: 25.02.2026), свободный.
 31. Liu Y. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting [Электронный ресурс] / Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long. –

2024. – arXiv:2310.06625v4 [cs.LG]. – Режим доступа: <https://arxiv.org/abs/2310.06625v4> (дата обращения: 25.02.2026), свободный.
32. Eldele E. TSLANet: Rethinking Transformers for Time Series Representation Learning [Электронный ресурс] / E. Eldele [и др.]. – 2024. – arXiv:2404.08472v2 [cs.LG]. – Режим доступа: <https://arxiv.org/abs/2404.08472v2> (дата обращения: 25.02.2026), свободный.
33. Yang X.Y. Research on information leakage in time series prediction based on empirical mode decomposition / X.Y. Yang, J.Y. Li, X.C. Jiang // Scientific Reports. – 2024. – Vol. 14. – Art. 28363. – DOI: 10.1038/s41598-024-80018-9.
34. Huang N.E. The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis / N.E. Huang, Z. Shen, S.R. Long [и др.] // Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences. – 1998. – Vol. 454, no. 1971. – P. 903–995. – DOI: 10.1098/rspa.1998.0193.
35. Lazcano de Rojas A. EMDFormer model for time series forecasting / A. Lazcano de Rojas, M.A. Jaramillo-Morán, J.E. Sandubete // AIMS Mathematics. – 2024. – Vol. 9, no. 4. – P. 9419–9434. – DOI: 10.3934/math.2024459.
36. Dragomiretskiy K. Variational Mode Decomposition / K. Dragomiretskiy, D. Zosso // IEEE Transactions on Signal Processing. – 2014. – Vol. 62, no. 3. – P. 531–544. – DOI: 10.1109/TSP.2013.2288675.
37. Putra H.R.K. Variational Mode Decomposition and Linear Embeddings are What You Need For Time-Series Forecasting [Электронный ресурс] / H.R.K. Putra, N. Yudistira, T.N. Fatyanosa. – 2024. – arXiv:2408.16122v2 [cs.LG]. – Режим доступа: <https://arxiv.org/abs/2408.16122> (дата обращения: 25.02.2026), свободный.
38. Ahajjam M.A. Experimental investigation of variational mode decomposition and deep learning for short-term multi-horizon residential electric load forecasting / M.A. Ahajjam, D. Bonilla Licea, M. Ghogho, A. Kobbane // Applied Energy. – 2022. – Vol. 326. – Art. 119963. – DOI: 10.1016/j.apenergy.2022.119963.
39. Li N. Variable Filtered-Waveform Variational Mode Decomposition and Its Application in Rolling Bearing Fault Feature Extraction / N. Li, H. Wang // Entropy. – 2025. – Vol. 27, no. 3. – Art. 277. – DOI: 10.3390/e27030277.
40. Zhang Q. Short-term load forecasting method with variational mode decomposition and stacking model fusion / Q. Zhang, J. Wu, Y. Ma, G. Li, J. Ma, C. Wang // Sustainable Energy, Grids and Networks. – 2022. – Art. 100622. – DOI: 10.1016/j.segan.2022.100622.
41. Zhang J. A hybrid approach of wavelet transforms, ARIMA and LSTM model for the share price index futures forecasting / J. Zhang, H. Liu, W. Bai, X. Li // The North American Journal of Economics and Finance. – 2024. – Vol. 69, Part B. – Art. 102022. – DOI: 10.1016/j.najef.2023.102022.
42. Kılıç D.K. Hybrid wavelet-neural network models for time series / D.K. Kılıç, Ö. Uğur // Applied Soft Computing. – 2023. – Vol. 144. – Art. 110469. – DOI: 10.1016/j.asoc.2023.110469.
43. Amini A. Real-time rainfall and runoff prediction by integrating BC-MODWT and automatically-tuned DNNs: Comparing different deep learning models / A. Amini, M. Dolatshahi, R. Kerachian // Journal of Hydrology. – 2024. – Vol. 631. – Art. 130804. – DOI: 10.1016/j.jhydrol.2024.130804.
44. Ni C. An integrated approach using empirical wavelet transform and deep learning network for power prediction of wave energy converter / C. Ni, [и др.] // Ocean Engineering. – 2023. – DOI: 10.1016/j.oceaneng.2023.114650.
45. Mohammadi H.A. Using empirical wavelet transform and high-order fuzzy cognitive maps for forecasting sharp-fluctuated time series / H.A. Mohammadi, [и др.] // Applied Soft Computing. – 2023. – DOI: 10.1016/j.asoc.2023.110469.

46. Guo Z. WEITS: A wavelet-enhanced residual framework for interpretable time series forecasting / Z. Guo, Y. Sun, T. Wu // *Neurocomputing*. – 2026. – Vol. 674. – Art. 132788. – DOI: 10.1016/j.neucom.2026.132788.
47. Li H. The importance of data noise reduction–wavelet transformation in stock price forecasting with Bidirectional Long Short-Term Memory network / H. Li, Y. Marudhasalam, Z. Huang, J. Yu // *Engineering Applications of Artificial Intelligence*. – 2026. – Vol. 165. – Art. 113390. – DOI: 10.1016/j.engappai.2025.113390.
48. Hassanzada M.J. Exploring the role of wavelet decomposition order in deep learning-based network-wide traffic prediction / M.J. Hassanzada, I. Yamnenko, C. Antoniou // *Journal of Traffic and Transportation Engineering (English Edition)*. – 2026. – DOI: 10.1016/j.jtte.2024.08.003.

Приложение

Полный код представлен в репозитории «<https://github.com/sheeelby/Hybrid-models-for-TS-processing>».

```
from __future__ import annotations

import math

import torch
import torch.nn as nn

class NBEATBlock(nn.Module):
    def __init__(
        self,
        lookback: int,
        horizon: int,
        width: int = 256,
        depth: int = 4,
        dropout: float = 0.1,
        block_type: str = "generic",
        polynomial_degree: int = 3,
        n_harmonics: int | None = None,
    ):

        super().__init__()
        self.lookback = lookback
        self.horizon = horizon
        self.block_type = block_type

        layers = []
        in_features = lookback
        for _ in range(depth):
            layers.extend(
                [
                    nn.Linear(in_features, width),
                    nn.ReLU(),
                    nn.Dropout(dropout),
                ]
            )
            in_features = width
        self.fc = nn.Sequential(*layers)

        if block_type == "generic":
            self.theta_b = nn.Linear(width, lookback)
            self.theta_f = nn.Linear(width, horizon)
            self.register_buffer("backcast_basis", None, persistent=False)
            self.register_buffer("forecast_basis", None, persistent=False)
            self.n_theta_backcast = lookback
```

```

self.n_theta_forecast = horizon
else:
    if block_type == "trend":
        degree = max(1, int(polynomial_degree))
        n_theta_backcast = degree + 1
        n_theta_forecast = degree + 1

        scale = float(max(1, lookback))
        t_b = torch.arange(-lookback + 1, 1, dtype=torch.float32) / scale
        t_f = torch.arange(1, horizon + 1, dtype=torch.float32) / scale
        backcast_basis = torch.stack([t_b ** i for i in range(n_theta_backcast)], dim=0)
        forecast_basis = torch.stack([t_f ** i for i in range(n_theta_forecast)], dim=0)
    elif block_type == "seasonality":
        max_harm = lookback // 2
        if max_harm <= 0:
            max_harm = 1
        K = int(n_harmonics) if n_harmonics is not None else min(10, max_harm)
        K = max(1, K)

        scale = float(max(1, lookback))
        t_b = torch.arange(-lookback + 1, 1, dtype=torch.float32) / scale
        t_f = torch.arange(1, horizon + 1, dtype=torch.float32) / scale
        freqs = torch.arange(1, K + 1, dtype=torch.float32)

        ang_b = 2 * math.pi * freqs.unsqueeze(1) * t_b.unsqueeze(0)
        ang_f = 2 * math.pi * freqs.unsqueeze(1) * t_f.unsqueeze(0)
        cos_b = torch.cos(ang_b)
        sin_b = torch.sin(ang_b)
        cos_f = torch.cos(ang_f)
        sin_f = torch.sin(ang_f)

        backcast_basis = torch.cat([cos_b, sin_b], dim=0)
        forecast_basis = torch.cat([cos_f, sin_f], dim=0)
        n_theta_backcast = backcast_basis.shape[0]
        n_theta_forecast = forecast_basis.shape[0]
    else:
        raise ValueError(f"Unsupported N-BEATS block_type: {block_type}")

self.n_theta_backcast = n_theta_backcast
self.n_theta_forecast = n_theta_forecast
self.theta = nn.Linear(width, n_theta_backcast + n_theta_forecast)
self.register_buffer("backcast_basis", backcast_basis, persistent=False)
self.register_buffer("forecast_basis", forecast_basis, persistent=False)

def forward(self, x: torch.Tensor):
    hidden = self.fc(x)
    if self.block_type == "generic":
        backcast = self.theta_b(hidden)
        forecast = self.theta_f(hidden)
    return backcast, forecast

```

```

theta = self.theta(hidden)
theta_b, theta_f = torch.split(
    theta, [self.n_theta_backcast, self.n_theta_forecast], dim=-1
)
backcast = torch.matmul(theta_b, self.backcast_basis)
forecast = torch.matmul(theta_f, self.forecast_basis)
return backcast, forecast

```

```
class NBEATSSStack(nn.Module):
```

```
    """Doubly residual stacking with optional weight sharing inside a stack."""
```

```

def __init__(
    self,
    lookback: int,
    horizon: int,
    *,
    width: int,
    depth: int,
    nblocks: int,
    dropout: float,
    block_type: str,
    polynomial_degree: int,
    n_harmonics: int | None,
    share_weights: bool = True,
) -> None:
    super().__init__()
    self.nblocks = int(max(1, nblocks))
    self.share_weights = bool(share_weights)

    if self.share_weights:
        self.block = NBEATBlock(
            lookback,
            horizon,
            width=width,
            depth=depth,
            dropout=dropout,
            block_type=block_type,
            polynomial_degree=polynomial_degree,
            n_harmonics=n_harmonics,
        )
        self.blocks = None
    else:
        self.block = None
        self.blocks = nn.ModuleList(
            [
                NBEATBlock(
                    lookback,
                    horizon,

```

```

        width=width,
        depth=depth,
        dropout=dropout,
        block_type=block_type,
        polynomial_degree=polynomial_degree,
        n_harmonics=n_harmonics,
    )
    for _ in range(self.nblocks)
]
)

def forward(self, backcast: torch.Tensor, forecast: torch.Tensor):
    if self.share_weights and self.block is not None:
        for _ in range(self.nblocks):
            b, f = self.block(backcast)
            backcast = backcast - b
            forecast = forecast + f
        return backcast, forecast

    if self.blocks is None:
        return backcast, forecast
    for blk in self.blocks:
        b, f = blk(backcast)
        backcast = backcast - b
        forecast = forecast + f
    return backcast, forecast

class NBEATSV2(nn.Module):
    def __init__(
        self,
        lookback: int,
        horizon: int,
        width: int = 256,
        depth: int = 4,
        nblocks: int = 4,
        dropout: float = 0.1,
        use_anchor: bool = False,
        share_weights: bool = True,
        use_trend: bool = True,
        use_seasonality: bool = True,
        use_generic: bool = True,
        diff_loss_weight: float = 0.08,
    ):
        super().__init__()
        self.use_anchor = bool(use_anchor)
        self.horizon = int(horizon)
        self.diff_loss_weight = float(diff_loss_weight)
        self.stack_kinds = []

```

```

if use_trend:
    stack_kinds.append("trend")
if use_seasonality:
    stack_kinds.append("seasonality")
if use_generic:
    stack_kinds.append("generic")
if not stack_kinds:
    stack_kinds = ["generic"]
stack_kinds = tuple(stack_kinds)
nblocks = int(max(1, nblocks))
base = nblocks // len(stack_kinds)
extra = nblocks % len(stack_kinds)
counts = [base + (i < extra) for i in range(len(stack_kinds))]
poly_deg = 3 if self.horizon < 18 else 4
n_harm = None
self.stacks = nn.ModuleList(
    [
        NBEATSSStack(
            lookback,
            self.horizon,
            width=width,
            depth=depth,
            nblocks=counts[i],
            dropout=dropout,
            block_type=kind,
            polynomial_degree=poly_deg,
            n_harmonics=n_harm,
            share_weights=share_weights,
        )
        for i, kind in enumerate(stack_kinds)
        if counts[i] > 0
    ]
)

def forward(self, x: torch.Tensor):
    batch, _, lookback = x.shape
    backcast = x.view(batch, lookback)
    forecast = torch.zeros(batch, self.horizon, device=x.device, dtype=x.dtype)
    for stack in self.stacks:
        backcast, forecast = stack(backcast, forecast)
    if (not self.training) and forecast.size(1) >= 3:
        diffs = torch.abs(forecast[:, 1:] - forecast[:, :-1]) # (B, H-1)
        rest = diffs[:, 1:] if diffs.size(1) >= 3 else diffs
        scale = torch.quantile(rest, 0.75, dim=1, keepdim=True) + 1e-6
        d01 = torch.abs(forecast[:, 0] - forecast[:, 1]).unsqueeze(1)
        d12 = torch.abs(forecast[:, 1] - forecast[:, 2]).unsqueeze(1)
        mask = (d01 > 6.0 * scale) & (d12 < 2.5 * scale)
        if torch.any(mask):
            forecast = forecast.clone()
            forecast[:, 0] = torch.where(mask.squeeze(1), forecast[:, 1], forecast[:, 0])

```



```

if not self.use_anchor:
    return forecast
anchor = x[:, :, -1].squeeze(1).unsqueeze(1)
return forecast + anchor

```

```
__all__ = ["NBEATSV2"]
```

Листинг 1. Архитектура нейросетевой модели N-Beats.

```

from __future__ import annotations

import torch
import torch.nn as nn

from .timesnet_blocks import MSUnit

class TimesBlockV2(nn.Module):
    def __init__(
        self,
        lookback: int,
        d_model: int,
        topk: int = 3,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()
        self.lookback = int(lookback)
        self.d_model = int(d_model)
        self.topk = int(topk)
        hidden_ch = max(16, min(64, self.d_model // 2))
        self.msunit = MSUnit(self.d_model, hidden_ch=hidden_ch, dropout=dropout)
        self.dropout = nn.Dropout(dropout)
        self.norm1 = nn.LayerNorm(self.d_model)
        self.norm2 = nn.LayerNorm(self.d_model)
        ffn_hidden = max(128, 4 * self.d_model)
        self.ffn = nn.Sequential(
            nn.Conv1d(self.d_model, ffn_hidden, kernel_size=1),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Conv1d(ffn_hidden, self.d_model, kernel_size=1),
            nn.Dropout(dropout),
        )

    def _ln(self, x: torch.Tensor, norm: nn.LayerNorm) -> torch.Tensor:
        return norm(x.transpose(1, 2)).transpose(1, 2)

    def _reshape_2d(self, x: torch.Tensor, period: int):
        batch, channels, length = x.shape

```

```

cols = (length + period - 1) // period
total = cols * period
pad = total - length
if pad > 0:
    last = x[:, :, -1:].expand(batch, channels, pad)
    x = torch.cat([x, last.to(device=x.device, dtype=x.dtype)], dim=-1)
reshaped = x.view(batch, channels, cols, period).transpose(-1, -2).contiguous()
return reshaped, pad, length

def _reshape_back(self, x2d: torch.Tensor, pad: int, original_len: int):
    batch, channels, period, freq = x2d.shape
    flattened = x2d.transpose(-1, -2).contiguous().view(batch, channels, period * freq)
    if pad > 0:
        flattened = flattened[..., :-pad]
    if flattened.shape[-1] != original_len:
        flattened = flattened[..., :original_len]
    return flattened

def forward(self, x: torch.Tensor) -> torch.Tensor:
    return self.forward_with_periods(x, period_list=None, weights=None)

def forward_with_periods(
    self,
    x: torch.Tensor,
    *,
    period_list: list[int] | None,
    weights: torch.Tensor | None,
) -> torch.Tensor:
    series_len = x.shape[-1]
    if period_list is None or weights is None:
        period_list = [series_len]
        weights = torch.ones(x.size(0), 1, device=x.device, dtype=x.dtype)

    outputs = []
    for period in period_list:
        x2d, pad, original_len = self._reshape_2d(x, period)
        y2d = self.msunit(x2d)
        outputs.append(self._reshape_back(y2d, pad, original_len))

    combined = torch.zeros_like(x)
    for i, out in enumerate(outputs):
        combined = combined + weights[:, i].view(-1, 1, 1) * out

    y = self._ln(x + self.dropout(combined), self.norm1)
    y = self._ln(y + self.ffn(y), self.norm2)
    return y

class TimesNetV2(nn.Module):
    def __init__(

```

```

self,
lookback: int,
horizon: int,
d_model: int = 64,
layers: int = 4,
topk: int = 3,
dropout: float = 0.1,
use_anchor: bool = True,
) -> None:
    super().__init__()
    self.lookback = int(lookback)
    self.horizon = int(horizon)
    self.d_model = int(d_model)
    self.topk = int(topk)
    self.use_anchor = bool(use_anchor)

    self.in_proj = nn.Conv1d(1, self.d_model, kernel_size=1)
    self.blocks = nn.ModuleList(
        [
            TimesBlockV2(
                lookback=self.lookback,
                d_model=self.d_model,
                topk=self.topk,
                dropout=dropout,
            )
            for _ in range(int(layers))
        ]
    )
    self.out_proj = nn.Conv1d(self.d_model, 1, kernel_size=1)

    head_hidden = max(128, 2 * self.d_model)
    self.head = nn.Sequential(
        nn.Linear(self.lookback, head_hidden),
        nn.GELU(),
        nn.Dropout(dropout),
        nn.Linear(head_hidden, head_hidden),
        nn.GELU(),
        nn.Dropout(dropout),
        nn.Linear(head_hidden, self.horizon),
    )

    def _periods(self, x_raw: torch.Tensor) -> tuple[list[int], torch.Tensor]:
        series_len = x_raw.shape[-1]
        amp = torch.fft.rfft(x_raw, dim=-1).abs()
        amp_nz = amp[:, 1:]
        if amp_nz.numel() == 0:
            return [series_len], torch.ones(x_raw.size(0), 1, device=x_raw.device,
dtype=x_raw.dtype)
        amp_mean = amp_nz.mean(0)
        k = int(min(self.topk, amp_mean.numel()))

```

```

        if k <= 0:
            return [series_len], torch.ones(x_raw.size(0), 1, device=x_raw.device,
dtype=x_raw.dtype)
        _, idx = torch.topk(amp_mean, k)
        freqs = idx + 1
        period_list = [max(1, int(round(series_len / int(freq.item())))) for freq in freqs]
        weights = torch.softmax(amp_nz[:, idx].to(x_raw.device, dtype=x_raw.dtype), dim=-1)
        return period_list, weights

def forward(self, x: torch.Tensor) -> torch.Tensor:
    period_list, weights = self._periods(x[:, 0, :])
    states = self.in_proj(x)
    for block in self.blocks:
        states = block.forward_with_periods(states, period_list=period_list, weights=weights)
    states = self.out_proj(states)
    out = self.head(states.squeeze(1))
    if not self.use_anchor:
        return out
    anchor = x[:, :, -1].squeeze(1).unsqueeze(1)
    return out + anchor

__all__ = ["TimesNetV2"]

```

Листинг 2. Архитектура нейросетевой модели TimesNet.

```

from __future__ import annotations

from contextlib import nullcontext
from typing import Iterable, Tuple

import numpy as np

try:
    from statsmodels.tsa.arima.model import ARIMA
    from statsmodels.tsa.holtwinters import ExponentialSmoothing
except Exception:
    ARIMA = None
    ExponentialSmoothing = None

try:
    from statsmodels.tsa.stattools import acf, pacf
except Exception:
    acf = None
    pacf = None

try:
    import pandas as pd
    from prophet import Prophet

```

```

try:
    from cmdstanpy.utils import disable_logging as _disable_cmdstanpy_logging
except Exception:
    _disable_cmdstanpy_logging = None
except Exception:
    pd = None
    Prophet = None
    _disable_cmdstanpy_logging = None

def _repeat_last(y: np.ndarray, horizon: int) -> np.ndarray:
    if horizon <= 0:
        raise ValueError("horizon must be positive")
    if y.size == 0:
        return np.zeros(horizon, dtype=float)
    return np.repeat(float(y[-1]), horizon).astype(float)

def _clean_series(y: Iterable[float]) -> np.ndarray:
    data = np.asarray(list(y), dtype=float)
    if data.size == 0:
        return data
    mask = np.isfinite(data)
    if not np.all(mask):
        data = data[mask]
    return data.astype(float, copy=False)

def _linear_extrapolation(y: np.ndarray, horizon: int, window: int = 32) -> np.ndarray:
    if horizon <= 0:
        raise ValueError("horizon must be positive")
    if y.size == 0:
        return np.zeros(horizon, dtype=float)
    n = int(y.size)
    w = int(min(max(2, window), n))
    t = np.arange(w, dtype=float)
    y_w = y[-w:].astype(float, copy=False)
    y_mean = float(np.mean(y_w))
    y_centered = y_w - y_mean
    slope, intercept = np.polyfit(t, y_centered, deg=1)
    future_t = np.arange(w, w + horizon, dtype=float)
    return (slope * future_t + intercept + y_mean).astype(float)

def _arima_trend_for_d(d: int) -> str:
    return "t" if int(d) > 0 else "c"

def _fit_forecast_arima(data: np.ndarray, horizon: int, order: Tuple[int, int, int]) -> np.ndarray:
    if ARIMA is None:
        raise RuntimeError("statsmodels не установлен, ARIMA недоступна")

```

```

p, d, q = (int(order[0]), int(order[1]), int(order[2]))
trend = _arima_trend_for_d(d)
model = ARIMA(
    data,
    order=(p, d, q),
    trend=trend,
    enforce_stationarity=False,
    enforce_invertibility=False,
)
fitted = model.fit(method_kwargs={"warn_convergence": False, "maxiter": 200})
forecast = fitted.forecast(steps=horizon)
return np.asarray(forecast, dtype=float)

def _auto_arima_order(
    data: np.ndarray,
    d: int = 1,
    max_lag: int | None = None,
    max_p: int = 3,
    max_q: int = 3,
) -> Tuple[int, int, int]:
    n = int(data.size)
    if n < 10 or acf is None or pacf is None: # type: ignore[truthy-function]
        return (1, d, 0)
    max_lag = int(max_lag or min(24, n // 2))
    max_lag = max(1, max_lag)
    try:
        acf_vals = acf(data, nlags=max_lag, fft=False)
        pacf_vals = pacf(data, nlags=max_lag, method="ywunbiased")
        crit = 1.96 / np.sqrt(n)
        p_candidates = [lag for lag in range(1, max_lag + 1) if abs(pacf_vals[lag]) > crit]
        q_candidates = [lag for lag in range(1, max_lag + 1) if abs(acf_vals[lag]) > crit]
        p = min(max(p_candidates) if p_candidates else 0, max_p)
        q = min(max(q_candidates) if q_candidates else 0, max_q)
        return (int(p), int(d), int(q))
    except Exception:
        return (1, d, 0)

def arima_forecast(
    y: Iterable[float],
    horizon: int,
    order: Tuple[int, int, int] = (1, 1, 1),
) -> np.ndarray:
    if ARIMA is None:
        raise RuntimeError("statsmodels не установлен, ARIMA недоступна")

    data = _clean_series(y)
    if data.size < 6:

```

```

        return _linear_extrapolation(data, horizon)
    try:
        forecast = _fit_forecast_arma(data, horizon, order=order)
        if np.nanstd(forecast) < 1e-12 and np.nanstd(np.diff(data[-min(16, data.size) :])) > 1e-12:
            return _linear_extrapolation(data, horizon)
        return forecast
    except Exception:
        for alt in ((0, 1, 1), (1, 1, 0), (0, 1, 0), (1, 0, 1)):
            try:
                return _fit_forecast_arma(data, horizon, order=alt)
            except Exception:
                continue
        return _linear_extrapolation(data, horizon)

def ets_forecast(
    y: Iterable[float],
    horizon: int,
    seasonal_periods: int | None = None,
    trend: str | None = "add",
    seasonal: str | None = "add",
) -> np.ndarray:
    if ExponentialSmoothing is None:
        raise RuntimeError("statsmodels не установлен, ETS недоступна")
    data = np.asarray(list(y), dtype=float)
    if data.size < 4:
        return _repeat_last(data, horizon)
    seasonal_periods = seasonal_periods if seasonal_periods and seasonal_periods > 1 else
    None
    seasonal = seasonal if seasonal_periods else None
    try:
        model = ExponentialSmoothing(
            data,
            trend=trend,
            seasonal=seasonal,
            seasonal_periods=seasonal_periods,
        )
        fitted = model.fit(optimized=True, use_brute=True)
        forecast = fitted.forecast(horizon)
        return np.asarray(forecast, dtype=float)
    except Exception:
        return _repeat_last(data, horizon)

def auto_arma_forecast(
    y: Iterable[float],
    horizon: int,
) -> np.ndarray:
    if ARIMA is None:
        raise RuntimeError("statsmodels не установлен, ARIMA недоступна")

```

```

data = _clean_series(y)
if data.size < 6:
    return _linear_extrapolation(data, horizon)
try:
    p, d, q = _auto_arma_order(data)
    forecast = _fit_forecast_arma(data, horizon, order=(p, d, q))
    if np.nanstd(forecast) < 1e-12 and np.nanstd(np.diff(data[-min(16, data.size) :])) > 1e-12:
        return _linear_extrapolation(data, horizon)
    return forecast
except Exception:
    best_aic = np.inf
    best_order: Tuple[int, int, int] | None = None
    for d in (0, 1):
        for p in range(0, 3):
            for q in range(0, 3):
                if p == 0 and q == 0 and d == 0:
                    continue
                try:
                    model = ARIMA(
                        data,
                        order=(p, d, q),
                        trend=_arma_trend_for_d(d),
                        enforce_stationarity=False,
                        enforce_invertibility=False,
                    )
                    fitted = model.fit(method_kwargs={"warn_convergence": False, "maxiter":
100})

                    aic = float(getattr(fitted, "aic", np.inf))
                    if np.isfinite(aic) and aic < best_aic:
                        best_aic = aic
                        best_order = (p, d, q)
                except Exception:
                    continue
    if best_order is not None:
        try:
            return _fit_forecast_arma(data, horizon, order=best_order)
        except Exception:
            pass
    return _linear_extrapolation(data, horizon)

def prophet_forecast(
    y: Iterable[float],
    horizon: int,
    freq: str = "D",
) -> np.ndarray:
    if Prophet is None or pd is None:
        raise RuntimeError("prophet не установлен, Prophet недоступен")
    data = np.asarray(list(y), dtype=float)
    if data.size < 2:

```



```

    return _repeat_last(data, horizon)
context = (
    _disable_cmdstanpy_logging()
    if _disable_cmdstanpy_logging is not None
    else nullcontext()
)
try:
    start = pd.Timestamp("2000-01-01")
    idx = pd.date_range(start=start, periods=data.size, freq=freq)
    df = pd.DataFrame({"ds": idx, "y": data})
    m = Prophet(
        seasonality_mode="additive",
        yearly_seasonality=False,
        weekly_seasonality=False,
        daily_seasonality=False,
    )
    with context:
        m.fit(df)
        future = m.make_future_dataframe(periods=horizon, freq=freq, include_history=False)
        forecast = m.predict(future)["yhat"].to_numpy()
    return np.asarray(forecast, dtype=float)
except Exception:
    return _repeat_last(data, horizon)

__all__ = ["arima_forecast", "ets_forecast", "auto_arima_forecast", "prophet_forecast"]

```

Листинг 3. Архитектура классических статистических моделей.

```

from __future__ import annotations

import numpy as np
import pywt
from scipy.ndimage import convolve1d

def _mode(mode: str) -> str:
    mode = str(mode).lower()
    if mode not in {"reflect", "constant", "nearest", "mirror", "wrap"}:
        raise ValueError(f"Unsupported boundary mode '{mode}'")
    return mode

def upArrow_op(li, j):
    if j == 0:
        return [1]
    N = len(li)
    li_n = np.zeros(2 ** (j - 1) * (N - 1) + 1)
    for i in range(N):

```

```

    li_n[2 ** (j - 1) * i] = li[i]
    return li_n

def period_list(li, N):
    n = len(li)
    n_app = N - np.mod(n, N)
    li = list(li)
    li = li + [0] * n_app
    if len(li) < 2 * N:
        return np.array(li)
    li = np.array(li)
    li = np.reshape(li, [-1, N])
    li = np.sum(li, axis=0)
    return li

def circular_convolve_mra(h_j_o, w_j, *, mode: str = "wrap"):
    mode = _mode(mode)
    return convolve1d(
        w_j,
        np.flip(h_j_o),
        mode=mode,
        origin=(len(h_j_o) - 1) // 2,
    )

def circular_convolve_d(h_t, v_j_1, j, *, mode: str = "wrap"):
    mode = _mode(mode)
    N = len(v_j_1)
    ker = np.zeros(len(h_t) * 2 ** (j - 1))
    for i, h in enumerate(h_t):
        ker[i * 2 ** (j - 1)] = h
    return convolve1d(v_j_1, ker, mode=mode, origin=-len(ker) // 2)

def circular_convolve_s(h_t, g_t, w_j, v_j, j, *, mode: str = "wrap"):
    mode = _mode(mode)
    N = len(v_j)
    h_ker = np.zeros(len(h_t) * 2 ** (j - 1))
    g_ker = np.zeros(len(g_t) * 2 ** (j - 1))
    for i, (h, g) in enumerate(zip(h_t, g_t)):
        h_ker[i * 2 ** (j - 1)] = h
        g_ker[i * 2 ** (j - 1)] = g
    v_j_1 = convolve1d(
        w_j,
        np.flip(h_ker),
        mode=mode,
        origin=(len(h_ker) - 1) // 2,
    )
    v_j_1 += convolve1d(

```

```

        v_j,
        np.flip(g_ker),
        mode=mode,
        origin=(len(g_ker) - 1) // 2,
    )
    return v_j_1

def modwt(x, filters, level, *, mode: str = "wrap"):
    mode = _mode(mode)
    wavelet = pywt.Wavelet(filters)
    h = wavelet.dec_hi
    g = wavelet.dec_lo
    h_t = np.array(h) / np.sqrt(2)
    g_t = np.array(g) / np.sqrt(2)
    wavecoeff = []
    v_j_1 = x
    for j in range(level):
        w = circular_convolve_d(h_t, v_j_1, j + 1, mode=mode)
        v_j_1 = circular_convolve_d(g_t, v_j_1, j + 1, mode=mode)
        wavecoeff.append(w)
    wavecoeff.append(v_j_1)
    return np.vstack(wavecoeff)

def imodwt(w, filters, *, mode: str = "wrap"):
    mode = _mode(mode)
    wavelet = pywt.Wavelet(filters)
    h = wavelet.dec_hi
    g = wavelet.dec_lo
    h_t = np.array(h) / np.sqrt(2)
    g_t = np.array(g) / np.sqrt(2)
    level = len(w) - 1
    v_j = w[-1]
    for jp in range(level):
        j = level - jp - 1
        v_j = circular_convolve_s(h_t, g_t, w[j], v_j, j + 1, mode=mode)
    return v_j

def modwtmra(w, filters, *, mode: str = "wrap"):
    mode = _mode(mode)
    wavelet = pywt.Wavelet(filters)
    h = wavelet.dec_hi
    g = wavelet.dec_lo
    level, N = w.shape
    level = level - 1
    D = []
    g_j_part = [1]
    for j in range(level):
        g_j_up = upArrow_op(g, j)

```

```

    g_j_part = np.convolve(g_j_part, g_j_up)
    h_j_up = upArrow_op(h, j + 1)
    h_j = np.convolve(g_j_part, h_j_up)
    h_j_t = h_j / (2 ** ((j + 1) / 2.0))
    if j == 0:
        h_j_t = h / np.sqrt(2)
    h_j_t_o = period_list(h_j_t, N)
    D.append(circular_convolve_mra(h_j_t_o, w[j], mode=mode))
j = level - 1
g_j_up = upArrow_op(g, j + 1)
g_j = np.convolve(g_j_part, g_j_up)
g_j_t = g_j / (2 ** ((j + 1) / 2.0))
g_j_t_o = period_list(g_j_t, N)
S = circular_convolve_mra(g_j_t_o, w[-1], mode=mode)
D.append(S)
return np.vstack(D)

__all__ = [
    "modwt",
    "imodwt",
    "modwtmra",
    "circular_convolve_d",
    "circular_convolve_mra",
    "circular_convolve_s",
]

```

Листинг 4. Логика вейвлет-разложения MODWT.

```

from __future__ import annotations

from dataclasses import dataclass
from typing import List, Optional, Sequence, Tuple

import numpy as np
import torch
from torch.utils.data import TensorDataset

from ..training import TrainConfig, WindowDatasetStd, train_model
from ..models.classic import auto_arima_forecast, ets_forecast
from . import modwt as ext_modw

def _filter_len(wname: str) -> int:
    try:
        import pywt

        return len(pywt.Wavelet(wname).dec_lo)
    except Exception:

```

```

        return 8

def _jmax_for_len(n: int, L: int) -> int:
    if n <= 2:
        return 1
    J = 1
    while (2 ** (J - 1)) * (L - 1) < n:
        J += 1
    return max(1, J - 1)

def modwt_decompose(y, wavelet: str = "db4", level: int = 1, check: bool = True):
    return modwt_decompose_with_boundary(y, wavelet=wavelet, level=level,
    boundary="wrap", check=check)

def modwt_decompose_with_boundary(
    y,
    *,
    wavelet: str = "db4",
    level: int = 1,
    boundary: str = "wrap",
    check: bool = True,
):
    y = np.asarray(y, float)
    N = y.size
    if N < 2:
        return y, []
    L = _filter_len(wavelet)
    Jmax = _jmax_for_len(N, L)
    J = int(max(1, min(level, Jmax)))

    boundary = str(boundary).lower()
    if boundary not in {"wrap", "reflect", "mirror", "nearest"}:
        raise ValueError(f"Unsupported boundary='{boundary}' (expected
wrap/reflect/mirror/nearest)")

def _pad(x: np.ndarray, pad_len: int) -> np.ndarray:
    if pad_len <= 0:
        return x
    if x.size < 2:
        return np.pad(x, (pad_len, pad_len), mode="edge")
    if boundary == "nearest":
        left = np.repeat(x[0], pad_len)
        right = np.repeat(x[-1], pad_len)
        return np.concatenate([left, x, right])
    if boundary == "mirror":
        left = x[:pad_len][::-1]
        right = x[-pad_len:][::-1]
        return np.concatenate([left, x, right])

```

```

left = x[1 : pad_len + 1][::-1] if x.size > 1 else np.repeat(x[0], pad_len)
right = x[-pad_len - 1 : -1][::-1] if x.size > 1 else np.repeat(x[-1], pad_len)
if left.size < pad_len:
    left = np.pad(left, (pad_len - left.size, 0), mode="edge")
if right.size < pad_len:
    right = np.pad(right, (0, pad_len - right.size), mode="edge")
return np.concatenate([left, x, right])

pad_len = 0
y_work = y

if boundary != "wrap" and N >= 3:
    pad_len = int(min(max(8, (2 ** J) * L), max(1, N - 1)))
    y_work = _pad(y, pad_len)

W = ext_modwt.modwt(y_work, wavelet, J, mode="wrap")
M = ext_modwt.modwtmra(W, wavelet, mode="wrap")
arr = (
    np.vstack([np.asarray(c, float) for c in M])
    if isinstance(M, (list, tuple))
    else np.asarray(M, float)
)
if pad_len > 0:
    arr = arr[:, pad_len : pad_len + N]
if check and (arr.ndim != 2 or arr.shape != (J + 1, N)):
    raise ValueError(f"modwtmra shape={arr.shape}, expected {(J + 1, N)}")
A = arr[-1]
D = [arr[j] for j in range(J - 1, -1, -1)]
return A, D

```

```

@dataclass
class HybridComponent:
    model: Optional[torch.nn.Module]
    mu: float
    sd: float
    lookback: Optional[int]
    per_series_scaling: bool = False

def build_global_hybrid_components(
    pairs: Sequence[Tuple[str, np.ndarray, np.ndarray]],
    cfg: TrainConfig,
    base_model_fn,
    wavelet: str = "db4",
    level: int = 1,
    boundary: str = "wrap",
) -> List[HybridComponent]:
    if not pairs:

```

```

return []

comp_values: List[List[np.ndarray]] = []
for _, y_tr, _ in pairs:
    y = np.asarray(y_tr, float)
    if y.size < 2:
        continue
    A, D = modwt_decompose_with_boundary(
        y, wavelet=wavelet, level=level, boundary=boundary, check=True
    )
    comps = [A] + D if len(D) else [A]
    if not comp_values:
        comp_values = [[] for _ in range(len(comps))]
    for idx, comp in enumerate(comps):
        if idx >= len(comp_values):
            comp_values.append([])
        comp_values[idx].append(np.asarray(comp, float))

components: List[HybridComponent] = []
H = cfg.horizon
L = cfg.lookback
for comp_series in comp_values:
    if not comp_series:
        components.append(HybridComponent(None, 0.0, 1.0, None,
per_series_scaling=True))
        continue
    X_list: List[np.ndarray] = []
    Y_list: List[np.ndarray] = []
    for arr in comp_series:
        arr_f = arr.astype(np.float32)
        mu = float(arr_f.mean())
        sd = float(arr_f.std() + 1e-8)
        z = (arr_f - mu) / sd
        if z.size < L + H:
            continue
        for t in range(0, len(z) - L - H + 1):
            X_list.append(z[t : t + L])
            Y_list.append(z[t + L : t + L + H])
    if not X_list:
        components.append(HybridComponent(None, 0.0, 1.0, None,
per_series_scaling=True))
        continue
    X = np.stack(X_list, axis=0).astype(np.float32)
    Y = np.stack(Y_list, axis=0).astype(np.float32)
    ds = TensorDataset(
        torch.from_numpy(X).unsqueeze(1),
        torch.from_numpy(Y),
    )
    model = base_model_fn(cfg)
    trained = train_model(model, ds, cfg)

```

```

        components.append(HybridComponent(trained, 0.0, 1.0, L, per_series_scaling=True))
    return components

class HybridPlus:
    def __init__(
        self,
        base_model_fn,
        cfg: TrainConfig,
        wavelet: str = "db4",
        level: int = 1,
        boundary: str = "wrap",
        pretrained_components: Optional[List[HybridComponent]] = None,
        seasonal_period: int | None = None,
    ):
        self.base_model_fn = base_model_fn
        self.cfg = cfg
        self.wavelet = wavelet
        self.level = level
        self.boundary = str(boundary).lower()
        self.components: List[HybridComponent] = []
        self.pretrained_components = pretrained_components
        self.seasonal_period = int(seasonal_period) if seasonal_period is not None else None

    def _prepare_component(self, comp: np.ndarray) -> HybridComponent:
        Lmax = len(comp) - self.cfg.horizon
        if Lmax < 16:
            return HybridComponent(None, float(comp.mean()), float(comp.std() + 1e-8), None,
per_series_scaling=False)
        L = min(self.cfg.lookback, Lmax)
        ds = WindowDatasetStd(comp, L, self.cfg.horizon, stride=1, scale=True)
        mu, sd = ds.scaler
        model = self.base_model_fn(self.cfg)
        trained = train_model(model, ds, self.cfg)
        return HybridComponent(trained, mu, sd, L, per_series_scaling=False)

    def fit(self, y, *, components_override: Optional[List[np.ndarray]] = None):
        y = np.asarray(y, float)
        if components_override is None:
            A, D = modwt_decompose_with_boundary(
                y, wavelet=self.wavelet, level=self.level, boundary=self.boundary, check=True
            )
            comps = [A] + D if len(D) else [A]
        else:
            comps = [np.asarray(c, float) for c in components_override]
            if not comps:
                comps = [y]
        if self.pretrained_components is not None and len(self.pretrained_components) ==
len(comps):
            # Reuse globally trained components; no per-series training.

```



```

        self.components = self.pretrained_components
    else:
        self.components = [self._prepare_component(comp) for comp in comps]
    return self

def forecast(self, y, *, components_override: Optional[List[np.ndarray]] = None):
    y = np.asarray(y, float)
    H = self.cfg.horizon
    if components_override is None:
        A, D = modwt_decompose_with_boundary(
            y, wavelet=self.wavelet, level=self.level, boundary=self.boundary, check=True
        )
        comps = [A] + D if len(D) else [A]
    else:
        comps = [np.asarray(c, float) for c in components_override]
        if not comps:
            comps = [y]
        A = comps[0]
    if not self.components:
        return np.repeat(y[-1], H).astype(float)

    comp_preds: List[np.ndarray] = []
    for component, comp in zip(self.components, comps):
        if component.model is None or component.lookback is None:
            comp_preds.append(np.repeat(float(comp[-1]) if comp.size else 0.0,
H).astype(float))
            continue
        if component.per_series_scaling:
            mu = float(np.mean(comp))
            sd = float(np.std(comp) + 1e-8)
        else:
            mu = float(component.mu)
            sd = float(component.sd)
        xb = ((comp[-component.lookback :] - mu) / sd).astype(np.float32)
        xb = xb.reshape(1, 1, -1)
        with torch.no_grad():
            forecast = component.model(torch.from_numpy(xb).to(self.cfg.device)).cpu().numpy().ravel()
            pred = forecast * sd + mu
            if pred.size > H:
                pred = pred[:H]
            if pred.size < H:
                pred = np.pad(pred, (0, H - pred.size), mode="edge")
            comp_preds.append(pred.astype(float, copy=False))

    if len(comp_preds) >= 2 and A.size > 0 and y.size > 0:
        a_last = float(A[-1])
        resid_last = float(y[-1] - a_last)
        resid_pred = np.sum(np.stack(comp_preds[1:], 0), axis=0)
        k = int(max(1, min(6, H)))

```

```

    if k == 1:
        resid_pred = resid_pred.astype(float, copy=True)
        resid_pred[0] = 0.6 * resid_last + 0.4 * float(resid_pred[0])
    else:
        resid_pred = resid_pred.astype(float, copy=True)
        alpha0 = 0.6
        for i in range(k):
            alpha = float(alpha0 * (k - 1 - i) / (k - 1))
            resid_pred[i] = alpha * resid_last + (1.0 - alpha) * resid_pred[i]
        yhat = comp_preds[0] + resid_pred
    else:
        yhat = np.sum(np.stack(comp_preds, 0), axis=0)
    if yhat.size > H:
        yhat = yhat[:H]
    if yhat.size < H:
        yhat = np.pad(yhat, (0, H - yhat.size), mode="edge")
    return _stabilize_reconstruction_boundary(y, yhat)

def forecast_components(self, y, *, components_override: Optional[List[np.ndarray]] =
None) -> dict[str, np.ndarray]:
    """Return per-component forecasts used for reconstruction.

    Keys: "A_J", "D_1", ..., "D_J".
    """
    y = np.asarray(y, float)
    H = self.cfg.horizon
    if components_override is None:
        A, D = modwt_decompose_with_boundary(
            y, wavelet=self.wavelet, level=self.level, boundary=self.boundary, check=True
        )
        comps = [A] + D if len(D) else [A]
    else:
        comps = [np.asarray(c, float) for c in components_override]
        if not comps:
            comps = [y]
        A = comps[0]
    if not self.components:
        return {"A_J": np.repeat(float(A[-1]) if A.size else 0.0, H).astype(float)}

    comp_preds: List[np.ndarray] = []
    for component, comp in zip(self.components, comps):
        if component.model is None or component.lookback is None:
            comp_preds.append(np.repeat(float(comp[-1]) if comp.size else 0.0,
H).astype(float))
            continue
        if component.per_series_scaling:
            mu = float(np.mean(comp))
            sd = float(np.std(comp) + 1e-8)
        else:
            mu = float(component.mu)

```

```

        sd = float(component.sd)
        xb = ((comp[-component.lookback :] - mu) / sd).astype(np.float32)
        xb = xb.reshape(1, 1, -1)
        with torch.no_grad():
            forecast =
component.model(torch.from_numpy(xb).to(self.cfg.device)).cpu().numpy().ravel()
        pred = forecast * sd + mu
        if pred.size > H:
            pred = pred[:H]
        if pred.size < H:
            pred = np.pad(pred, (0, H - pred.size), mode="edge")
        comp_preds.append(pred.astype(float, copy=False))

if len(comp_preds) >= 2 and A.size > 0 and y.size > 0:
    a_last = float(A[-1])
    resid_last = float(y[-1] - a_last)
    resid_pred = np.sum(np.stack(comp_preds[1:], 0), axis=0).astype(float, copy=True)
    k = int(max(1, min(6, H)))
    if k == 1:
        resid_pred[0] = 0.6 * resid_last + 0.4 * float(resid_pred[0])
    else:
        alpha0 = 0.6
        for i in range(k):
            alpha = float(alpha0 * (k - 1 - i) / (k - 1))
            resid_pred[i] = alpha * resid_last + (1.0 - alpha) * resid_pred[i]

    raw = np.sum(np.stack(comp_preds[1:], 0), axis=0)
    diff = resid_pred - raw
    details = [p.astype(float, copy=True) for p in comp_preds[1:]]
    weights = np.stack([np.abs(p) for p in details], 0)
    denom = np.sum(weights, axis=0)
    if np.all(denom < 1e-8):
        for j in range(len(details)):
            details[j] = details[j] + diff / float(len(details))
    else:
        for j in range(len(details)):
            details[j] = details[j] + diff * (weights[j] / (denom + 1e-8))
    comp_preds = [comp_preds[0]] + details

out: dict[str, np.ndarray] = {"A_J": np.asarray(comp_preds[0], float)}
for j, pred in enumerate(comp_preds[1:], start=1):
    out[f"D_{j}"] = np.asarray(pred, float)
return out

class VWHybridMixed:
    def __init__(
        self,
        *,
        aj_model_fn,

```

```

detail_method: str,
cfg: TrainConfig,
wavelet: str = "db4",
level: int = 1,
seasonal_period: int | None = None,
boundary: str = "reflect",
detail_anchor: bool = True,
detail_transition_steps: int = 3,
detail_dampen: bool = True,
detail_dampen_tau: float = 3.0,
residual_anchor: bool = True,
residual_transition_steps: int = 6,
) -> None:
    self.aj_model_fn = aj_model_fn
    self.detail_method = str(detail_method).lower()
    self.cfg = cfg
    self.wavelet = wavelet
    self.level = int(level)
    self.seasonal_period = int(seasonal_period) if seasonal_period is not None else None
    self.boundary = str(boundary).lower()
    self.detail_anchor = bool(detail_anchor)
    self.detail_transition_steps = int(detail_transition_steps)
    self.detail_dampen = bool(detail_dampen)
    self.detail_dampen_tau = float(detail_dampen_tau)
    self.residual_anchor = bool(residual_anchor)
    self.residual_transition_steps = int(residual_transition_steps)
    self.aj_component: HybridComponent | None = None

def _prepare_component(self, comp: np.ndarray) -> HybridComponent:
    Lmax = len(comp) - self.cfg.horizon
    if Lmax < 16:
        return HybridComponent(None, float(comp.mean()), float(comp.std() + 1e-8), None)
    L = min(self.cfg.lookback, Lmax)
    ds = WindowDatasetStd(comp, L, self.cfg.horizon, stride=1, scale=True)
    mu, sd = ds.scaler
    model = self.aj_model_fn(self.cfg)
    trained = train_model(model, ds, self.cfg)
    return HybridComponent(trained, mu, sd, L)

def fit(self, y, *, components_override: Optional[List[np.ndarray]] = None):
    y = np.asarray(y, float)
    if components_override is None:
        A, _ = modwt_decompose_with_boundary(
            y, wavelet=self.wavelet, level=self.level, boundary=self.boundary, check=True
        )
    else:
        comps = [np.asarray(c, float) for c in components_override]
        A = comps[0] if comps else y
    self.aj_component = self._prepare_component(A)
    return self

```

```

def _forecast_neural(self, component: HybridComponent, comp: np.ndarray) -> np.ndarray:
    H = self.cfg.horizon
    if component.model is None or component.lookback is None:
        return np.repeat(float(comp[-1]) if comp.size else 0.0, H).astype(float)
    xb = ((comp[-component.lookback:] - component.mu) /
component.sd).astype(np.float32)
    xb = xb.reshape(1, 1, -1)
    with torch.no_grad():
        forecast = (
            component.model(torch.from_numpy(xb).to(self.cfg.device)).cpu().numpy().ravel()
        )
    pred = forecast * component.sd + component.mu
    if pred.size > H:
        pred = pred[:H]
    if pred.size < H:
        pred = np.pad(pred, (0, H - pred.size), mode="edge")
    return pred.astype(float, copy=False)

def _forecast_detail(self, comp: np.ndarray) -> np.ndarray:
    H = self.cfg.horizon
    comp = np.asarray(comp, float)
    if comp.size == 0:
        return np.zeros(H, dtype=float)

    if self.detail_method in {"ets", "holtwinters"}:
        seasonal_periods = self.seasonal_period if self.seasonal_period and
self.seasonal_period > 1 else None
        seasonal = "add" if seasonal_periods else None
        pred = ets_forecast(comp, H, seasonal_periods=seasonal_periods, trend=None,
seasonal=seasonal)
    elif self.detail_method in {"arima_auto", "auto_arima"}:
        pred = auto_arima_forecast(comp, H)
    else:
        raise ValueError(
            f"Unknown detail_method='{self.detail_method}' (expected 'ets' or 'arima_auto')"
        )

    pred = np.asarray(pred, float)
    if pred.size > H:
        pred = pred[:H]
    if pred.size < H:
        pred = np.pad(pred, (0, H - pred.size), mode="edge")

    out = pred.astype(float, copy=True)

    if self.detail_dampen and H > 0:
        tau = max(0.5, float(self.detail_dampen_tau))
        w = np.exp(-np.arange(H, dtype=float) / tau)
        out = out * w

```

```

if not self.detail_anchor or comp.size < 2 or H <= 0:
    return out.astype(float, copy=False)

last = float(comp[-1])
k = int(max(1, min(self.detail_transition_steps, H)))
for i in range(k):
    alpha = float((k - 1 - i) / max(1, k - 1)) if k > 1 else 1.0
    out[i] = alpha * last + (1.0 - alpha) * out[i]
return out

def forecast(self, y, *, components_override: Optional[List[np.ndarray]] = None):
    y = np.asarray(y, float)
    H = self.cfg.horizon
    if components_override is None:
        A, D = modwt_decompose_with_boundary(
            y, wavelet=self.wavelet, level=self.level, boundary=self.boundary, check=True
        )
    else:
        comps = [np.asarray(c, float) for c in components_override]
        A = comps[0] if comps else y
        D = comps[1:] if len(comps) > 1 else []
    if self.aj_component is None:
        raise RuntimeError("Call fit() before forecast()")

    aj_pred = self._forecast_neural(self.aj_component, A)
    detail_preds: List[np.ndarray] = []
    for dj in D:
        pred = self._forecast_detail(np.asarray(dj, float))
        if pred.size > H:
            pred = pred[:H]
        if pred.size < H:
            pred = np.pad(pred, (0, H - pred.size), mode="edge")
        detail_preds.append(pred.astype(float, copy=False))

    if detail_preds:
        resid_pred = np.sum(np.stack(detail_preds, 0), axis=0)
    else:
        resid_pred = np.zeros(H, dtype=float)

    if self.residual_anchor and y.size > 0 and A.size > 0 and H > 0:
        resid_last = float(y[-1] - A[-1])
        k = int(max(1, min(self.residual_transition_steps, H)))
        if k == 1:
            resid_pred = resid_pred.astype(float, copy=True)
            resid_pred[0] = 0.6 * resid_last + 0.4 * float(resid_pred[0])
        else:
            resid_pred = resid_pred.astype(float, copy=True)
            alpha0 = 0.6
            for i in range(k):

```

```

        alpha = float(alpha0 * (k - 1 - i) / (k - 1))
        resid_pred[i] = alpha * resid_last + (1.0 - alpha) * resid_pred[i]

    yhat = aj_pred + resid_pred
    if yhat.size > H:
        yhat = yhat[:H]
    if yhat.size < H:
        yhat = np.pad(yhat, (0, H - yhat.size), mode="edge")
    return _stabilize_reconstruction_boundary(y, yhat)

def forecast_components(self, y, *, components_override: Optional[List[np.ndarray]] =
None) -> dict[str, np.ndarray]:

    y = np.asarray(y, float)
    H = self.cfg.horizon
    if components_override is None:
        A, D = modwt_decompose_with_boundary(
            y, wavelet=self.wavelet, level=self.level, boundary=self.boundary, check=True
        )
    else:
        comps = [np.asarray(c, float) for c in components_override]
        A = comps[0] if comps else y
        D = comps[1:] if len(comps) > 1 else []
    if self.aj_component is None:
        raise RuntimeError("Call fit() before forecast_components()")

    aj_pred = self._forecast_neural(self.aj_component, A)
    detail_preds: List[np.ndarray] = []
    for dj in D:
        pred = self._forecast_detail(np.asarray(dj, float))
        if pred.size > H:
            pred = pred[:H]
        if pred.size < H:
            pred = np.pad(pred, (0, H - pred.size), mode="edge")
        detail_preds.append(pred.astype(float, copy=False))

    if detail_preds:
        resid_pred = np.sum(np.stack(detail_preds, 0), axis=0).astype(float, copy=True)
    else:
        resid_pred = np.zeros(H, dtype=float)

    if self.residual_anchor and y.size > 0 and A.size > 0 and H > 0:
        resid_last = float(y[-1] - A[-1])
        k = int(max(1, min(self.residual_transition_steps, H)))
        if k == 1:
            resid_pred[0] = 0.6 * resid_last + 0.4 * float(resid_pred[0])
        else:
            alpha0 = 0.6
            for i in range(k):
                alpha = float(alpha0 * (k - 1 - i) / (k - 1))

```

```

        resid_pred[i] = alpha * resid_last + (1.0 - alpha) * resid_pred[i]

    if detail_preds:
        raw = np.sum(np.stack(detail_preds, 0), axis=0)
        diff = resid_pred - raw
        details = [p.astype(float, copy=True) for p in detail_preds]
        weights = np.stack([np.abs(p) for p in details], 0)
        denom = np.sum(weights, axis=0)
        if np.all(denom < 1e-8):
            for j in range(len(details)):
                details[j] = details[j] + diff / float(len(details))
        else:
            for j in range(len(details)):
                details[j] = details[j] + diff * (weights[j] / (denom + 1e-8))
    else:
        details = []

    out: dict[str, np.ndarray] = {"A_J": np.asarray(aj_pred, float)}
    for j, pred in enumerate(details, start=1):
        out[f"D_{j}"] = np.asarray(pred, float)
    return out

def _stabilize_reconstruction_boundary(y: np.ndarray, yhat: np.ndarray) -> np.ndarray:
    if yhat.size == 0 or y.size < 3:
        return yhat.astype(float, copy=False)
    k = int(min(24, y.size - 1))
    diffs = y[-k:] - y[-k - 1 : -1]
    abs_diffs = np.abs(diffs)
    scale = float(np.quantile(abs_diffs, 0.75) + 1e-8)
    max_jump = 4.0 * scale
    jump0 = float(yhat[0] - y[-1])
    last_diff = float(y[-1] - y[-2])
    bad_direction = (last_diff * jump0 < 0) and (abs(last_diff) > 0.5 * scale) and (abs(jump0) >
0.5 * scale)
    yhat2 = yhat.astype(float, copy=True)

    if abs(jump0) > max_jump or bad_direction:
        if bad_direction and abs(jump0) <= max_jump:
            jump0 = float(np.sign(last_diff) * abs(jump0))
            yhat2[0] = float(y[-1]) + float(np.clip(jump0, -max_jump, max_jump))

    if yhat2.size >= 3:
        pred_diffs = np.diff(yhat2[: min(8, yhat2.size)])
        pred_scale = float(np.quantile(np.abs(pred_diffs), 0.75) + 1e-8) if pred_diffs.size else
scale
        spike_scale = float(max(scale, pred_scale))
        d01 = float(yhat2[0] - yhat2[1])
        d12 = float(yhat2[1] - yhat2[2])
        if (abs(d01) > 6.0 * spike_scale) and (abs(d12) < 2.5 * spike_scale) and (abs(yhat2[0] -

```



```

y[-1]) > 4.0 * spike_scale):
    target = float(yhat2[1])
    yhat2[0] = float(y[-1]) + float(np.clip(target - float(y[-1]), -max_jump, max_jump))

    return yhat2

__all__ = [
    "HybridComponent",
    "HybridPlus",
    "VWHybridMixed",
    "build_global_hybrid_components",
    "modwt_decompose",
    "modwt_decompose_with_boundary",
]

```

Листинг 5. Архитектура HybridPlus (MODWT + NN), VWHybridMixed (MODWT + NN+statsmodels).